


# ระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้า

Electrical Status Notification System

โดย

ด.ช.กฤตภาส ใจใหญ่ เลขที่ 1 ม.3/1

ด.ช.กิตติศักดิ์ วงษา เลขที่ 4 ม.3/1

ด.ช.ปกรณ์เกียรติ แหวนแก้ว เลขที่ 15 ม.3/1

ด.ช.วรัญชัย พุทธวงค์ เลขที่ 19 ม.3/1

ด.ช.วิชญะ เสาไธสง เลขที่ 20 ม.3/1

ด.ช.อชิระ ลิ้มสุวรรณ เลขที่ 21 ม.3/1

โครงงานนี้เป็นส่วนหนึ่งของการศึกษา

วิชา I20201 การศึกษาค้นคว้าอิสระ

ภาคเรียนที่ 1 ปีการศึกษา 2569

โรงเรียนนครสวรรค์

อำเภอเมืองนครสวรรค์ จังหวัดนครสวรรค์

# กิตติกรรมประกาศ

โครงงานเรื่อง ระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้า ฉบับนี้สำเร็จลุล่วงได้ด้วยความกรุณาและความช่วยเหลืออย่างดียิ่งจากครูที่ปรึกษาโครงงาน ได้แก่ ครูศิรประภา ปิ่นเทศ ครูสุพรรณี อภิชัยเอนก และครูธันยาณัฐ รังสิยานนท์ ที่ได้ให้คำแนะนำ ข้อคิดเห็น และตรวจสอบแก้ไขข้อบกพร่องต่างๆ ตลอดระยะเวลาการดำเนินงาน ผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ ที่นี้

ขอขอบพระคุณโรงเรียนนครสวรรค์ที่ให้การสนับสนุนสถานที่และอุปกรณ์ในการทำโครงงาน ตลอดจนขอบคุณครอบครัวและเพื่อน ๆ ที่คอยให้กำลังใจและช่วยเหลือในด้านต่าง ๆ มาโดยตลอด คุณค่าและประโยชน์อันพึงมีจากโครงงานฉบับนี้ ผู้จัดทำขอมอบแด่ผู้มีพระคุณทุกท่านที่กล่าวมาข้างต้น

คณะผู้จัดทำ

# บทคัดย่อ

**ชื่อโครงงาน :** ระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้า

**ชื่อนักวิจัย :** 1. ด.ช.กฤตภาส ใจใหญ่ เลขที่ 1 ม.3/1

2. ด.ช.กิตติศักดิ์ วงษา เลขที่ 4 ม.3/1

3. ด.ช.ปกรณ์เกียรติ แหวนแก้ว เลขที่ 15 ม.3/1

4. ด.ช.วรัญชัย พุทธวงค์ เลขที่ 19 ม.3/1

5. ด.ช.วิชญะ เสาไธสง เลขที่ 20 ม.3/1

6. ด.ช.อชิระ ลิ้มสุวรรณ เลขที่ 21 ม.3/1

โครงงานเรื่องนี้มีวัตถุประสงค์เพื่อออกแบบและพัฒนาระบบแจ้งเตือนสถานะการเปิด-ปิดของอุปกรณ์ไฟฟ้ากระแสตรง (DC) แรงดัน 3–9 โวลต์ โดยใช้ไมโครคอนโทรลเลอร์ ESP32 ร่วมกับเซนเซอร์วัดกระแสไฟฟ้า INA219 สองตัว ตัดสินสถานะด้วยเกณฑ์กระแสแบบสองระดับ (Hysteresis) ส่งข้อมูลผ่านอินเทอร์เน็ตไปยัง Cloudflare Worker บันทึกในฐานข้อมูล Cloudflare D1 แสดงสถานะบนเว็บไซต์ที่เผยแพร่ผ่าน GitHub Pages และส่งข้อความสรุปการใช้งานประจำวันไปยังผู้ใช้ผ่าน LINE Messaging API เวลา 16:10 น. ทุกวัน

เครื่องมือที่ใช้ในการวิจัย ได้แก่ ด้านฮาร์ดแวร์ ประกอบด้วย ESP32 DevKit เซนเซอร์ INA219 และแผงวงจรขยายที่ออกแบบขึ้นเอง ด้านซอฟต์แวร์ ประกอบด้วย PlatformIO, LazyVim, KiCad, Cloudflare Workers และ D1, LINE Messaging API และ GitHub Pages วิเคราะห์ข้อมูลด้วยสถิติเชิงพรรณนา ได้แก่ ร้อยละ ค่าเฉลี่ย ค่าความคลาดเคลื่อนร้อยละ และส่วนเบี่ยงเบนมาตรฐาน

**ผลการวิจัยพบว่า**

1. ระบบทำงานครบตามขั้นตอน ตั้งแต่การวัดกระแส การส่งข้อมูล การแสดงผลบนเว็บไซต์และการสรุปผ่านไลน์

2. หากไม่ได้รับข้อมูลเกิน 30 วินาที เว็บไซต์จะแสดงสถานะ N/A แทนการแสดงว่าอุปกรณ์ปิด

3. ผลการทดสอบความแม่นยำ [เติมเมื่อวัดผลจริงครบ เช่น ร้อยละความถูกต้องของการตรวจจับ ค่าคลาดเคลื่อนของกระแส และความหน่วงเวลา]

**คำสำคัญ :** ระบบแจ้งเตือน ESP32 INA219 อุปกรณ์ไฟฟ้า Cloudflare

---

# Abstract

**Project Title** : Electrical Status Notification System

**Authors** : 1.Krittapas Jaiyai, Grade 9/1, No. 1

2.Kittisak Wongsa, Grade 9/1, No. 4

3.Pakorn-kiat Waenkaew, Grade 9/1, No. 15

4.Waranchai Phutthawong, Grade 9/1, No. 19

5.Wichaya Saothaisong, Grade 9/1, No. 20

6.Achira Limsuwan, Grade 9/1, No. 21

This project aims to design and develop a notification system that reports the on/off status of low-voltage DC appliances (3–9 V). The system uses an ESP32 microcontroller with two INA219 current sensors. The on/off state is decided by a two-threshold (hysteresis) current rule, sent over the Internet to a Cloudflare Worker, stored in a Cloudflare D1 database, displayed on a website hosted on GitHub Pages, and summarized to the user once a day at 16:10 (UTC+7) through the LINE Messaging API.

The hardware tools were an ESP32 DevKit, INA219 sensors and a self-designed expansion board. The software tools were PlatformIO, LazyVim, Cloudflare Workers and D1, the LINE Messaging API and GitHub Pages. Data were analyzed with descriptive statistics: percentage, mean, percentage error and standard deviation.

**The results showed that:**

1. The system works end to end, from sensing and transmission to display and the daily summary.

2. If no data are received for more than 30 seconds, the website shows N/A instead of reporting the appliance as off.

3. Test results: [to be completed after the measurements, e.g. detection accuracy, current measurement error and latency].

**Keywords** : notification system, ESP32, INA219, electrical appliances, Cloudflare

---

# สารบัญ

**หน้า**


กิตติกรรมประกาศ....1

บทคัดย่อภาษาไทย....2

บทคัดย่อภาษาอังกฤษ....4

สารบัญ....6

สารบัญตาราง....8

สารบัญภาพ....9

**บทที่ 1 บทนำ**....1

1. ความเป็นมาและความสำคัญของปัญหา....1

2. วัตถุประสงค์....2

3. สมมติฐาน....3

4. ขอบเขตของการศึกษาค้นคว้า....4

5. นิยามศัพท์....6

6. ประโยชน์ที่คาดว่าจะได้รับ....9

**บทที่ 2 เอกสาร / ทฤษฎีที่เกี่ยวข้อง**....10

1. ไมโครคอนโทรลเลอร์ ESP32....10

2. หลักการวัดกระแสไฟฟ้าด้วยเซนเซอร์ INA219 และการสื่อสารแบบ I2C....10

3. อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things: IoT)....14

4. Cloudflare Workers และ Cloudflare D1....15

5. LINE Messaging API....18

6. การตัดสินสถานะด้วยเกณฑ์สองระดับ (Hysteresis)....18

7. เว็บไซต์บน GitHub Pages....21

8. งานวิจัยที่เกี่ยวข้อง....21

9. กรอบแนวคิดการวิจัย....23

**บทที่ 3 วิธีการดำเนินงาน**.....25

1. ประชากรและกลุ่มตัวอย่าง.....26

2. เครื่องมือที่ใช้ในการวิจัย.....27

3. การเก็บรวบรวมข้อมูล.....34

4. การวิเคราะห์ข้อมูล.....39

5. สถิติที่ใช้ในการวิเคราะห์.....39

**บทที่ 4 ผลการวิจัย/ผลการวิเคราะห์ข้อมูล**.....41

1. ผลการพัฒนาระบบ.....41

2. ผลการหาเกณฑ์ ON/OFF (การทดลองที่ 1).....42

3. ผลความแม่นยำของกระแส (การทดลองที่ 2).....43

4. ผลความถูกต้องของการตรวจจับ (การทดลองที่ 3).....43

5. ผลความหน่วงเวลา (การทดลองที่ 4).....44

6. ผลการแสดง N/A และความเสถียร (การทดลองที่ 5).....45

7. ผลข้อความสรุปทาง LINE (การทดลองที่ 6).....45

8. ผลการทดสอบแผงวงจรขยายที่ออกแบบเอง (การทดลองที่ 7).....46

9. ผลการเปรียบเทียบ Hysteresis กับเกณฑ์เดียว (การทดลองที่ 8).....46

10. ผลการทดสอบการใช้งานหน้าเว็บไซต์ (การทดลองที่ 9).....47

11. ปัญหาที่พบและวิธีแก้ไข.....48

**บทที่ 5 สรุปผล อภิปรายผลและข้อเสนอแนะ**.....53

1. สรุปผลการวิจัย.....53

2. การอภิปรายผล.....54

3. ข้อเสนอแนะ.....55

**บรรณานุกรม**.....57

**ภาคผนวก ก เว็บไซต์และซอร์สโค้ด**.....59

**ภาคผนวก ข ขั้นตอนติดตั้งโดยสรุป**.....61

**ภาคผนวก ค แผงวงจรและรายการอุปกรณ์**.....63

**ภาคผนวก ง โค้ดสำคัญของระบบ**.....64

**ภาคผนวก จ แบบบันทึกและแบบสอบถามที่ใช้ในการทดลอง**.....70

**ประวัติผู้จัดทำ**.....72

# สารบัญตาราง

**หน้า**

**ตารางที่**


1 ผลการวัดกระแสช่วงปิดและช่วงเปิด และเกณฑ์ที่เลือก....42

2 เปรียบเทียบกระแสที่ INA219 วัดได้กับมัลติมิเตอร์....43

3 ความถูกต้องในการตรวจจับสถานะเปิด-ปิด....44

4 เวลาตั้งแต่สถานะจริงเปลี่ยนจนหน้าเว็บแสดงสถานะใหม่....44

5 ผลการทดสอบ N/A และความเสถียร....45

6 ผลการตรวจข้อความสรุปประจำวัน....46

7 ผลการทดสอบแผงวงจรขยายที่ออกแบบเอง....46

8 จำนวนครั้งที่สถานะเปลี่ยนผิดพลาดของเกณฑ์เดียวและเกณฑ์สองระดับ....47

9 ผลการทดสอบการใช้งานหน้าเว็บไซต์....47

10 สรุปผลการทดลองเทียบกับสมมติฐาน....50

# สารบัญภาพ

**หน้า**

**ภาพที่**


1 ตัวอย่างกราฟการใช้ไฟฟ้าแบบ Hysteresis....3

2 ตัวอย่างกราฟการใช้ไฟฟ้าของ servo....5

3 ตัวอย่างการส่งผ่าน Endpoint ด้วย GET/POST method ระหว่าง user/client - server....9

4 แผนภาพบล็อกอย่างง่ายของเซนเซอร์ INA219....11

5 การเชื่อมต่อ ESP32 กับ INA219 สองตัวบนบัส I2C....12

6 หลักการวัดกระแสไฟฟ้าด้วยตัวต้านทานชันต์....13

7 ลำดับการส่งข้อมูลและการใช้ Offline Buffer ของ ESP32....15

8 ลำดับการตรวจสอบคำขอ POST /update ของ Worker....17

9 เปรียบเทียบเกณฑ์เดียวกับ Hysteresis (ข้อมูลสมมติเพื่ออธิบาย ไม่ใช่ผลการทดลอง)....19

10 ภาพประกอบแนวคิด Hysteresis (ข้อมูลสมมติเพื่ออธิบาย ไม่ใช่ผลการทดลอง)....20

11 ขั้นตอนการตัดสินสถานะด้วย Hysteresis....21

12 แผนภาพการทำงานของระบบ....24

13 ขั้นตอนการดำเนินงานของโครงงาน....25

14 ผังวงจรการเชื่อมต่อ ESP32, INA219 และโหลด....30

15 โครงสร้างโมดูลของเฟิร์มแวร์ ESP32....32

16 ความสัมพันธ์ของตารางในฐานข้อมูล D1....33

17 การจัดวางอุปกรณ์ในการทดลองความแม่นยำของกระแส....36

18 ช่วงเวลาที่ใช้คำนวณความหน่วงของการแสดงสถานะ....37

19 เส้นทางของข้อมูลตั้งแต่การทดลองจนถึงการรายงานผล....51

# บทที่ 1
# บทนำ

## 1. ความเป็นมาและความสำคัญของปัญหา

ปัจจุบันอุปกรณ์ไฟฟ้าภายในบ้านเรือนและสถานที่ต่าง ๆ มีจำนวนมากขึ้นตามความก้าวหน้าของเทคโนโลยี จึงเกิดปัญหาที่พบบ่อยคือการลืมปิดอุปกรณ์ไฟฟ้าหลังใช้งาน ทำให้สิ้นเปลืองพลังงานและอาจเพิ่มความเสี่ยงต่อความปลอดภัย โดยเฉพาะเมื่อผู้ใช้ออกจากสถานที่นั้นๆ แล้วและไม่สามารถตรวจสอบได้ว่าอุปกรณ์ยังทำงานอยู่หรือไม่

คณะผู้จัดทำจึงมีแนวคิดพัฒนาระบบที่ตรวจสอบสถานะของอุปกรณ์จาก “กระแสไฟฟ้าที่ไหลจริง” ด้วยไมโครคอนโทรลเลอร์ ESP32 ร่วมกับเซนเซอร์วัดกระแส INA219 แล้วส่งข้อมูลผ่านอินเทอร์เน็ตให้ผู้ใช้ดูสถานะได้จากเว็บไซต์ และรับข้อความสรุปการใช้งานประจำวันทางแอปพลิเคชันไลน์ (LINE) เพื่อให้ผู้ใช้ทราบสถานะและตัดสินใจปิดอุปกรณ์ได้ทันเวลา ซึ่งนำไปสู่การประหยัดพลังงานและลดความเสี่ยงที่อาจเกิดขึ้น

การตรวจสอบสถานะการทำงานของอุปกรณ์ไฟฟ้าโดยทั่วไปอาจอาศัยการสังเกตจากผู้ใช้งานโดยตรง เช่น การตรวจสอบไฟแสดงสถานะ การสังเกตการหมุนของมอเตอร์ หรือการตรวจสอบสวิตช์เปิด-ปิด อย่างไรก็ตาม วิธีดังกล่าวจำเป็นต้องอยู่ใกล้กับอุปกรณ์ และไม่สามารถทราบสถานะการทำงานได้อย่างสะดวกเมื่อผู้ใช้งานอยู่ห่างจากอุปกรณ์ นอกจากนี้ สถานะของสวิตช์อาจไม่สอดคล้องกับการทำงานจริงของอุปกรณ์ในบางกรณี เช่น อุปกรณ์ได้รับคำสั่งเปิดแต่เกิดความผิดปกติจนไม่มีกระแสไฟฟ้าไหล

การตรวจวัดกระแสไฟฟ้าที่ไหลผ่านอุปกรณ์เป็นอีกแนวทางหนึ่งที่สามารถใช้ประเมินการทำงานของโหลดได้ เนื่องจากอุปกรณ์ไฟฟ้าที่กำลังทำงานมักมีการใช้กระแสไฟฟ้าตามลักษณะของโหลด ผู้จัดทำจึงเลือกใช้เซนเซอร์ INA219 ซึ่งสามารถตรวจวัดแรงดัน กระแส และกำลังไฟฟ้า แล้วส่งค่าที่วัดได้ไปยัง ESP32 ผ่านระบบสื่อสาร I2C เพื่อนำข้อมูลมาประมวลผลและจำแนกสถานะของอุปกรณ์ตามเกณฑ์ที่กำหนด

โครงงานนี้จึงมุ่งพัฒนาระบบต้นแบบที่รวมการตรวจวัด การประมวลผล การจัดเก็บข้อมูล และการแจ้งเตือนไว้ในระบบเดียว โดยไม่ได้จำกัดอยู่เพียงการแสดงค่ากระแสไฟฟ้าบนหน้าจอเท่านั้น แต่ยังให้ผู้ใช้สามารถตรวจสอบสถานะผ่านเว็บไซต์ได้จากระยะไกล และรับข้อความสรุปผ่านแอปพลิเคชัน LINE อีกทั้งยังศึกษาการใช้เกณฑ์ Hysteresis เพื่อช่วยลดการเปลี่ยนสถานะที่ไม่จำเป็นเมื่อค่ากระแสไฟฟ้าผันผวนใกล้เกณฑ์ตัดสิน

เนื่องจากผู้จัดทำเป็นนักเรียนระดับมัธยมศึกษา โครงงานนี้จึงจำกัดการทดลองไว้กับโหลดไฟฟ้ากระแสตรงแรงดันต่ำ (3–9 โวลต์) เพื่อความปลอดภัย ไม่เกี่ยวข้องกับไฟฟ้ากระแสสลับ 220 โวลต์ของบ้านเรือน

## 2. วัตถุประสงค์

1. เพื่อออกแบบและพัฒนาระบบตรวจจับสถานะเปิด-ปิดของอุปกรณ์ไฟฟ้ากระแสตรงโดยใช้ ESP32 ร่วมกับเซนเซอร์วัดกระแส INA219

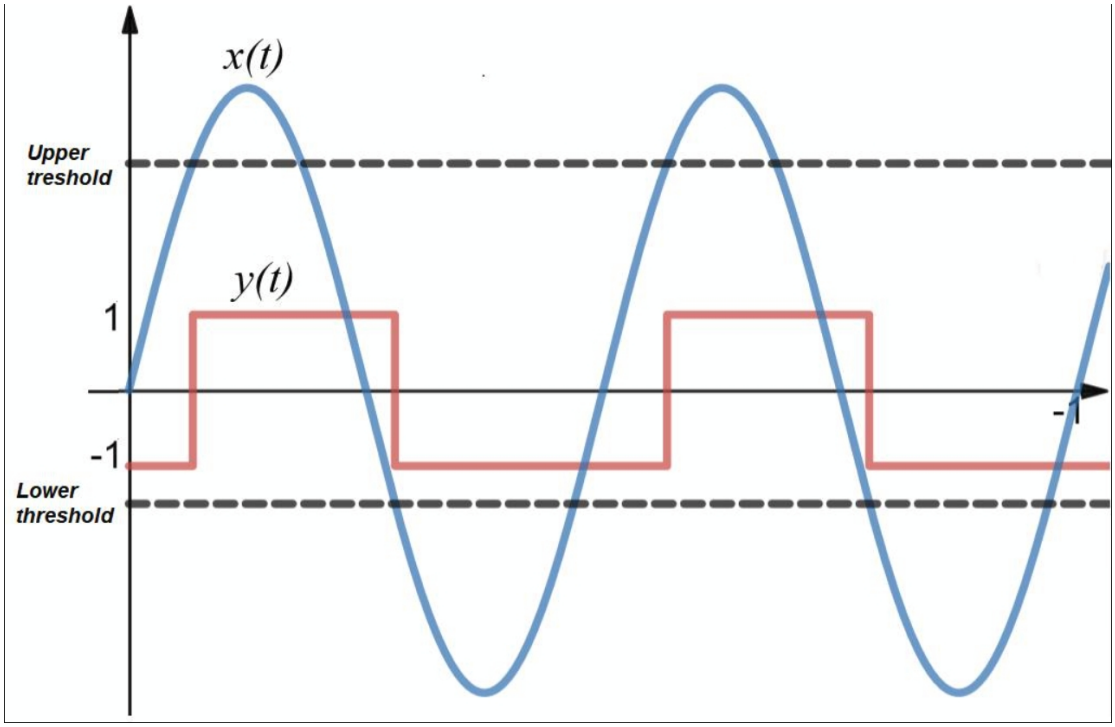
2. เพื่อพัฒนาการแสดงสถานะบนเว็บไซต์และการส่งข้อความสรุปการใช้งานประจำวันผ่านแอปพลิเคชันไลน์

3. เพื่อทดสอบและประเมินความแม่นยำ ความหน่วงเวลา และความเสถียรของระบบ เปรียบเทียบกับสถานะการทำงานจริงและค่าที่วัดด้วยมัลติมิเตอร์

4. เพื่อศึกษาวงจรของแผงวงจรขยาย ESP32 ที่มีจำหน่ายทั่วไป แล้วออกแบบและสร้างแผงวงจรของผู้จัดทำเองสำหรับใช้กับระบบนี้

5. เพื่อเปรียบเทียบการตัดสินสถานะด้วยเกณฑ์สองระดับ (Hysteresis) กับเกณฑ์เดียว และประเมินความง่ายในการใช้งานหน้าเว็บไซต์

6. เพื่อศึกษาความสัมพันธ์ระหว่างการเปลี่ยนแปลงของกระแสไฟฟ้ากับลักษณะการทำงานของโหลด เช่น Servo Motor โดยวิเคราะห์ค่ากระแสไฟฟ้าสูงสุดชั่วขณะ (Peak Current) และรูปแบบการเปลี่ยนแปลงของกระแสไฟฟ้า




**ภาพที่ 1 ตัวอย่างกราฟการใช้ไฟฟ้าแบบ Hysteresis**

**ที่มา : https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/References/hysteresis_function.html**

## 3. สมมติฐาน

ผู้จัดทำกำหนดสมมติฐานเป็นเป้าหมายล่วงหน้าที่ตรวจสอบได้ด้วยการวัดจริง โดยรายงานผลด้วยความเป็นจริงในบทที่ 4 ดังนี้

1. ระบบสามารถตรวจจับสถานะเปิด-ปิดของอุปกรณ์ได้ถูกต้องไม่น้อยกว่าร้อยละ 90 เมื่อเปรียบเทียบกับสถานะอ้างอิง โดยทดสอบการเปลี่ยนสถานะของโหลดแต่ละชนิดจำนวน 10–30 รอบ (หนึ่งรอบประกอบด้วยการเปลี่ยนจากสถานะปิดเป็นเปิดและจากสถานะเปิดเป็นปิด) และคำนวณความถูกต้องจากสัดส่วนจำนวนครั้งที่ระบบจำแนกสถานะได้ถูกต้องต่อจำนวนครั้งที่ทดสอบทั้งหมด

2. ค่ากระแสที่วัดได้จาก INA219 มีความคลาดเคลื่อนไม่เกินร้อยละ 5 เมื่อเปรียบเทียบกับมัลติมิเตอร์ดิจิทัล โดยทดสอบในช่วงกระแสที่เครื่องมือทั้งสองสามารถวัดได้อย่างเหมาะสม บันทึกย่านการวัด ความละเอียด และค่าความแม่นยำตามข้อกำหนดของเครื่องมือ และไม่นำค่ากระแสอ้างอิงที่ใกล้ศูนย์มาใช้คำนวณเปอร์เซ็นต์ความคลาดเคลื่อน

3. เมื่อสถานะของโหลดเปลี่ยนแปลงจริง ระบบสามารถแสดงสถานะใหม่บนเว็บไซต์ได้ภายในเวลาไม่เกิน 10 วินาที โดยเริ่มจับเวลาตั้งแต่โหลดเปลี่ยนสถานะจนถึงเวลาที่เว็บไซต์แสดงสถานะใหม่

4. เมื่อไม่ได้รับข้อมูลจากอุปกรณ์เกิน 30 วินาที หน้าเว็บแสดง N/A หมายถึง ไม่มีข้อมูลใหม่เกิน 30 วินาที ไม่ใช่สถานะโหลด

5. การใช้เกณฑ์สองระดับ (Hysteresis) สามารถลดจำนวนครั้งของการเปลี่ยนสถานะที่ไม่ตรงกับสถานะอ้างอิงได้ เมื่อเปรียบเทียบกับการใช้เกณฑ์เดียว โดยทดสอบกับชุดข้อมูลกระแสไฟฟ้าเดียวกันภายใต้เงื่อนไขเดียวกัน และทำการทดสอบซ้ำ 10 ชุดข้อมูล

6. แผงวงจรที่พัฒนาขึ้นสามารถจ่ายแรงดันเอาต์พุต 3.3 โวลต์ โดยมีความคลาดเคลื่อนไม่เกินร้อยละ 5 จากค่าแรงดันเป้าหมาย และสามารถทำงานร่วมกับ ESP32 และ INA219 ได้อย่างเสถียรภายใต้เงื่อนไขโหลดที่กำหนด

7. ผู้ทดสอบสามารถทำภารกิจพื้นฐานบนเว็บไซต์ที่กำหนดได้สำเร็จไม่น้อยกว่าร้อยละ 80 ของจำนวนครั้งที่ได้รับมอบหมาย โดยกำหนดภารกิจ ได้แก่ การตรวจสอบสถานะอุปกรณ์ การอ่านค่ากระแสไฟฟ้า และการตรวจสอบเวลาที่ได้รับข้อมูลล่าสุด

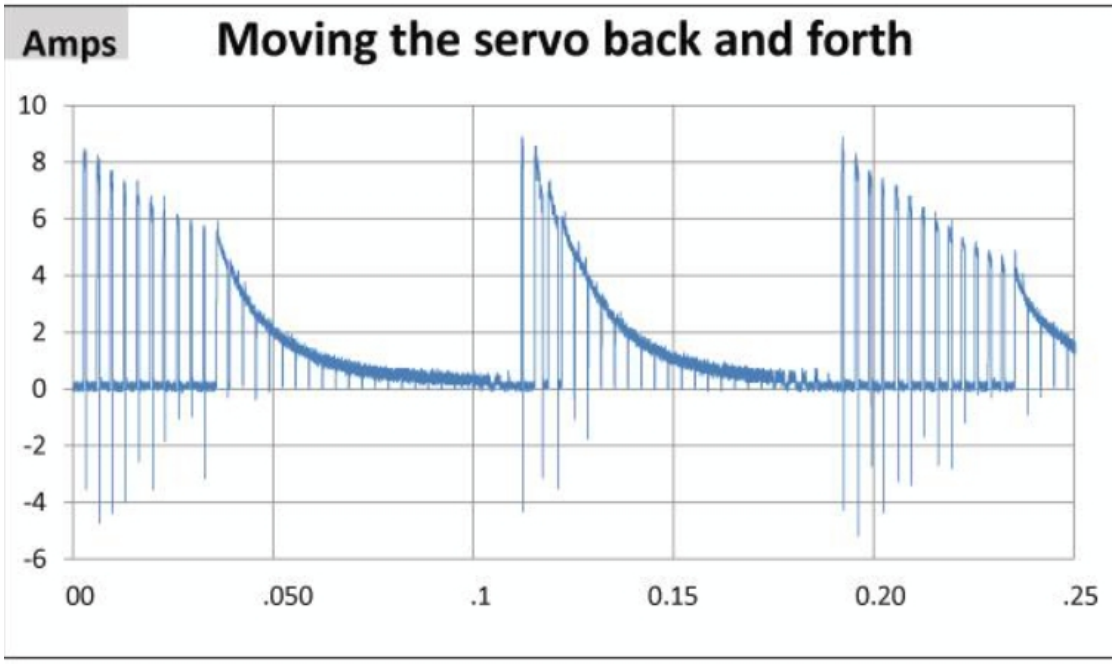
## 4. ขอบเขตของการศึกษาค้นคว้า

**4.1 ขอบเขตด้านเนื้อหา** ออกแบบและพัฒนาระบบตรวจจับและแจ้งเตือนสถานะเปิด-ปิดของอุปกรณ์ไฟฟ้า ประกอบด้วยส่วนวัด (ESP32 + INA219) ส่วนหลังบ้าน (Cloudflare Worker + D1) ส่วนแสดงผล (เว็บไซต์) และส่วนแจ้งเตือน (LINE)

**4.2 ขอบเขตด้านอุปกรณ์ที่ทดสอบ** ทดสอบกับโหลดไฟฟ้ากระแสตรงแรงดัน 3–9 โวลต์ จำนวน 1–4 ชนิด โดยจำกัดกระแสให้อยู่ภายในช่วงที่วงจรตรวจวัดและอุปกรณ์ทุกส่วนรองรับได้อย่างปลอดภัย ทั้งนี้พิจารณาค่าของตัวต้านทานชันต์ (Shunt Resistor) กำลังไฟฟ้าที่สูญเสียในตัวต้านทาน และความสามารถในการรองรับกระแสของแผงวงจรประกอบด้วย

**4.3 ขอบเขตด้านระยะเวลา** ดำเนินการศึกษาและพัฒนาระบบในภาคเรียนที่ 2 ปีการศึกษา 2569

**4.4 ข้อจำกัดของระบบ** ระบบทำหน้าที่ตรวจสอบสถานะของอุปกรณ์เท่านั้น ไม่สามารถควบคุมหรือสั่งตัดกระแสไฟฟ้าของอุปกรณ์ได้ โดยต้องติดตั้ง INA219 ในลักษณะอนุกรมกับสายไฟด้านบวก (V+) ของโหลด และจำเป็นต้องเชื่อมต่อกับเครือข่าย Wi-Fi ความถี่ 2.4 GHz ทั้งนี้ เกณฑ์กระแสสำหรับจำแนกสถานะต้องปรับให้เหมาะสมกับโหลดแต่ละชนิด โดยระบบกำหนดค่าเริ่มต้นสำหรับเกณฑ์เปิดที่ 25 mA และเกณฑ์ปิดที่ 15 mA ตามหลักการ Hysteresis




**ภาพที่ 2 ตัวอย่างกราฟการใช้ไฟฟ้าของ servo**

**ที่มา :** https://www.modelaviation.com/digital-servos

**4.5 ขอบเขตด้านซอฟต์แวร์และบริการที่ใช้** พัฒนา Firmware สำหรับบอร์ด ESP32-DevKit โดยใช้ภาษา C/C++ ร่วมกับเซนเซอร์ INA219 เพื่อวัดค่าทางไฟฟ้า 3 รายการ ได้แก่ แรงดันไฟฟ้า (Voltage) กระแสไฟฟ้า (Current) และกำลังไฟฟ้า (Power) สำหรับระบบฝั่ง Backend ใช้บริการ Cloudflare Workers ในการประมวลผลและรับส่งข้อมูล และใช้ Cloudflare D1 เป็นฐานข้อมูลสำหรับจัดเก็บข้อมูลของระบบ ส่วนเว็บไซต์ทำหน้าที่แสดงค่าทางไฟฟ้าและสถานะของอุปกรณ์ รวมถึงส่งข้อความสรุปการใช้งานผ่าน LINE Official Account ทุกวันเวลา 16:10 น. (UTC+7) นอกจากนี้ ระบบจะตรวจสอบความสดใหม่ของข้อมูล โดยหากไม่ได้รับข้อมูลใหม่จากอุปกรณ์เกิน 30 วินาที จะแสดงสถานะเป็น N/A (Not Available)

**4.6 ขอบเขตด้านการประเมินผล** การประเมินผลระบบประกอบด้วยการทดสอบความถูกต้องของการจำแนกสถานะเปิด-ปิด โดยเปรียบเทียบผลที่ระบบตรวจจับได้กับสถานะการทำงานจริงของอุปกรณ์ และทดสอบความคลาดเคลื่อนของค่ากระแสไฟฟ้าที่วัดจาก INA219 เมื่อเปรียบเทียบกับมัลติมิเตอร์ดิจิทัล นอกจากนี้ จะวัดระยะเวลาตั้งแต่โหลดมีการเปลี่ยนแปลงสถานะจนกระทั่งเว็บไซต์แสดงผลสถานะใหม่ รวมถึงทดสอบการทำงานเมื่อเครือข่าย Wi-Fi หรือระบบ Backend ไม่สามารถใช้งานได้ โดยพิจารณาความสามารถในการจัดเก็บข้อมูลชั่วคราวด้วย Offline Buffer และการส่งข้อมูลย้อนหลังเมื่อระบบกลับมาทำงานตามปกติการทดสอบความถูกต้องของข้อมูลจะครอบคลุมกรณีการส่งข้อมูลซ้ำ การส่งข้อมูลที่มีลำดับไม่ถูกต้อง และการตรวจสอบข้อมูลย้อนหลัง

ส่วนการประเมินเว็บไซต์จะพิจารณาจากความสำเร็จในการทำภารกิจพื้นฐานของผู้ทดสอบ และความคิดเห็นต่อความสะดวกในการใช้งานสำหรับการศึกษา Servo Motor จะเก็บข้อมูลกระแสไฟฟ้าแยกจากการตรวจจับสถานะตามรอบการทำงานปกติ โดยเพิ่มความถี่ในการเก็บข้อมูลให้เหมาะสมกับการเปลี่ยนแปลงของกระแส เพื่อศึกษาค่ากระแสสูงสุดชั่วขณะและลักษณะการเปลี่ยนแปลงของกระแสในแต่ละช่วงการทำงาน

## 5. นิยามศัพท์

**ระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้า** หมายถึง ระบบอิเล็กทรอนิกส์ที่ประกอบด้วยเซนเซอร์วัดกระแส ไมโครคอนโทรลเลอร์ และบริการบนอินเทอร์เน็ต ทำหน้าที่ตรวจว่าอุปกรณ์ไฟฟ้าทำงานอยู่หรือไม่ แล้วแจ้งให้ผู้ใช้ทราบ

**ESP32** หมายถึง ไมโครคอนโทรลเลอร์ที่มีความสามารถในการเชื่อมต่อ Wi-Fi และ Bluetooth ในตัว ทำหน้าที่ประมวลผลข้อมูลจากเซนเซอร์และส่งข้อมูลจากส่วนฮาร์ดแวร์ไปยังระบบ Backend (Cloudflare Workers)

**INA219** หมายถึง ไอซีสำหรับตรวจวัดแรงดันไฟฟ้า กระแสไฟฟ้า และกำลังไฟฟ้า โดยตรวจวัดกระแสผ่านแรงดันตกคร่อมตัวต้านทานชันต์ (Shunt Resistor) และสื่อสารกับไมโครคอนโทรลเลอร์ผ่านระบบ I2C

**Hysteresis** หมายถึง การใช้เกณฑ์ตัดสินสองระดับ (เกณฑ์เปิดสูงกว่าเกณฑ์ปิด) เพื่อไม่ให้สถานะกระพริบสลับไปมาเมื่อกระแสผันผวนรอบค่าเดียว

**N/A** หมายถึง (ย่อมาจาก Not Available) สถานะ “ไม่ทราบ” ที่เว็บไซต์แสดงเมื่อไม่ได้รับข้อมูลเกิน 30 วินาที ซึ่งไม่ได้แปลว่าอุปกรณ์ปิด

**Cloudflare Workers และ D1** หมายถึง บริการของ Cloudflare โดย Workers ใช้สำหรับประมวลผลคำสั่งฝั่งเซิร์ฟเวอร์ ส่วน D1 เป็นบริการฐานข้อมูลเชิงสัมพันธ์ที่ใช้ SQLite เป็นระบบจัดการฐานข้อมูล ทำหน้าที่รับ บันทึก และให้บริการข้อมูลของระบบผ่าน API Endpoint

**IDE (Integrated Development Environment)** หมายถึง สภาพแวดล้อมหรือเครื่องมือที่ช่วยในการพัฒนาโปรแกรม ประกอบด้วยเครื่องมือสำหรับเขียน แก้ไข ตรวจสอบ และแปลโปรแกรม ในโครงงานนี้ใช้ LazyVim ซึ่งเป็นชุดการตั้งค่าสำหรับ Neovim ในการแก้ไขโค้ด และใช้ PlatformIO เป็นแพลตฟอร์มสำหรับจัดการการพัฒนา แปล และอัปโหลด Firmware ไปยังบอร์ด ESP32

**PCB (Printed Circuit Board)** หมายถึง แผงวงจรพิมพ์ ซึ่งเป็นแผ่นที่มีลายทองแดงสำหรับเชื่อมต่อขาของอุปกรณ์อิเล็กทรอนิกส์เข้าด้วยกันตามการออกแบบวงจร

**Endpoint** หมายถึง จุดปลายทางของ API ที่ใช้รับหรือให้บริการข้อมูลผ่านเครือข่าย เช่น /update, /status และ /health

**Current Threshold** หมายถึง ค่ากระแสไฟฟ้าที่กำหนดให้เป็นเกณฑ์สำหรับจำแนกสถานะการทำงานของอุปกรณ์

**Offline Buffer** หมายถึง พื้นที่พักข้อมูลชั่วคราวบนอุปกรณ์ เพื่อเก็บข้อมูลไว้เมื่อไม่สามารถส่งข้อมูลผ่านเครือข่ายได้

**Sequence Number** หมายถึง หมายเลขลำดับของข้อมูลที่ใช้ตรวจสอบลำดับการรับส่งและป้องกันข้อมูลซ้ำหรือข้อมูลเก่า

**Boot ID** หมายถึง รหัสประจำรอบการเริ่มต้นทำงานของอุปกรณ์ ใช้แยกข้อมูลก่อนและหลังการรีสตาร์ต

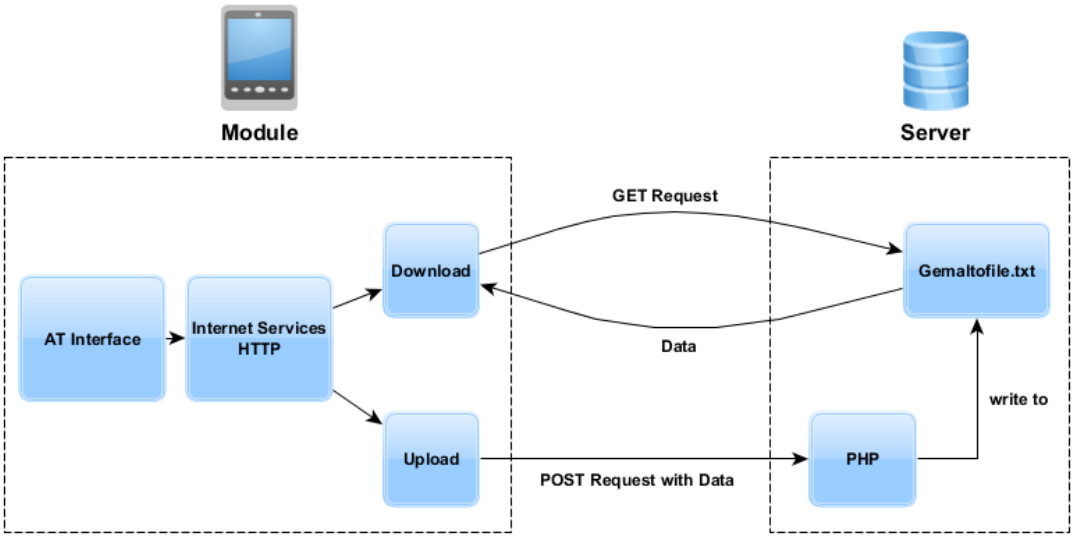
**Polling** หมายถึง วิธีที่เว็บไซต์ร้องขอข้อมูลจาก Backend เป็นระยะตามช่วงเวลาที่กำหนด

**Stale Data** หมายถึง ข้อมูลที่ไม่ได้รับการอัปเดตภายในระยะเวลาที่กำหนด จึงไม่ควรนำไปแสดงเป็นสถานะปัจจุบัน

**REST API** หมายถึง แนวทางการออกแบบ API สำหรับการแลกเปลี่ยนข้อมูลผ่านเครือข่าย โดยอาศัยหลักการจัดการทรัพยากรและ HTTP Methods เช่น GET และ POST

**I2C** หมายถึง ระบบสื่อสารแบบอนุกรมที่ใช้สายสัญญาณหลัก SDA และ SCL เพื่อแลกเปลี่ยนข้อมูลระหว่างอุปกรณ์

**High-side Current Sensing** หมายถึง วิธีการตรวจวัดกระแสไฟฟ้าโดยติดตั้งตัวต้านทานชันต์ (Shunt Resistor) ไว้ทางด้านสายบวกของแหล่งจ่ายไฟ และวัดแรงดันตกคร่อมตัวต้านทานดังกล่าวเพื่อนำไปคำนวณกระแสไฟฟ้า




**ภาพที่ 3 ตัวอย่างการส่งผ่าน Endpoint ด้วย GET/POST method ระหว่าง user/client - server ที่มา: https://cee5730304498-5730304668.blogspot.com/2015/12/mini-project-dimmer_17.html**

## 6. ประโยชน์ที่คาดว่าจะได้รับ

1. ได้ระบบต้นแบบที่ตรวจสอบสถานะอุปกรณ์ไฟฟ้ากระแสตรงจากระยะไกลได้

2. ช่วยลดโอกาสลืมปิดอุปกรณ์ และส่งเสริมการประหยัดพลังงาน

3. ผู้จัดทำได้เรียนรู้การทำงานร่วมกันของฮาร์ดแวร์ ซอฟต์แวร์ และบริการคลาวด์ ตลอดจนกระบวนการทดลองและวิเคราะห์ข้อมูล

4. ได้ศึกษาและเข้าใจหลักการตรวจวัดแรงดัน กระแส และกำลังไฟฟ้าด้วยเซนเซอร์ INA219 รวมถึงการประยุกต์ใช้ไมโครคอนโทรลเลอร์ในระบบตรวจสอบอุปกรณ์ไฟฟ้า

5. ได้แนวทางในการออกแบบและพัฒนาแผงวงจรอิเล็กทรอนิกส์ที่สามารถนำไปประยุกต์ใช้งานร่วมกับไมโครคอนโทรลเลอร์และเซนเซอร์วัดกระแสได้

6. ได้ข้อมูลผลการทดลองเกี่ยวกับความเปลี่ยนแปลงของกระแสไฟฟ้าในโหลดชนิดต่าง ๆ ซึ่งสามารถนำไปใช้เป็นพื้นฐานในการพัฒนาระบบตรวจสอบพลังงานและระบบติดตามการทำงานของอุปกรณ์ในอนาคต

# บทที่ 2
# เอกสารและทฤษฎีที่เกี่ยวข้อง

การพัฒนาระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้าในโครงงานนี้ ผู้จัดทำได้ศึกษาเอกสารและทฤษฎีที่เกี่ยวข้อง เพื่อนำมาเป็นแนวทางในการออกแบบและพัฒนาระบบ โดยมีหัวข้อดังต่อไปนี้

## 1. ไมโครคอนโทรลเลอร์ ESP32

ESP32 เป็นไมโครคอนโทรลเลอร์แบบระบบบนชิป (System on Chip: SoC) ที่พัฒนาโดยบริษัท Espressif Systems โดย ESP32-DevKit รุ่นที่ใช้ในโครงงานมีหน่วยประมวลผลและรองรับการเชื่อมต่อ Wi-Fi และ Bluetooth ในตัว จึงสามารถเชื่อมต่อเครือข่ายอินเทอร์เน็ตได้โดยไม่จำเป็นต้องใช้อุปกรณ์สื่อสารไร้สายเพิ่มเติม นอกจากนี้ยังมีขา GPIO และรองรับการสื่อสารแบบ I2C ซึ่งเหมาะสำหรับการเชื่อมต่อและอ่านค่าจากเซนเซอร์

## 2. หลักการวัดกระแสไฟฟ้าด้วยเซนเซอร์ INA219 และการสื่อสารแบบ I2C

INA219 เป็นไอซีสำหรับตรวจวัดกระแสไฟฟ้าและกำลังไฟฟ้าแบบ High-side Current Sensing โดยตรวจวัดแรงดันตกคร่อมตัวต้านทานชันต์ (Shunt Resistor) ผ่านขา VIN+ และ VIN- และตรวจวัดแรงดันบัส (Bus Voltage) เพื่อนำค่าที่ได้มาประมวลผลเป็นกระแสไฟฟ้าและกำลังไฟฟ้าภายในประกอบด้วยวงจรแปลงสัญญาณแอนะล็อกเป็นดิจิทัล (ADC) และรีจิสเตอร์สำหรับจัดเก็บค่าที่ตรวจวัดได้ โดยการคำนวณกระแสอาศัยกฎของโอห์ม ($V = I \cdot R$) ร่วมกับค่าความต้านทานของตัวต้านทานชันต์ สำหรับบอร์ดสำเร็จรูปของ Adafruit รุ่นที่ใช้อ้างอิงมีตัวต้านทานชันต์ขนาด 0.1 โอห์ม และระบุช่วงการวัดกระแสสูงสุดประมาณ ±3.2 แอมแปร์ ทั้งนี้ ความสามารถในการรองรับกระแสของวงจรจริงขึ้นอยู่กับตัวต้านทานชันต์และการออกแบบแผงวงจรด้วย

โครงสร้างภายในอย่างง่ายของ INA219 แสดงในภาพที่ 4 ส่วนหัวข้อถัดไปอธิบายการสื่อสารแบบ I2C ที่ใช้อ่านค่าจากเซนเซอร์ และหลักการวัดกระแสด้วยตัวต้านทานชันต์ซึ่งเป็นพื้นฐานของการวัด

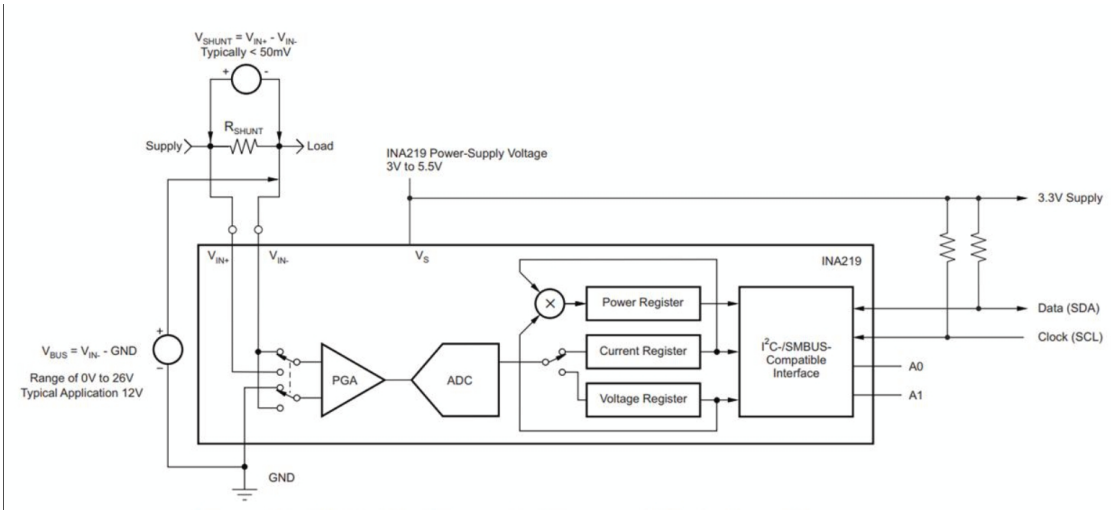



**Figure 13. INA219 Configured for Shunt and Bus Voltage Measurement**

**ภาพที่ 4 แผนภาพบล็อกอย่างง่ายของเซนเซอร์ INA219**

**ที่มา : https://e2e.ti.com/support/amplifiers-group/amplifiers/f/amplifiers-forum/815371/ina219-supply-voltage-range**

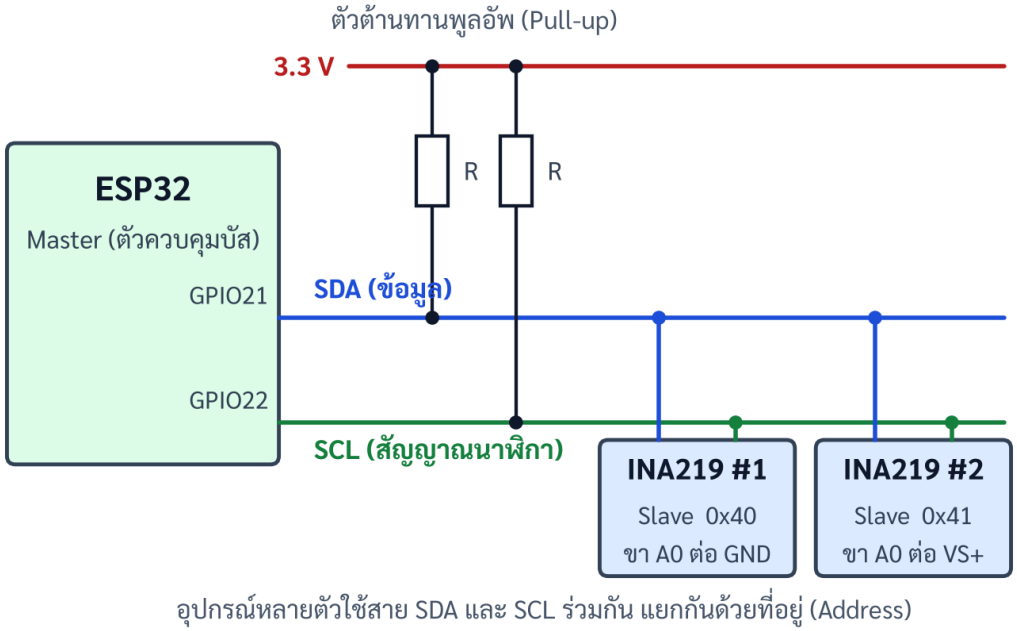
### 2.1 การสื่อสารแบบ I2C (Inter-Integrated Circuit)

I2C เป็นมาตรฐานการสื่อสารอนุกรมแบบซิงโครนัสที่ใช้สายสัญญาณเพียงสองเส้น คือ SDA (Serial Data) สำหรับรับส่งข้อมูล และ SCL (Serial Clock) สำหรับส่งสัญญาณนาฬิกาเพื่อกำหนดจังหวะการรับส่ง (NXP Semiconductors, 2021) อุปกรณ์บนบัสแบ่งเป็นมาสเตอร์ (Master) ซึ่งเป็นผู้เริ่มการสื่อสารและกำหนดสัญญาณนาฬิกา และสเลฟ (Slave) ซึ่งตอบสนองเมื่อถูกเรียกด้วยที่อยู่ของตนเอง ในโครงงานนี้ ESP32 ทำหน้าที่เป็นมาสเตอร์ และ INA219 ทั้งสองตัวเป็นสเลฟ

สาย SDA และ SCL เป็นแบบโอเพนเดรน (Open-drain) คืออุปกรณ์แต่ละตัวทำได้เพียงดึงสายลงสู่ระดับต่ำ จึงต้องมีตัวต้านทานพูลอัพ (Pull-up Resistor) ต่อกับแรงดัน 3.3 โวลต์ เพื่อดึงสายกลับสู่ระดับสูงเมื่อไม่มีอุปกรณ์ใดดึงสาย โมดูลบางรุ่นมีตัวต้านทานนี้ติดตั้งมาแล้ว จึงควรตรวจสอบกับโมดูลที่ใช้ การสื่อสารแต่ละครั้งเริ่มจากมาสเตอร์ส่งที่อยู่ของสเลฟ 7 บิตพร้อมบิตบอกทิศทางอ่านหรือเขียน สเลฟที่มีที่อยู่ตรงกันจะตอบรับด้วยสัญญาณ ACK จากนั้นจึงรับส่งข้อมูลเป็นไบต์

INA219 กำหนดที่อยู่ได้ด้วยขา A0 และ A1 โดยที่อยู่เริ่มต้นคือ 0x40 และเปลี่ยนเป็น 0x41 ได้เมื่อต่อขา A0 เข้ากับ VS+ ตามตาราง Address Selection ในเอกสารข้อมูลของผู้ผลิต จึงต่อ INA219 ได้หลายตัวบนบัสเดียวกันหากที่อยู่ไม่ซ้ำกัน โครงงานนี้ใช้ INA219 สองตัวที่ที่อยู่ 0x40 และ 0x41 ต่อเข้ากับขา GPIO21 (SDA) และ GPIO22 (SCL) ของ ESP32 ดังแสดงในภาพที่ 5 โปรแกรมอ่านค่าผ่านไลบรารี Adafruit INA219 หากเซนเซอร์ตัวใดไม่ตอบสนอง เช่น สายหลุดหรือที่อยู่ไม่ตรง

ระบบจะทำเครื่องหมายว่าเซนเซอร์ตัวนั้นอ่านค่าไม่ได้ (ok = false) และแสดงเป็น FAULT บนหน้าเว็บ ขณะที่เซนเซอร์อีกตัวยังทำงานต่อได้




**ภาพที่ 5 การเชื่อมต่อ ESP32 กับ INA219 สองตัวบนบัส I2C**

**ที่มา : ผู้จัดทำ**

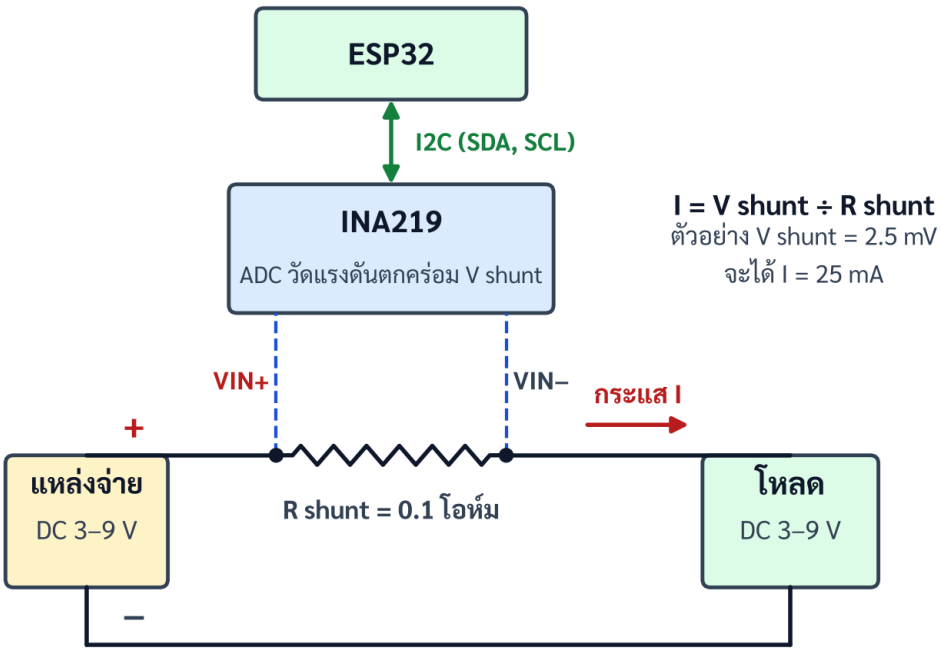
### 2.2 หลักการวัดกระแสไฟฟ้าด้วยตัวต้านทานชันต์

แรงดันไฟฟ้า (Voltage: V) คือความต่างศักย์ที่ผลักดันให้ประจุไฟฟ้าไหล กระแสไฟฟ้า (Current: I) คืออัตราการไหลของประจุ และความต้านทาน (Resistance: R) คือสมบัติที่ต้านการไหลของกระแส ทั้งสามค่าสัมพันธ์กันตามกฎของโอห์ม (Ohm's Law)

$$V = I \times R$$

ไมโครคอนโทรลเลอร์อ่านกระแสได้ไม่โดยตรง จึงวัดกระแสทางอ้อมด้วยการต่อตัวต้านทานค่าต่ำที่เรียกว่าตัวต้านทานชันต์ (Shunt Resistor) อนุกรมกับโหลด เมื่อกระแสไหลผ่านจะเกิดแรงดันตกคร่อมตัวต้านทานนี้ในสัดส่วนตรงกับกระแส INA219 วัดแรงดันตกคร่อมดังกล่าวผ่านขา VIN+ และ VIN− แล้วคำนวณกระแสจากสมการต่อไปนี้ ดังแสดงในภาพที่ 6

$$I = V_{shunt} \div R_{shunt}$$




**ภาพที่ 6 หลักการวัดกระแสไฟฟ้าด้วยตัวต้านทานชันต์**

**ที่มา : ผู้จัดทำ**

ตัวอย่างการคำนวณ โมดูลที่ใช้อ้างอิงมี R shunt เท่ากับ 0.1 โอห์ม ที่กระแส 25 mA ซึ่งเป็นเกณฑ์เปิดเริ่มต้นของระบบ จะเกิดแรงดันตกคร่อม 0.025 A × 0.1 โอห์ม = 2.5 mV และที่เกณฑ์ปิด 15 mA เท่ากับ 1.5 mV ขณะที่ ADC ภายใน INA219 วัดแรงดันตกคร่อมได้ละเอียดหน่วยละ 10 ไมโครโวลต์ตามเอกสารข้อมูลของผู้ผลิต ซึ่งเทียบเท่ากระแสประมาณ 0.1 mA ต่อหนึ่งหน่วย จึงละเอียดเพียงพอที่จะแยกเกณฑ์ที่ห่างกัน 10 mA ได้

การใส่ตัวต้านทานชันต์ทำให้เกิดกำลังไฟฟ้าสูญเสียและแรงดันตกคร่อมในวงจรของโหลด กำลังที่สูญเสียคำนวณจากสมการต่อไปนี้

$$P = I^2 \times R$$

เช่น ที่กระแส 1 แอมแปร์ สูญเสียกำลัง $1^2 \times 0.1 = 0.1$ วัตต์ (100 mW) และแรงดันที่โหลดได้รับลดลง 0.1 โวลต์ ซึ่งคิดเป็นร้อยละ 3.3 ของแหล่งจ่าย 3 โวลต์ แต่เพียงร้อยละ 1.1 ของแหล่งจ่าย 9 โวลต์ และที่กระแสสูงสุดของโมดูลประมาณ 3.2 แอมแปร์ ตัวต้านทานต้องทนกำลังได้มากกว่า 1 วัตต์ ($3.2^2 \times 0.1 \approx 1.02$ วัตต์) ผู้จัดทำจึงจำกัดกระแสของโหลดที่ทดสอบให้อยู่ในช่วงที่วงจรรองรับได้ตามขอบเขตการศึกษา ทั้งนี้ ค่าข้างต้นอ้างอิงโมดูลสำเร็จรูป หากออกแบบวงจรวัดกระแสเอง ต้องคำนวณค่าตัวต้านทานชันต์ กำลังที่ทนได้ และช่วงการวัดใหม่ตามวงจรจริง

## 3. อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things: IoT)

IoT (Internet of Things) คือแนวคิดการเชื่อมต่ออุปกรณ์อิเล็กทรอนิกส์เข้ากับอินเทอร์เน็ต เพื่อรับส่งข้อมูลกับระบบอื่นหรือผู้ใช้ โดยทั่วไปประกอบด้วยส่วนตรวจวัด ส่วนประมวลผลและเชื่อมต่อเครือข่าย และส่วนแสดงผลหรือแจ้งเตือน โครงงานนี้ประยุกต์ใช้แนวคิดดังกล่าวโดยให้ ESP32 ส่งข้อมูลไปยัง Backend ผ่าน HTTPS ซึ่งช่วยเข้ารหัสข้อมูลระหว่างการรับส่ง และใช้ Bearer Token สำหรับตรวจสอบสิทธิ์ของคำขอจากอุปกรณ์ก่อนนำข้อมูลเข้าสู่ระบบ

### 3.1 ความน่าเชื่อถือของการส่งข้อมูล: Offline Buffer และ Data Integrity

เครือข่ายไร้สายและบริการคลาวด์อาจหยุดชะงักได้เสมอ ระบบ IoT จึงต้องออกแบบให้ข้อมูลไม่สูญหายและไม่ถูกบันทึกซ้ำเมื่อการส่งล้มเหลว โครงงานนี้ใช้แนวคิด 4 ประการ ดังนี้

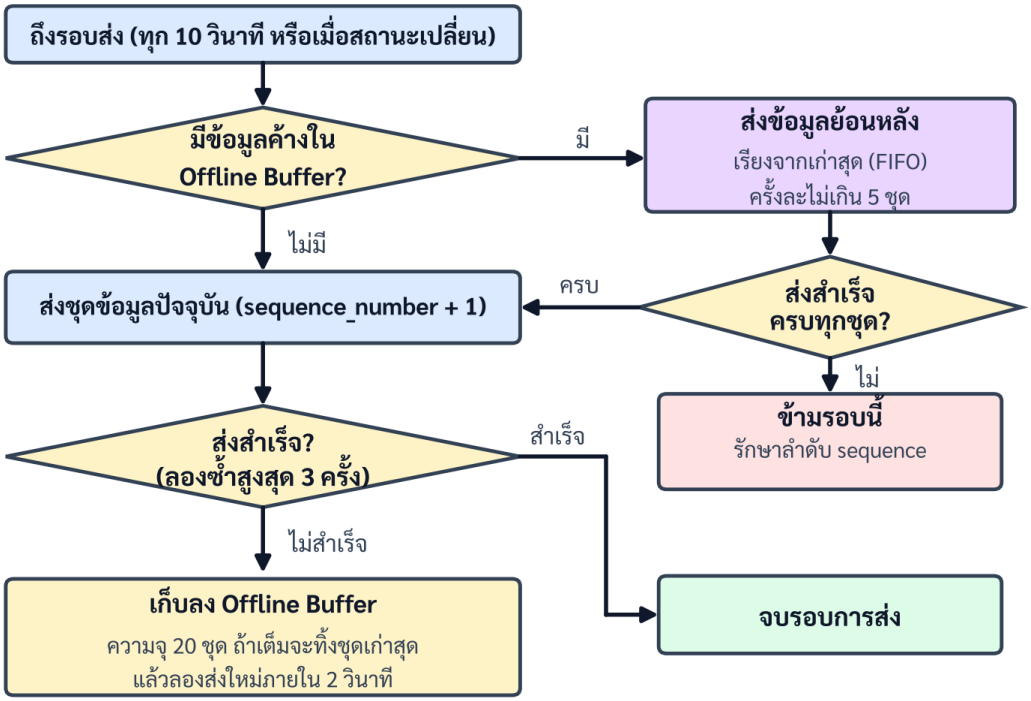
1) Offline Buffer คือหน่วยความจำชั่วคราวใน ESP32 สำหรับเก็บข้อมูลที่ส่งไม่สำเร็จเพื่อส่งใหม่ภายหลัง โครงงานนี้ใช้บัฟเฟอร์แบบวงแหวน (Ring Buffer) ความจุ 20 ชุดข้อมูล ซึ่งที่รอบส่งปกติ 10 วินาที รองรับการขาดการเชื่อมต่อได้ประมาณ 3 นาที เมื่อบัฟเฟอร์เต็มจะทิ้งชุดที่เก่าที่สุดเพื่อเก็บชุดล่าสุดไว้ และเมื่อเชื่อมต่อกลับมาจะส่งย้อนหลังเรียงจากชุดเก่าสุด (FIFO) ครั้งละไม่เกิน 5 ชุด ก่อนส่งข้อมูลปัจจุบัน ทั้งนี้บัฟเฟอร์อยู่ในหน่วยความจำ RAM ข้อมูลที่ค้างอยู่จึงหายหากบอร์ดรีเซ็ตหรือไฟดับ

2) Sequence Number คือหมายเลขลำดับที่เพิ่มขึ้นทีละหนึ่งทุกชุดข้อมูล Backend จะรับเฉพาะชุดที่มีหมายเลขมากกว่าค่าล่าสุดที่เคยรับ ทำให้ข้อมูลซ้ำหรือข้อมูลเก่าที่ถูกส่งซ้ำไม่เขียนทับข้อมูลใหม่ (Replay Protection)

3) Boot ID คือรหัสสุ่มที่ ESP32 สร้างทุกครั้งที่เริ่มทำงาน เนื่องจากหมายเลขลำดับในหน่วยความจำจะเริ่มนับใหม่หลังรีบูต Backend จึงใช้ boot_id แยกรอบการทำงาน เมื่อ boot_id เปลี่ยนจะเริ่มนับลำดับของรอบใหม่ ไม่ถือเป็นข้อมูลซ้ำ อย่างไรก็ตาม กลไกนี้ป้องกันข้อมูลซ้ำจากความผิดพลาดของระบบเป็นหลัก ไม่ใช่การพิสูจน์ตัวตนด้วยการเข้ารหัส

4) Stale Data คือข้อมูลที่เก่าเกินเวลาที่กำหนด ระบบถือว่าข้อมูลที่ไม่ได้รับการอัปเดตเกิน 30 วินาทีเป็น N/A แทนที่จะแสดงค่าเดิมค้างไว้หรือสรุปว่าอุปกรณ์ปิด เพื่อไม่ให้ผู้ใช้เข้าใจสถานะผิด

ลำดับการทำงานของ ESP32 ตั้งแต่ถึงรอบส่งจนถึงการเก็บลงบัฟเฟอร์เมื่อส่งไม่สำเร็จ แสดงในภาพที่ 7




**ภาพที่ 7 ลำดับการส่งข้อมูลและการใช้ Offline Buffer ของ ESP32**

**ที่มา : ผู้จัดทำ**

## 4. Cloudflare Workers และ Cloudflare D1

Cloudflare Workers เป็นบริการรันโค้ดฝั่งเซิร์ฟเวอร์แบบ serverless ที่ไม่ต้องดูแลเครื่องเซิร์ฟเวอร์เอง รองรับการตั้งเวลาทำงานด้วย Cron Triggers ส่วน Cloudflare D1 เป็นฐานข้อมูล SQL แบบ serverless ที่ใช้งานร่วมกับ Workers ได้ ระบบนี้ใช้ Cloudflare Workers เป็นส่วนประมวลผลและให้บริการ API โดยรับข้อมูลจาก ESP32 ผ่าน Endpoint /update ด้วย HTTP Method POST และให้เว็บไซต์ร้องขอข้อมูลสถานะผ่าน Endpoint /status ด้วย HTTP Method GET นอกจากนี้ยังกำหนด Cron Trigger ให้ทำงานเวลา 09:10 UTC (16:10 น. ตามเวลาประเทศไทย) เพื่อส่งข้อความสรุปการใช้งานประจำวัน ข้อมูลสถานะและประวัติการทำงานที่ได้รับจากอุปกรณ์จะถูกจัดเก็บใน Cloudflare D1 เพื่อให้ระบบสามารถเรียกดูข้อมูลย้อนหลังและนำไปใช้ในการแสดงผลได้

### 4.1 REST API, HTTP และ JSON

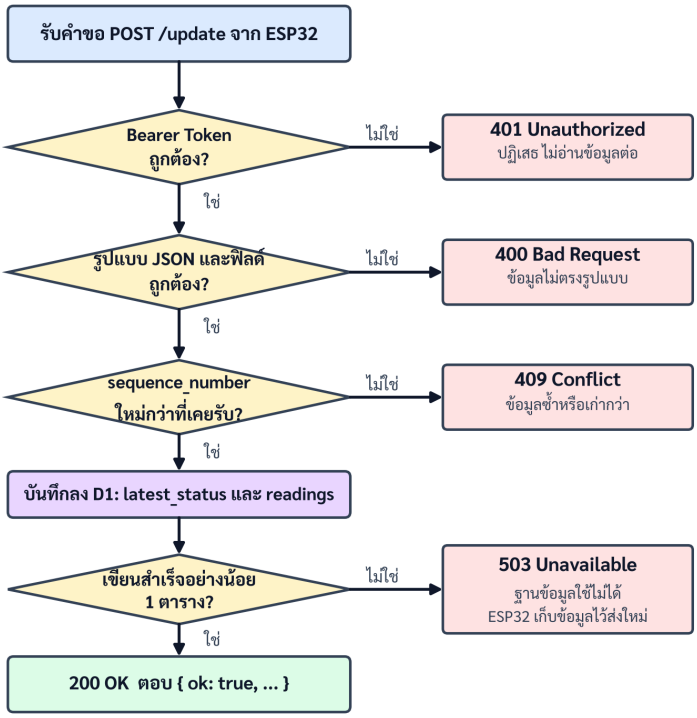
HTTP (Hypertext Transfer Protocol) เป็นโปรโตคอลแบบคำขอและคำตอบ ฝั่งไคลเอนต์ส่งคำขอ (Request) ไปยังที่อยู่ของบริการที่เรียกว่า Endpoint ฝั่งเซิร์ฟเวอร์ประมวลผลแล้วส่งคำตอบ (Response) กลับพร้อมรหัสสถานะ (Status Code) REST API คือแนวทางออกแบบบริการบน

HTTP ที่แยกการทำงานตาม Endpoint และ Method โดยใช้ GET เพื่ออ่านข้อมูล และ POST เพื่อส่งข้อมูลเข้าไปบันทึก Worker ของโครงงานนี้มี Endpoint หลักคือ POST /update สำหรับรับข้อมูลจาก ESP32 และ GET /status สำหรับให้เว็บไซต์อ่านสถานะล่าสุด รวมถึง /health, /summary และ /history สำหรับตรวจสอบระบบและสรุปข้อมูล

ข้อมูลถูกส่งผ่าน HTTPS ซึ่งเข้ารหัสระหว่างทาง และแนบ Bearer Token ใน Header Authorization เพื่อให้ Worker ตรวจสอบว่าคำขอมาจากอุปกรณ์ที่ได้รับอนุญาต ค่า Token เก็บเป็นความลับ (Secret) ของ Worker ไม่ฝังในซอร์สโค้ดที่เผยแพร่ ข้อมูลรับส่งอยู่ในรูปแบบ JSON (JavaScript Object Notation) ซึ่งเป็นข้อความที่มนุษย์อ่านได้และโปรแกรมทุกภาษาประมวลผลได้ ตัวอย่างข้อมูลที่ ESP32 ส่งไปยัง POST /update เป็นดังนี้ (ค่าตัวเลขเป็นตัวอย่างเพื่ออธิบายรูปแบบ)

```
{
  "device_id": "ESP32-01",
  "boot_id": "A1B2C3D4E5F60718",
  "sequence_number": 128,
  "sensors": [
    { "sensor_id": "INA219-01", "voltage_v": 5.12,
      "current_ma": 340.5, "power_mw": 1743.3,
      "is_on": true, "ok": true },
    { "sensor_id": "INA219-02", ... }
  ]
}
```

Worker ตอบรหัสสถานะตามผลการตรวจสอบ ได้แก่ 200 รับข้อมูลสำเร็จ, 400 ข้อมูลไม่ตรงรูปแบบ, 401 Token ไม่ถูกต้อง, 409 ข้อมูลซ้ำหรือเก่ากว่าที่เคยรับ, 503 ฐานข้อมูลใช้งานไม่ได้ ซึ่งทำให้ ESP32 เก็บข้อมูลไว้ส่งใหม่ และ 500 เกิดข้อผิดพลาดภายใน ลำดับการตรวจสอบของ Worker แสดงในภาพที่ 8




**ภาพที่ 8 ลำดับการตรวจสอบคำขอ POST /update ของ Worker**

**ที่มา : ผู้จัดทำ**

### 4.2 หลักการจัดเก็บข้อมูลด้วยฐานข้อมูล SQL

ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) จัดเก็บข้อมูลเป็นตาราง (Table) ที่ประกอบด้วยแถว (Row) แทนข้อมูลหนึ่งรายการ และคอลัมน์ (Column) แทนคุณสมบัติของข้อมูล แต่ละตารางมีคีย์หลัก (Primary Key) เป็นค่าที่ไม่ซ้ำกันใช้ระบุแถว และเรียกใช้ข้อมูลด้วยภาษา SQL (Structured Query Language) เช่น SELECT สำหรับอ่านข้อมูล และ INSERT สำหรับเพิ่มข้อมูล Cloudflare D1 เป็นฐานข้อมูล SQL แบบ serverless ที่ Worker เรียกใช้ได้โดยตรง

ฐานข้อมูลของโครงงานแบ่งเป็น 5 ตาราง ได้แก่ devices เก็บข้อมูลอุปกรณ์ sensors เก็บเซนเซอร์ของแต่ละอุปกรณ์ readings เก็บประวัติค่าที่วัดได้ทีละแถวต่อเซนเซอร์ latest_status เก็บสถานะล่าสุดของแต่ละอุปกรณ์ โดยมี device_id เป็นคีย์หลักจึงมีเพียงหนึ่งแถวต่ออุปกรณ์และถูกเขียนทับเมื่อมีข้อมูลใหม่ และ device_sequence เก็บหมายเลขลำดับกับ boot_id ล่าสุดสำหรับตรวจข้อมูลซ้ำ การแยกสถานะล่าสุดออกจากประวัติทำให้หน้าเว็บอ่านสถานะปัจจุบันได้โดยไม่ต้องค้นตารางประวัติที่ใหญ่ขึ้นเรื่อย ๆ และประวัติที่เก่ากว่า 7 วันจะถูกลบโดย Cron Trigger เพื่อควบคุมขนาดฐานข้อมูล ตัวอย่างคำสั่งที่ Worker ใช้อ่านสถานะล่าสุดเป็นดังนี้

```
SELECT device_id, sensors_json, updated_at
FROM latest_status WHERE device_id = ?
```

เครื่องหมาย ? คือพารามิเตอร์ที่ผูกค่าแยกจากตัวคำสั่ง (Prepared Statement) ซึ่งช่วยป้องกันการแทรกคำสั่ง SQL (SQL Injection)

### 5. LINE Messaging API

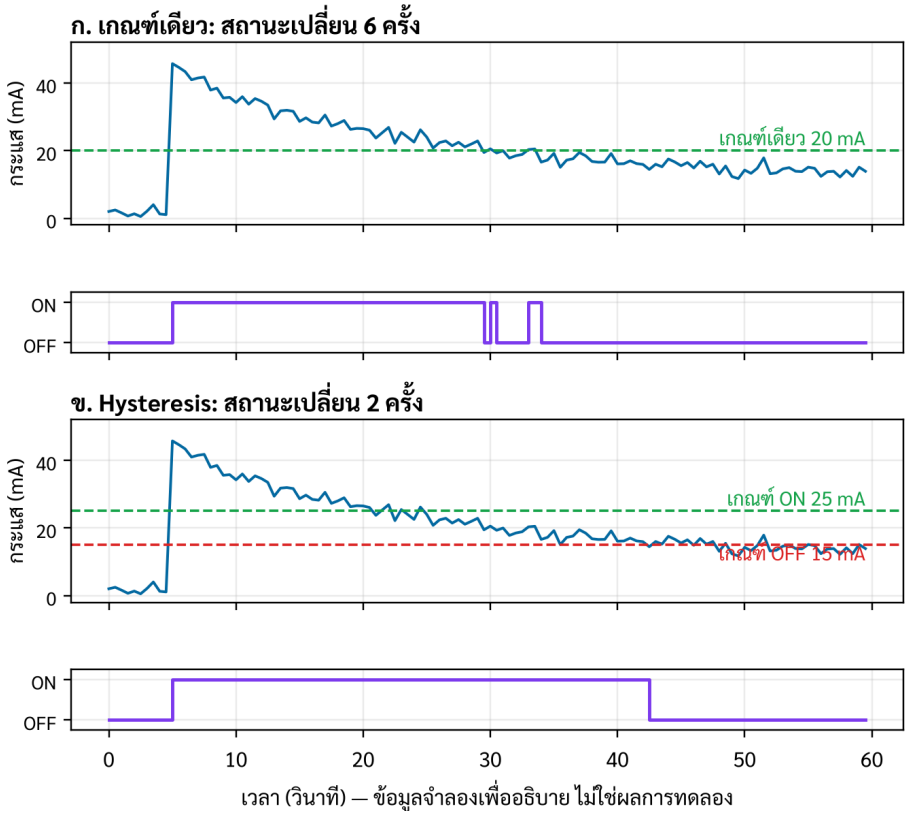
LINE Messaging API เป็นบริการที่ช่วยให้ระบบภายนอกสามารถส่งข้อความไปยังผู้ใช้ผ่าน LINE Official Account ได้ โดยรองรับรูปแบบการส่งข้อความหลายประเภท เช่น Broadcast และ Push Message โครงงานนี้เลือกส่งข้อความสรุปการใช้งานวันละหนึ่งครั้ง แทนการแจ้งเตือนทุกครั้งที่สถานะอุปกรณ์เปลี่ยนแปลง เพื่อลดจำนวนข้อความที่ส่งไปยังผู้ใช้และช่วยควบคุมการใช้งานให้อยู่ภายใต้ข้อจำกัดของบริการ

### 6. การตัดสินสถานะด้วยเกณฑ์สองระดับ (Hysteresis)

ค่ากระแสของโหลดจริงไม่นิ่งและไม่ได้เปลี่ยนเป็นขั้นบันไดสมบูรณ์เสมอไป การตัดสินสถานะด้วยกระแสจึงต้องออกแบบให้ทนต่อความผันผวน หัวข้อนี้อธิบายปัญหาของเกณฑ์เดียว หลักการ Hysteresis และขั้นตอนตัดสินที่ระบบใช้

#### 6.1 ปัญหาของเกณฑ์เดียวและการแกว่งของสถานะ

หากกำหนดเกณฑ์กระแสเพียงค่าเดียว เช่น 20 mA แล้วตัดสินว่าเป็น ON เมื่อกระแสมากกว่าเกณฑ์ และเป็น OFF เมื่อน้อยกว่า เมื่อกระแสแกว่งอยู่ใกล้เกณฑ์ เช่น 19.8 และ 20.2 mA จากสัญญาณรบกวนในการวัดหรือการทำงานของโหลด สถานะจะสลับ ON และ OFF ถี่ ๆ ทั้งที่โหลดไม่ได้ถูกเปิดหรือปิด ทำให้ข้อมูลสถานะไม่น่าเชื่อถือ ภาพที่ 9 แสดงข้อมูลจำลองที่กระแสค่อย ๆ ลดลงผ่านเกณฑ์ ซึ่งเกณฑ์เดียวทำให้สถานะเปลี่ยน 6 ครั้ง ขณะที่ Hysteresis เปลี่ยนเพียง 2 ครั้ง

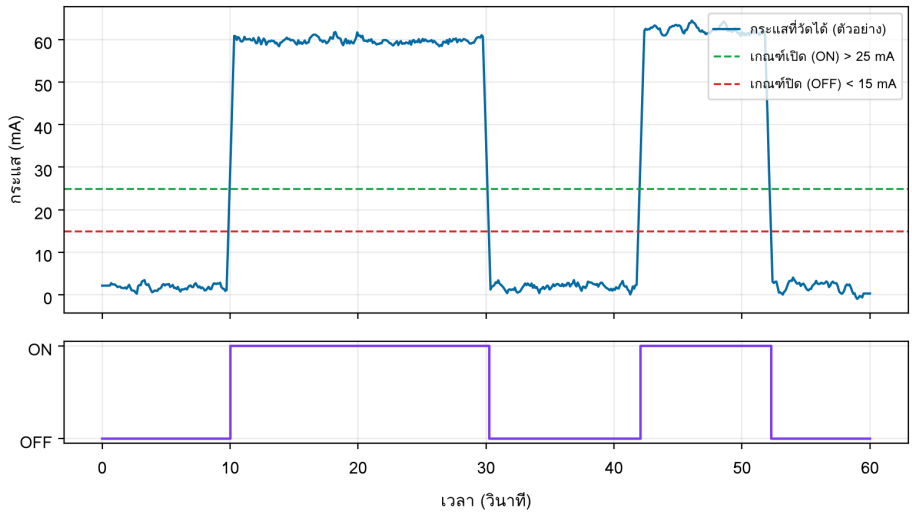



**ภาพที่ 9 เปรียบเทียบเกณฑ์เดียวกับ Hysteresis (ข้อมูลสมมติเพื่ออธิบาย ไม่ใช่ผลการทดลอง)**

**ที่มา : ผู้จัดทำ**

### 6.2 Hysteresis แบบสองระดับ

ระบบนี้จึงใช้หลักการ Hysteresis โดยกำหนดเกณฑ์สองค่า คือ เปลี่ยนเป็น ON เมื่อกระแสมากกว่า 25 mA และเปลี่ยนเป็น OFF เมื่อกระแสน้อยกว่า 15 mA ส่วนช่วงกระแสระหว่าง 15–25 mA จะคงสถานะเดิมไว้ เช่น หากระบบอยู่ในสถานะ ON แล้วกระแสลดลงเหลือ 20 mA ระบบจะยังคงเป็น ON จนกว่ากระแสจะลดต่ำกว่า 15 mA และในทางกลับกัน หากอยู่ในสถานะ OFF ระบบจะไม่เปลี่ยนเป็น ON จนกว่ากระแสจะสูงกว่า 25 mA ช่วงกลางกว้าง 10 mA จึงเป็นระยะกันชนที่ทำให้สัญญาณรบกวนเล็กน้อยไม่ทำให้สถานะเปลี่ยน ทั้งนี้ค่า 25 และ 15 mA เป็นค่าเริ่มต้น ค่าที่ใช้จริงกำหนดจากการทดลองที่ 1 ให้อยู่ระหว่างกระแสสูงสุดช่วงปิดและกระแสต่ำสุดช่วงเปิดของโหลดแต่ละชนิด ดังแสดงในภาพที่ 10




**ภาพที่ 10 ภาพประกอบแนวคิด Hysteresis (ข้อมูลสมมติเพื่ออธิบาย ไม่ใช่ผลการทดลอง)**

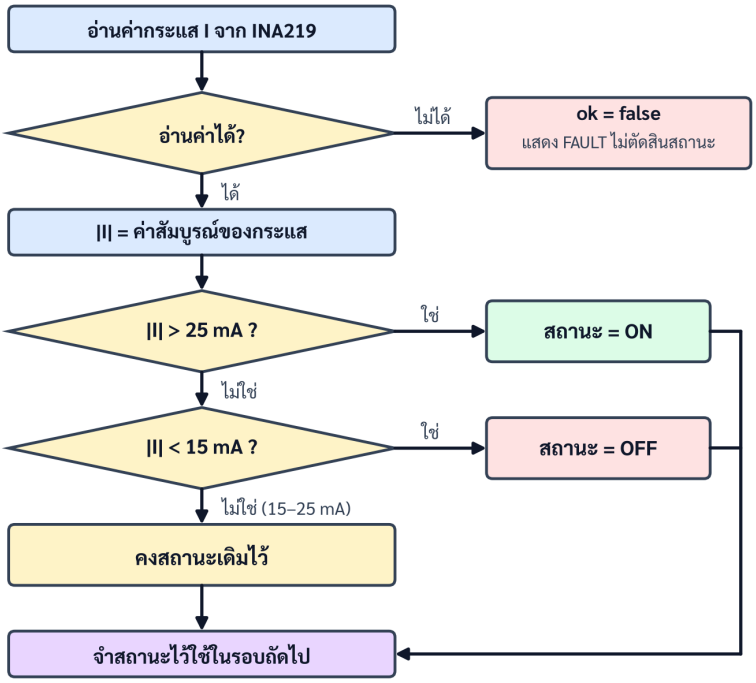
**ที่มา : ผู้จัดทำ**

### 6.3 ขั้นตอนการตัดสินสถานะ

ขั้นตอนตัดสินสถานะของ ESP32 แสดงในภาพที่ 11 ระบบใช้ค่าสัมบูรณ์ของกระแส เพราะค่าที่อ่านจาก INA219 อาจเป็นลบตามทิศทางการไหล หากอ่านค่าเซนเซอร์ไม่ได้ จะทำเครื่องหมาย ok = false และแสดงเป็น FAULT แยกต่างหากจาก OFF เพื่อไม่ให้ความผิดพลาดของเซนเซอร์ถูกตีความว่าอุปกรณ์ปิด เมื่ออ่านค่าได้ ขั้นตอนตัดสินเขียนเป็นรหัสเทียมได้ดังนี้

```
if (abs(I) > 25) state = ON;
else if (abs(I) < 15) state = OFF;
else state = previous_state;
```

ควรแยกความหมายของ OFF กับ N/A ให้ชัดเจน OFF หมายถึงอ่านค่าได้และกระแสต่ำกว่าเกณฑ์ ส่วน N/A หมายถึงไม่ได้รับข้อมูลใหม่จากอุปกรณ์เกิน 30 วินาที ซึ่งระบบไม่สามารถสรุปสถานะของอุปกรณ์ไฟฟ้าได้




**ภาพที่ 11 ขั้นตอนการตัดสินสถานะด้วย Hysteresis**

**ที่มา : ผู้จัดทำ**

## 7. เว็บไซต์บน GitHub Pages

GitHub Pages เป็นบริการสำหรับเผยแพร่เว็บไซต์แบบสถิตจาก Repository บน GitHub โครงงานนี้ใช้เผยแพร่หน้าแนะนำโครงงานและหน้า Dashboard สำหรับแสดงข้อมูลสถานะของอุปกรณ์ โดยหน้า Dashboard ทำหน้าที่ร้องขอข้อมูลจาก Cloudflare Workers ทุก 5 วินาที และนำข้อมูลที่ได้รับมาแสดงผลบนเว็บไซต์ หากข้อมูลไม่มีการอัปเดตเกิน 30 วินาที จะแสดงสถานะ N/A เพื่อระบุว่าไม่มีข้อมูลใหม่จากอุปกรณ์

เหตุผลที่เลือกให้เว็บไซต์สอบถามข้อมูลเป็นรอบ (Polling) แทนการเชื่อมต่อค้างไว้ คือ ทำงานได้บนเว็บไซต์แบบสถิตโดยไม่ต้องมีเซิร์ฟเวอร์คอยรักษาการเชื่อมต่อ และมีโครงสร้างง่ายต่อการตรวจสอบ ข้อจำกัดคือความหน่วงที่ผู้ใช้เห็นประกอบด้วยเวลาที่ ESP32 ส่งข้อมูลและช่วง Polling ของเว็บไซต์ ซึ่งโครงงานนี้วัดในการทดลองที่ 4

## 8. งานวิจัยที่เกี่ยวข้อง

มีโครงงานที่เกี่ยวข้องดังนี้

**1. วิโรจน์ ขนเกษม (2566)** ได้ศึกษาการพัฒนาระบบติดตามและแจ้งเตือนสถานะการทำงานของเครื่องอัดขึ้นรูปพิมพ์ร้อนด้วยเทคโนโลยีอินเทอร์เน็ตของสรรพสิ่ง เพื่อเฝ้าติดตามและป้องกันความเสียหายในกระบวนการผลิตตามเวลาจริง ระบบดังกล่าวใช้เซนเซอร์ PZEM-004T, NodeMCU ESP8266, คลาวด์แพลตฟอร์ม NETPIE และระบบแจ้งเตือนผ่าน LINE Application ซึ่งมีความเกี่ยวข้องกับการประยุกต์ใช้ IoT ร่วมกับเซนเซอร์วัดพลังงานไฟฟ้าในการติดตามสถานะเครื่องจักรแบบออนไลน์ เมื่อเปรียบเทียบกับโครงงานนี้ พบว่าทั้งสองระบบมีแนวคิดร่วมกันในการใช้เซนเซอร์ตรวจวัดค่าทางไฟฟ้าและส่งข้อมูลผ่านเครือข่ายอินเทอร์เน็ต อย่างไรก็ตาม โครงงานนี้มุ่งเน้นการจำแนกสถานะเปิด-ปิดของอุปกรณ์จากค่ากระแสไฟฟ้าโดยใช้เกณฑ์ Hysteresis รวมถึงการจัดการกรณีที่อุปกรณ์ขาดการเชื่อมต่อและการจัดเก็บประวัติข้อมูล

**2. พิมลดา ค้ำดินพิทักษ์, ธัญลักษณ์ กัลปดี และเอกชัย แน่นอุดร (2567)** ได้ศึกษาเรื่อง การพัฒนาระบบการตรวจสอบและการจัดการเครื่องสำรองไฟฟ้า (UPS Monitoring and Management System) โดยมีวัตถุประสงค์เพื่อพัฒนาระบบสำหรับเฝ้าระวังและติดตามสถานะการทำงานของเครื่องสำรองไฟฟ้า (UPS) รวมถึงตรวจวัดพารามิเตอร์ทางไฟฟ้าและสภาพแวดล้อมภายในห้องศูนย์ข้อมูล (Data Center) แบบเรียลไทม์ เพื่อช่วยลดความเสี่ยงจากเหตุการณ์ไฟฟ้าขัดข้อง งานวิจัยดังกล่าวใช้คอมพิวเตอร์บอร์ดเดี่ยว Orange Pi Zero 3 ร่วมกับโมดูลสื่อสารและอุปกรณ์เชื่อมต่อสัญญาณ เพื่อรวบรวมข้อมูลจากเครื่องสำรองไฟฟ้าและสภาพแวดล้อม โดยข้อมูลที่ตรวจวัดประกอบด้วยแรงดันไฟฟ้าขาเข้าและขาออก อุณหภูมิ ความชื้นสัมพัทธ์ และสถานะความผิดปกติของระบบไฟฟ้า จากนั้นนำข้อมูลมาประมวลผลและแสดงผลผ่านเว็บแดชบอร์ด

ผลการศึกษาพบว่าระบบสามารถรับข้อมูลพารามิเตอร์ทางไฟฟ้าและสภาพแวดล้อมผ่านโปรโตคอลการสื่อสาร นำมาประมวลผล และแสดงผลบนเว็บแดชบอร์ดได้ตามเงื่อนไขที่กำหนด นอกจากนี้ยังมีการทดสอบการรับส่งข้อมูลผ่านเครือข่ายและระบบแจ้งเตือนตามเกณฑ์การทดสอบของผู้วิจัย ทั้งนี้ เอกสารต้นฉบับไม่ได้ระบุค่าความแม่นยำทางสถิติหรือค่าความคลาดเคลื่อนของเซนเซอร์ไว้อย่างชัดเจน

งานวิจัยนี้มีความเกี่ยวข้องกับโครงงานของผู้จัดทำในด้านการใช้คอมพิวเตอร์บอร์ดเดี่ยวในการรวบรวมและประมวลผลข้อมูลจากอุปกรณ์ไฟฟ้า ก่อนส่งข้อมูลไปแสดงผลผ่านระบบออนไลน์ อย่างไรก็ตาม งานวิจัยดังกล่าวมุ่งเน้นการตรวจสอบเครื่องสำรองไฟฟ้าและสภาพแวดล้อมภายในห้องศูนย์ข้อมูล ในขณะที่โครงงานนี้มุ่งเน้นการตรวจจับสถานะเปิด-ปิดของอุปกรณ์จากค่ากระแสไฟฟ้าที่ตรวจวัดได้จริง

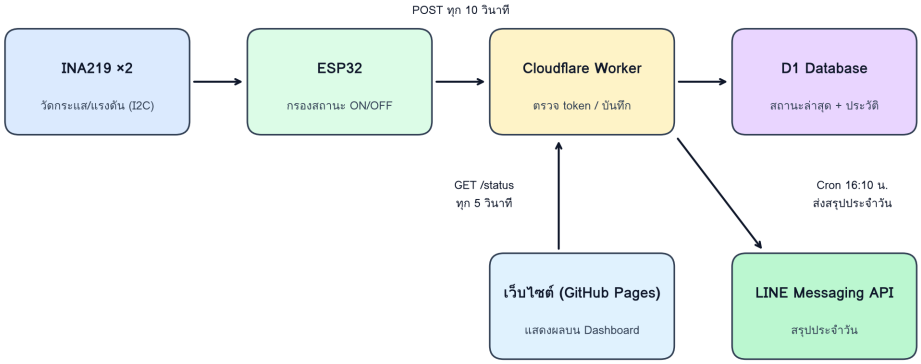
เมื่อพิจารณางานวิจัยที่เกี่ยวข้องทั้งสองเรื่อง พบว่ามีแนวคิดร่วมกันในการประยุกต์ใช้เทคโนโลยี IoT เพื่อรวบรวมข้อมูลจากอุปกรณ์ไฟฟ้าและนำไปแสดงผลผ่านระบบออนไลน์ โครงงานนี้จึงนำแนวคิดดังกล่าวมาพัฒนาต่อยอด โดยเพิ่มการจำแนกสถานะด้วยเกณฑ์ Hysteresis แบบสองระดับ เพื่อลดการเปลี่ยนสถานะที่ผิดพลาดจากความผันผวนของกระแสไฟฟ้า รวมถึงการแสดงสถานะ N/A เมื่อไม่ได้รับข้อมูลภายในระยะเวลาที่กำหนด การจัดเก็บประวัติการทำงานผ่าน Cloudflare D1 และการส่งข้อความสรุปการใช้งานผ่าน LINE Messaging API เป็นประจำทุกวัน ซึ่งช่วยให้สามารถตรวจสอบและนำข้อมูลย้อนหลังไปวิเคราะห์พฤติกรรมการใช้พลังงานไฟฟ้าได้

## 9. กรอบแนวคิดการวิจัย

จากการศึกษาเอกสารและทฤษฎีที่เกี่ยวข้อง ผู้จัดทำได้กำหนดกรอบแนวคิดการทำงานของระบบตามลำดับการไหลของข้อมูล โดยแบ่งออกเป็น 4 ส่วนหลัก ดังนี้

1. **ส่วนตรวจวัด (Sensing)** ใช้เซนเซอร์ INA219 ในการตรวจวัดแรงดันและกระแสไฟฟ้า เพื่อนำค่าที่ได้ไปใช้ในการประมวลผลและคำนวณกำลังไฟฟ้า
2. **ส่วนประมวลผลและเชื่อมต่อ (Processing and Connectivity)** ใช้ไมโครคอนโทรลเลอร์ ESP32 ประมวลผลค่าที่ได้รับจาก INA219 จำแนกสถานะของอุปกรณ์ด้วยหลักการ Hysteresis และจัดการส่งข้อมูลผ่านเครือข่าย Wi-Fi
3. **ส่วนบริการและจัดเก็บข้อมูล (Backend and Database)** ใช้ Cloudflare Workers ในการรับและประมวลผลข้อมูลจาก ESP32 และใช้ Cloudflare D1 สำหรับจัดเก็บข้อมูลสถานะและประวัติการทำงาน
4. **ส่วนแสดงผลและแจ้งเตือน (Output)** ใช้เว็บไซต์ที่เผยแพร่ผ่าน GitHub Pages สำหรับแสดงสถานะและข้อมูลการทำงานของอุปกรณ์ และใช้ LINE Messaging API สำหรับส่งข้อความสรุปการใช้งานประจำวัน

ดังแสดงในภาพที่ 12




**ภาพที่ 12 แผนภาพการทำงานของระบบ**

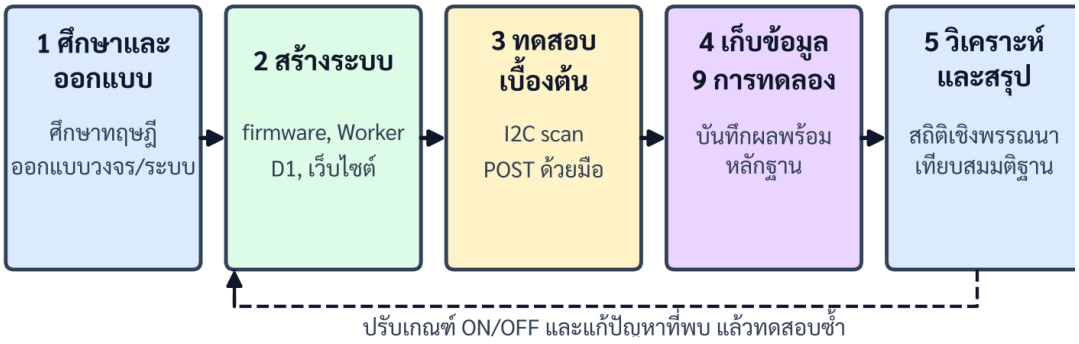
**ที่มา : ผู้จัดทำ**

# บทที่ 3

# วิธีการดำเนินงาน

โครงงานนี้เป็นการวิจัยและพัฒนาเชิงทดลอง แบ่งเป็นขั้นตอนออกแบบระบบ สร้างและทดสอบเบื้องต้น เก็บข้อมูลเพื่อปรับเกณฑ์ และวิเคราะห์ความแม่นยำ

การดำเนินงานแบ่งเป็น 5 ขั้นตอนต่อเนื่องกัน ดังแสดงในภาพที่ 13 โดยขั้นตอนที่ 5 อาจย้อนกลับไปปรับระบบได้ เช่น เมื่อผลการทดลองที่ 1 แสดงว่าเกณฑ์เริ่มต้นไม่เหมาะกับโหลดบางชนิด ผู้จัดทำจะปรับเกณฑ์แล้วทดสอบซ้ำก่อนสรุปผล




**ภาพที่ 13 ขั้นตอนการดำเนินงานของโครงงาน**

**ที่มา : ผู้จัดทำ**

ขั้นตอนที่ 1 ศึกษาและออกแบบ ศึกษาเอกสารและทฤษฎีตามบทที่ 2 ศึกษาและแกะวงจรของแผงวงจรขยาย ESP32 ที่มีจำหน่ายทั่วไป แล้วกำหนดโครงสร้างของระบบเป็น 4 ส่วนตามกรอบแนวคิด คือ ส่วนตรวจวัด ส่วนประมวลผลและเชื่อมต่อ ส่วนบริการและจัดเก็บข้อมูล และส่วนแสดงผลและแจ้งเตือน

ขั้นตอนที่ 2 สร้างระบบ เขียนเฟิร์มแวร์ ESP32 ส่วนหลังบ้านบน Cloudflare Worker ฐานข้อมูล D1 และเว็บไซต์บน GitHub Pages รวมทั้งออกแบบและสร้างแผงวงจรของผู้จัดทำเอง

ขั้นตอนที่ 3 ทดสอบเบื้องต้น ตรวจว่า INA219 ทั้งสองตัวตอบที่อยู่ 0x40 และ 0x41 ด้วยโปรแกรมสแกน I2C ทดสอบส่งข้อมูลเข้า Worker ทั้งกรณีปกติและกรณีผิดพลาด และทดสอบหน้าเว็บกับข้อมูลที่ส่งจากโปรแกรมจำลอง ESP32

ขั้นตอนที่ 4 เก็บข้อมูล ดำเนินการทดลองทั้ง 9 ชุดตามหัวข้อที่ 3 ของบทนี้ และบันทึกผลพร้อมหลักฐานภาพถ่ายหรือวิดีโอทุกครั้ง

ขั้นตอนที่ 5 วิเคราะห์และสรุป วิเคราะห์ข้อมูลด้วยสถิติเชิงพรรณนาตามหัวข้อที่ 4 และ 5 เทียบกับสมมติฐานที่ตั้งไว้ในบทที่ 1 แล้วรายงานผลตามความเป็นจริงในบทที่ 4

**ตัวแปรที่ใช้ในการวิจัย**

ตัวแปรต้น ได้แก่ ชนิดของโหลดไฟฟ้า ระดับกระแสที่ไหลผ่านโหลด และวิธีตัดสินสถานะ (เกณฑ์เดียวหรือ Hysteresis) ตัวแปรตาม ได้แก่ ความถูกต้องในการตรวจจับสถานะ ความคลาดเคลื่อนของกระแส ความหน่วงเวลาที่หน้าเว็บแสดงผล จำนวนครั้งที่สถานะเปลี่ยนผิดพลาด และคะแนนความพึงพอใจของผู้ทดสอบ ตัวแปรควบคุม ได้แก่ แรงดันของแหล่งจ่ายในแต่ละชุดการทดลอง ค่าความต้านทานชันต์ของโมดูล INA219 ช่วงเวลาส่งข้อมูล 10 วินาที ช่วง polling ของเว็บไซต์ 5 วินาที เครือข่าย Wi-Fi ที่ใช้ และมัลติมิเตอร์เครื่องเดียวกันตลอดการทดลอง

**1. ประชากรและกลุ่มตัวอย่าง**

ประชากร ได้แก่ อุปกรณ์ไฟฟ้ากระแสตรงแรงดันต่ำที่สามารถนำมาทดสอบร่วมกับระบบที่พัฒนาขึ้นได้ กลุ่มตัวอย่างได้แก่ อุปกรณ์ไฟฟ้ากระแสตรงแรงดัน 3–9 โวลต์ จำนวน 1–4 ชนิด ที่ผู้จัดทำเลือกแบบเจาะจง (Purposive Sampling) ให้ครอบคลุมช่วงกระแสต่ำถึงสูง ได้แก่

1. SG90 RC Mini Servo Motor
2. MG995 Full Metal Gear High Torque Servo Motor

สำหรับการทดสอบการใช้งานหน้าเว็บไซต์ กลุ่มตัวอย่างคือผู้ทดสอบจำนวน 10 คน ที่ผู้จัดทำเลือกแบบเจาะจงเพื่อทดสอบการใช้งานและความสะดวกสบาย

เหตุผลที่เลือกกลุ่มตัวอย่างแบบเจาะจง เนื่องจากโครงงานนี้ต้องการทดสอบการทำงานของระบบกับโหลดที่ทราบลักษณะกระแสแน่ชัดและสามารถเปิด-ปิดซ้ำได้ตามต้องการ ไม่ได้ต้องการสรุปไปยังอุปกรณ์ไฟฟ้าทุกชนิด การเลือกโหลดใช้เกณฑ์ 4 ประการ คือ 1) ใช้แรงดันกระแสตรง 3–9 โวลต์ ซึ่งปลอดภัยต่อผู้ทดลองและอยู่ในย่านที่แผงวงจรรองรับ 2) กระแสขณะทำงานอยู่ในช่วงที่ INA219 วัดได้ 3) มีลักษณะกระแสแตกต่างกัน ทั้งกระแสนิ่งและกระแสที่เปลี่ยนตามภาระ เช่น มอเตอร์เซอร์โว และ 4) เปิด-ปิดได้ทันทีโดยไม่ต้องรอ

โหลดที่ใช้ทดสอบหลักคือ SG90 และ MG995 ซึ่งมีกระแสขณะหยุดนิ่งต่ำและกระแสสูงชั่วขณะเมื่อเริ่มหมุนหรือรับภาระ จึงเหมาะกับการศึกษาการจำแนกสถานะด้วยเกณฑ์สองระดับและการวิเคราะห์ Peak Current ตามวัตถุประสงค์ข้อ 6 ส่วนโหลดชนิดอื่นที่เพิ่มในการทดลองจริง ผู้จัดทำจะระบุชื่อรุ่นและแรงดันใช้งานไว้ในตารางผลการทดลองที่เกี่ยวข้อง [ระบุชนิดโหลดเพิ่มเติม ถ้ามี]

ผู้ทดสอบการใช้งานเว็บไซต์ 10 คน เป็นบุคคลที่ไม่ได้มีส่วนร่วมในการพัฒนาระบบ เพื่อให้สะท้อนมุมมองของผู้ใช้ทั่วไปมากกว่าผู้พัฒนา ผู้จัดทำแจ้งวัตถุประสงค์ของการทดสอบก่อนเริ่ม และไม่เก็บข้อมูลที่ระบุตัวตนของผู้ทดสอบ เช่น ชื่อหรือข้อมูลติดต่อ บันทึกเพียงผลการทำภารกิจและคะแนนความพึงพอใจ ทั้งนี้ผลที่ได้จากกลุ่มตัวอย่างขนาดเล็กใช้ประเมินความเข้าใจง่ายของหน้าเว็บเบื้องต้นเท่านั้น ไม่สามารถสรุปแทนผู้ใช้ทุกกลุ่มได้

## 2. เครื่องมือที่ใช้ในการวิจัย

### 2.1 เครื่องมือด้านฮาร์ดแวร์ (Hardware)

**2.1.1) บอร์ด ESP32 DevKit (ESP32-WROOM-32)** ทำหน้าที่เป็นหน่วยประมวลผลหลักและเชื่อมต่อเครือข่ายไร้สาย (Wi-Fi)

**2.1.2) เซนเซอร์วัดกระแสไฟฟ้า INA219 จำนวน 2 ตัว** ที่อยู่ I2C 0x40 และ 0x41 (โดยบัดกรีจัมเปอร์ A0 ที่ตัวที่สอง) วัดกระแสโดยต่ออนุกรมกับสายไฟขั้วบวกของอุปกรณ์

**2.1.3) แผงวงจรขยาย (Expansion Board)** สำหรับต่อ ESP32 กับเซนเซอร์และอุปกรณ์อื่นๆ

**2.1.4) อุปกรณ์ประกอบวงจร** ได้แก่ แหล่งจ่ายไฟกระแสตรง สายจัมเปอร์ เบรดบอร์ด และมัลติมิเตอร์สำหรับใช้เป็นค่าอ้างอิงเปรียบเทียบความแม่นยำของเซนเซอร์

### 2.1.5) ข้อมูลจำเพาะของอุปกรณ์หลัก

INA219 ตามเอกสารข้อมูลของผู้ผลิตวัดแรงดันบัสได้ในช่วง 0–26 โวลต์ และวัดแรงดันตกคร่อมตัวต้านทานชันต์ได้สูงสุด ±320 มิลลิโวลต์ด้วย ADC ความละเอียด 12 บิต เมื่อใช้ตัวต้านทานชันต์ 0.1 โอห์มและค่าตั้งต้นของไลบรารี Adafruit (Calibration แบบ 32V 2A) จะอ่านกระแสได้ละเอียดหน่วยละ 0.1 มิลลิแอมแปร์ ใช้ไฟเลี้ยง 3.3 โวลต์ตรงกับระดับสัญญาณของ ESP32 และรองรับการสื่อสาร I2C ได้ถึงโหมด Fast (400 กิโลเฮิรตซ์) ผู้จัดทำใช้ความเร็วเริ่มต้นของไลบรารีและไม่ได้ปรับแต่งค่านี้ [ตรวจสอบตัวเลขเหล่านี้กับเอกสารข้อมูลฉบับที่ใช้อ้างอิงก่อนส่งงาน]

ESP32-WROOM-32 ทำงานที่ระดับสัญญาณ 3.3 โวลต์ ขา GPIO21 และ GPIO22 เป็นขา SDA และ SCL ตามค่าเริ่มต้นของไลบรารี Wire บนบอร์ดรุ่นนี้ จึงไม่ต้องกำหนดขาเอง การเชื่อมต่อ Wi-Fi ใช้ย่านความถี่ 2.4 กิกะเฮิรตซ์

### 2.1.6) ข้อควรระวังด้านความปลอดภัยในการทดลอง

การทดลองทั้งหมดใช้แรงดันกระแสตรงต่ำไม่เกิน 9 โวลต์เท่านั้น ไม่ต่อกับไฟบ้าน (AC) ในทุกกรณี เพราะ INA219 และแผงวงจรไม่ได้ออกแบบให้แยกวงจรจากไฟสูง ก่อนจ่ายไฟทุกครั้งผู้จัดทำตรวจขั้วบวกลบ ตรวจว่าไม่มีการลัดวงจรระหว่างสายไฟเลี้ยงกับกราวด์ และตั้งแหล่งจ่ายให้จำกัดกระแสไว้ไม่เกินค่าที่โหลดและเซนเซอร์รองรับ นอกจากนี้ต้องต่อ INA219 แบบอนุกรมกับสายไฟบวกเสมอ ห้ามต่อขนานกับโหลดเพราะจะทำให้ตัวต้านทานชันต์ได้รับกระแสเกินพิกัด

## 2.2 เครื่องมือด้านซอฟต์แวร์ (Software)

**2.2.1) โปรแกรม PlatformIO** ใช้สำหรับแปลและอัปโหลดโค้ดลงบอร์ด ESP32 (แพลตฟอร์ม pioarduino, Arduino core 3.x) พร้อมไลบรารี Adafruit INA219 และ ArduinoJson

**2.2.2) โปรแกรม LazyVim** ใช้สำหรับเขียนและปรับโครงสร้าง (Refactor) โค้ดได้ง่าย โดยใช้ PlatformIO ทำงานเบื้องหลัง

**2.2.3) Cloudflare Workers (TypeScript) และ Cloudflare D1** ใช้สำหรับรับและบันทึกข้อมูลค่ากระแสไฟฟ้าและสถานะการทำงาน เพื่อนำไปวิเคราะห์ในบทที่ 4

**2.2.4) LINE Messaging API** ใช้สำหรับส่งข้อความสรุปการใช้งานประจำวันไปยังผู้ใช้งาน

**2.2.5) Git** เป็นเครื่องมือควบคุมเวอร์ชัน ใช้บันทึกประวัติการแก้ไขโค้ดและเป็นตัวกลางระหว่างผู้พัฒนากับ GitHub โดยสั่งงานผ่านบรรทัดคำสั่ง (CLI) ได้

**2.2.6) GitHub และ GitHub Pages** ใช้เก็บโค้ดของโครงงานทั้งหมด และเผยแพร่เว็บไซต์โครงงานได้โดยไม่มีค่าใช้จ่าย

**2.2.7) โปรแกรม KiCad** ใช้วาดผังวงจร (Schematic) และออกแบบเลย์เอาต์แผงวงจรพิมพ์ (PCB) ของแผงวงจรขยาย

**2.2.8) ไลบรารีและเครื่องมือสนับสนุน**

เฟิร์มแวร์ใช้ไลบรารี Adafruit INA219 (เวอร์ชัน 1.2.3 ขึ้นไป) สำหรับอ่านเซนเซอร์ และ ArduinoJson (เวอร์ชัน 7 ขึ้นไป) สำหรับสร้างข้อมูล JSON ตามที่กำหนดไว้ในไฟล์ platformio.ini ซึ่งมีสองโปรไฟล์ คือ main สำหรับเฟิร์มแวร์ใช้งานจริง และ testing สำหรับโปรแกรมทดสอบ I2C ส่วนการเผยแพร่เว็บไซต์ใช้ GitHub Actions ที่ทำงานอัตโนมัติเมื่อมีการแก้ไฟล์ในโฟลเดอร์ frontend บนสาขา master

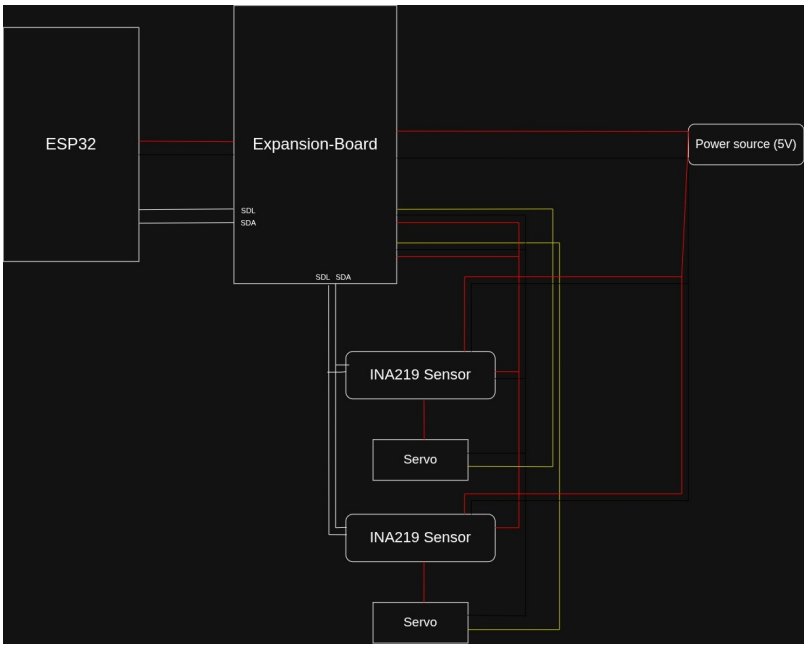
การบันทึกค่ากระแสดิบสำหรับการทดลองที่ 8 ใช้โปรแกรม ina-logger ซึ่งเป็นโปรแกรมแยกต่างหากที่อ่านค่าจาก INA219 แล้วบันทึกเป็นไฟล์ CSV เพื่อให้ได้ข้อมูลชุดเดียวกันไปจำลองการตัดสินสถานะได้ทั้งสองวิธี ส่วนการคำนวณสถิติและสร้างตารางผลใช้โปรแกรมตารางคำนวณ โดยกำหนดสูตรไว้ล่วงหน้าเพื่อไม่ให้เกิดความผิดพลาดจากการคำนวณด้วยมือ

## 2.3 ขั้นตอนการออกแบบวงจรไฟฟ้า (PCB)

ผู้จัดทำออกแบบวงจรไฟฟ้าของระบบ โดยศึกษาและแกะวงจรจากแผงวงจรขยาย ESP32 (ESP32 expansion board) ที่มีจำหน่ายทั่วไป แล้วออกแบบแผงวงจรของตนเองให้เหมาะกับระบบ แบ่งออกเป็น 4 ส่วนหลัก ดังนี้

**2.3.1) การศึกษาและแกะวงจรบอร์ดต้นแบบ** ถ่ายภาพทั้งสองด้านของแผงวงจรขยาย ESP32 ที่เลือกมาเป็นต้นแบบ ไล่เส้นทางวงจรด้วยโหมดวัดความต่อเนื่อง (Continuity) ของมัลติมิเตอร์ ระบุชนิดและค่าของอุปกรณ์แต่ละตัว เทียบกับเอกสารข้อมูล (Datasheet) แล้วบันทึกเป็นผังวงจรของ ESP32-DevKit

**2.3.2) การต่อวงจรวัดกระแส** ต่อ INA219 แบบอนุกรมที่สายไฟบวก (V+) ระหว่างแหล่งจ่ายกับโหลด โดยกระแสของโหลดไหลผ่าน VIN+ ไปยัง VIN- ใช้ไฟเลี้ยง INA219 ที่ 3.3 โวลต์ และต่อสายสื่อสาร I2C เข้ากับ ESP32 ที่ SDA = GPIO21 และ SCL = GPIO22 ดังแสดงในภาพที่ 14




**ภาพที่ 14 ผังวงจรการเชื่อมต่อ ESP32, INA219 และโหลด**

**ที่มา : ผู้จัดทำ**

**2.3.3) วงจรจ่ายไฟ (Power Supply Circuit)** รองรับการจ่ายไฟ 3 ช่องทาง ได้แก่ DC Jack, USB Micro-B และ USB Type-C โดยแต่ละช่องทางผ่านไดโอดกันกระแสย้อนกลับ (Diode-OR) เพื่อป้องกันความเสียหายเมื่อต่อแหล่งจ่ายมากกว่าหนึ่งช่องทางพร้อมกัน

**2.3.4) แผงวงจรขยาย (Expansion Board)** เชื่อมต่อกับบอร์ด ESP32 DevKit ผ่านขั้วต่อพิน (Pin Header) ประกอบด้วยจุดต่อเซนเซอร์กระแสไฟฟ้า ไฟแสดงสถานะการทำงาน (Status LED) และสวิตช์สำหรับเลือกแรงดันไฟฟ้าขาออก (3.3 V หรือ 5 V) เพื่อรองรับการต่อขยายกับอุปกรณ์อื่นในอนาคต ผู้จัดทำวาดผังวงจรและเลย์เอาต์ด้วยโปรแกรม KiCad แล้วนำไปสร้างเป็นแผงวงจรจริง (ไฟล์ออกแบบอยู่ในโฟลเดอร์ hardware/)

**2.3.5) ขั้นตอนตรวจสอบแผงวงจรก่อนจ่ายไฟ**

ก่อนต่อ ESP32 และ INA219 เข้ากับแผงวงจรที่สร้างขึ้น ผู้จัดทำตรวจตามลำดับต่อไปนี้ เพื่อลดความเสี่ยงที่อุปกรณ์จะเสียหายจากความผิดพลาดในการบัดกรี

1) ตรวจดูด้วยสายตาว่าจุดบัดกรีไม่เชื่อมติดกันและไม่มีเศษตะกั่วค้างระหว่างขา

2) วัดความต่อเนื่อง (Continuity) ระหว่างสายไฟเลี้ยงกับกราวด์ ต้องไม่ลัดวงจร และตรวจเส้นทาง SDA SCL ว่าต่อถึงขั้วต่อที่ถูกต้อง

3) จ่ายไฟโดยยังไม่เสียบ ESP32 แล้ววัดแรงดันขาออก 3.3 โวลต์และ 5 โวลต์ด้วยมัลติมิเตอร์ ว่าอยู่ในค่าที่คาดไว้

4) ทดสอบแหล่งจ่ายไฟแต่ละช่องทาง (DC Jack, USB Micro-B, USB Type-C) แยกกันและตรวจว่าเมื่อต่อมากกว่าหนึ่งช่องพร้อมกัน ไดโอดกันกระแสย้อนกลับทำงานโดยไม่มีแรงดันไหลย้อนไปยังแหล่งจ่ายอื่น

5) เสียบ ESP32 และ INA219 แล้วรันโปรแกรมสแกน I2C เพื่อยืนยันว่าพบอุปกรณ์ที่ 0x40 และ 0x41 จึงเริ่มทดสอบด้วยเฟิร์มแวร์จริง

### 2.3.6) การตรวจสอบที่อยู่ I2C ด้วยโปรแกรมสแกน

โปรแกรมสแกน I2C (ไฟล์ i2c_scanner.cpp ในโปรไฟล์ testing) ส่งคำขอไปยังที่อยู่ 1 ถึง 126 ทีละที่อยู่ และพิมพ์ที่อยู่ที่ตอบกลับผ่านพอร์ตอนุกรม หากพบอุปกรณ์เพียงตัวเดียวแสดงว่ายังไม่ได้บัดกรีจุด A0 ของโมดูลตัวที่สอง หากไม่พบเลยให้ตรวจสายไฟเลี้ยง สาย SDA และ SCL ซึ่งตรงกับปัญหาข้อ 2 และ 3 ที่บันทึกไว้ในบทที่ 4

## 2.4 ขั้นตอนการพัฒนาซอฟต์แวร์ของระบบ

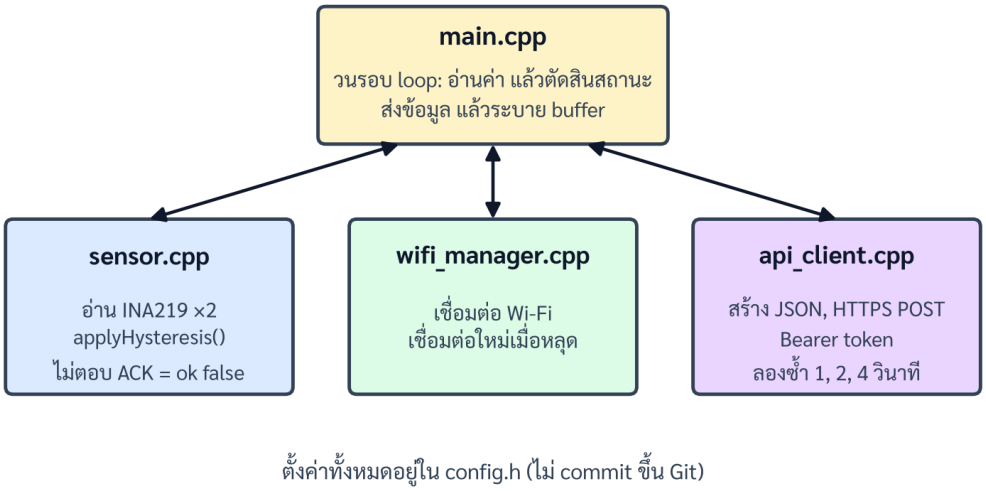
**2.4.1) เฟิร์มแวร์ ESP32** อ่านค่าแรงดัน กระแส และกำลังจาก INA219 ทุก 1 วินาที ตัดสินสถานะด้วย Hysteresis (ค่าเริ่มต้น ON เมื่อ > 25 mA และ OFF เมื่อ < 15 mA) ส่งข้อมูลแบบ JSON ผ่าน HTTPS ไปยัง Worker ทุก 10 วินาที พร้อม Bearer token ในส่วนหัว (header) หากอ่านเซนเซอร์ไม่ได้ จะรายงานว่าเซนเซอร์นั้นผิดปกติ (ok = false) ไม่รายงานเป็น OFF และหาก Wi-Fi หลุด จะพยายามเชื่อมต่อใหม่โดยไม่หยุดการอ่านค่า

**2.4.2) ส่วนหลังบ้าน (Cloudflare Worker)** POST /update ตรวจสอบ Bearer token (ปฏิเสธเมื่อไม่ตรงหรือไม่ได้ตั้งค่า) แล้วบันทึกลงตาราง readings และอัปเดตตาราง latest_status ส่วน GET /status ส่งสถานะล่าสุดของแต่ละเซนเซอร์ให้เว็บไซต์ และ Cron Trigger เวลา 09:10 UTC (16:10 น.) สรุปการใช้งานส่งผ่าน LINE พร้อมลบข้อมูล readings ที่เก่ากว่า 7 วัน

**2.4.3) หน้าเว็บไซต์** เรียก GET /status ทุก 5 วินาที แสดงสถานะ ON/OFF กระแส แรงดัน และกำลังของแต่ละเซนเซอร์ หากข้อมูลล่าสุดเก่ากว่า 30 วินาที จะแสดง N/A

**2.4.4) โครงสร้างของเฟิร์มแวร์**

เฟิร์มแวร์แบ่งเป็น 4 ไฟล์ตามหน้าที่ ดังแสดงในภาพที่ 15 เพื่อให้แก้ไขหรือทดสอบแต่ละส่วนได้โดยไม่กระทบส่วนอื่น ไฟล์ main.cpp ควบคุมวงรอบหลัก ไฟล์ sensor.cpp อ่านค่าและตัดสินสถานะ ไฟล์ wifi_manager.cpp ดูแลการเชื่อมต่อ Wi-Fi และไฟล์ api_client.cpp สร้างข้อมูล JSON แล้วส่งผ่าน HTTPS ค่าตั้งทั้งหมด เช่น ชื่อ Wi-Fi รหัสผ่าน ที่อยู่ Worker และ Bearer token เก็บไว้ในไฟล์ config.h ซึ่งไม่ถูกนำขึ้นที่เก็บโค้ด โดยมีไฟล์ config.example.h เป็นต้นแบบ




**ภาพที่ 15 โครงสร้างโมดูลของเฟิร์มแวร์ ESP32**

**ที่มา : ผู้จัดทำ**

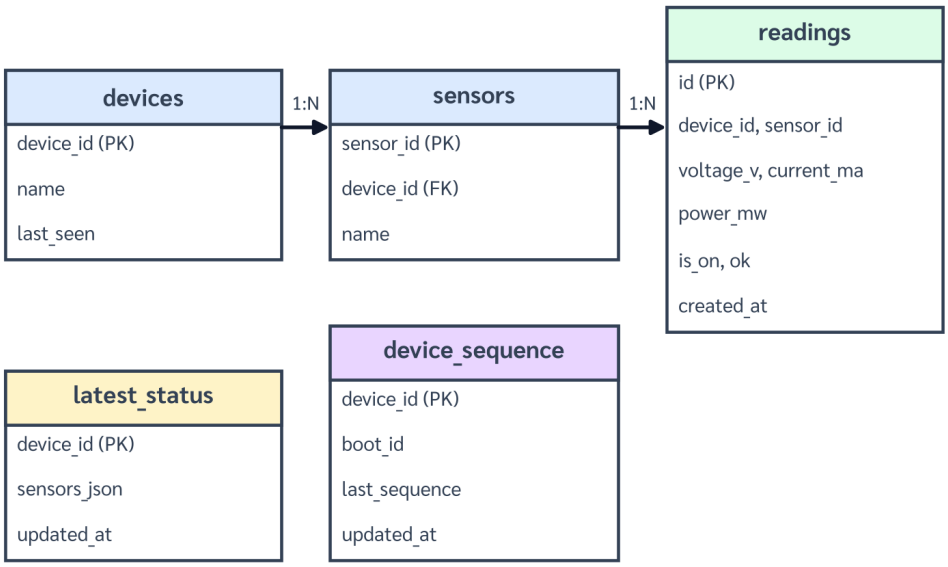
ในแต่ละรอบการอ่าน เฟิร์มแวร์ตรวจก่อนว่า INA219 ตอบสัญญาณ ACK บนบัส I2C หรือไม่ หากไม่ตอบหรือค่าที่อ่านเป็น NaN จะรายงานเซนเซอร์ตัวนั้นว่า ok เป็นเท็จทันที แทนที่จะรายงานเป็น OFF และการตั้งเวลารอ (Timeout) ของบัส I2C ไว้ที่ 50 มิลลิวินาที ช่วยไม่ให้โปรแกรมค้างเมื่อสายหลุด ในกรณีที่พบเซนเซอร์เพียงตัวเดียวตอนเริ่มต้นระบบก็ยังทำงานต่อได้ เพื่อไม่ให้ความเสียหายของเซนเซอร์หนึ่งตัวทำให้ทั้งระบบหยุด

### 2.4.5) รูปแบบข้อมูลและการตรวจสอบที่ Worker

ข้อมูลที่ ESP32 ส่งเป็น JSON ตามตัวอย่างในบทที่ 2 หัวข้อ 4.1 Worker ตรวจสอบรูปแบบก่อนบันทึกทุกครั้ง โดยต้องมี device_id เป็นข้อความที่ไม่ว่าง มี sensors เป็นอาร์เรย์ที่มีอย่างน้อยหนึ่งรายการ ทุกรายการต้องมี sensor_id เป็นข้อความที่ไม่ว่าง ค่า voltage_v current_ma และ power_mw เป็นตัวเลขจำกัดค่า และ is_on กับ ok เป็นค่าจริง/เท็จ ส่วน sequence_number ถ้ามีต้องเป็นจำนวนเต็มที่ไม่ติดลบ ข้อมูลที่ไม่ตรงรูปแบบจะถูกปฏิเสธด้วยรหัส 400 และไม่ถูกเขียนลงฐานข้อมูล การตรวจ Bearer token ใช้การเทียบค่าแบบใช้เวลาคงที่ (Constant-time Comparison) และถ้า Worker ไม่ได้ตั้งค่า token ไว้ ระบบจะปฏิเสธทุกคำขอ (Fail-closed) แทนที่จะยอมรับทุกคำขอ

### 2.4.6) โครงสร้างฐานข้อมูล

ฐานข้อมูล D1 มี 5 ตาราง ความสัมพันธ์แสดงในภาพที่ 16 ตาราง devices และ sensors เก็บข้อมูลอุปกรณ์และเซนเซอร์ โดยหนึ่งอุปกรณ์มีได้หลายเซนเซอร์ ทำให้เพิ่มเซนเซอร์ในอนาคตได้โดยไม่ต้องแก้โครงสร้าง ตาราง readings เก็บค่าที่วัดได้ทุกครั้งที่ได้รับข้อมูลพร้อมเวลาแบบ UTC และมีดัชนี (Index) ตามอุปกรณ์กับเวลา เพื่อให้ค้นประวัติเร็วขึ้น ตาราง latest_status เก็บสถานะล่าสุดของแต่ละอุปกรณ์เป็น JSON หนึ่งแถวต่ออุปกรณ์ เพื่อให้ GET /status อ่านได้เร็ว และตาราง device_sequence เก็บ boot_id กับหมายเลขลำดับล่าสุดสำหรับตรวจข้อมูลซ้ำ




**ภาพที่ 16 ความสัมพันธ์ของตารางในฐานข้อมูล D1**

**ที่มา : ผู้จัดทำ**

#### 2.4.7) การเผยแพร่และการตั้งค่าความลับ

ส่วนหลังบ้านเผยแพร่ด้วยคำสั่ง wrangler deploy โดยค่า DEVICE_TOKEN ตั้งเป็นความลับของ Worker ด้วยคำสั่ง wrangler secret put ไม่เขียนลงในซอร์สโค้ด ฐานข้อมูลต้องรันไฟล์ schema.sql กับฐานข้อมูลจริงด้วยตัวเลือก --remote มิฉะนั้นจะได้ฐานข้อมูลในเครื่องที่ Worker ใช้งานจริงมองไม่เห็น ซึ่งเป็นสาเหตุของปัญหาข้อ 1 ที่รายงานในบทที่ 4 ส่วนเว็บไซต์เผยแพร่อัตโนมัติผ่าน GitHub Actions เมื่อมีการ push ไฟล์ในโฟลเดอร์ frontend ขึ้นสาขา master

#### 2.4.8) การทดสอบซอฟต์แวร์เบื้องต้น

ผู้จัดทำทดสอบซอฟต์แวร์ก่อนเก็บข้อมูลจริง 3 ส่วน ส่วนแรกคือชุดทดสอบอัตโนมัติของ Worker (ไฟล์ worker.test.mjs) ซึ่งครอบคลุม 7 กรณี ได้แก่ POST /update ที่ไม่มี token ต้องได้รหัส 401 ข้อมูลไม่ตรงรูปแบบต้องได้ 400 ข้อมูลถูกต้องพร้อมหมายเลขลำดับและ boot_id ต้องได้ 200 การส่งหมายเลขลำดับซ้ำหรือเก่ากว่าในรอบการทำงานเดียวกันต้องได้ 409 การส่งด้วย boot_id ใหม่หลังรีบูตต้องได้ 200 และ GET /health ต้องได้ 200 เมื่อฐานข้อมูลปกติ และ 503 เมื่อฐานข้อมูลใช้ไม่ได้ ส่วนที่สองคือโปรแกรมจำลอง ESP32 (simulator.py) ที่ส่งข้อมูลเข้า Worker ตามตารางเวลาเปิด-ปิดที่กำหนด ใช้ตรวจหน้าเว็บโดยไม่ต้องต่อวงจรจริง และส่วนที่สามคือโปรแกรมสแกน I2C ที่อธิบายไว้ในหัวข้อ 2.3.6

## 3. การเก็บรวบรวมข้อมูล

ผู้จัดทำเก็บข้อมูลจากการทดลอง 9 ชุด ดังนี้ โดยบันทึกเหตุการณ์ทุกครั้งพร้อมหลักฐาน (ภาพถ่ายหรือวิดีโอ) ไว้ในโฟลเดอร์ docs/evidence ของที่เก็บโค้ด

**3.0 หลักการบันทึกข้อมูลที่ใช้ร่วมกันทุกการทดลอง**

ทุกการทดลองบันทึกข้อมูลตามรูปแบบเดียวกัน คือ วันที่และเวลา ชนิดและรุ่นของโหลด แรงดันของแหล่งจ่ายที่ตั้ง ค่าที่วัดได้ และหลักฐาน (ภาพถ่ายหรือวิดีโอที่เห็นทั้งโหลด มัลติมิเตอร์ และหน้าจอเว็บไซต์ในภาพเดียวกัน) ผู้จัดทำบันทึกผลลงไฟล์ตารางคำนวณที่แยกชีตตามการทดลอง และกำหนดสูตรคำนวณไว้ล่วงหน้า เมื่อกรอกค่าดิบแล้วค่าร้อยละ ค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน และเปอร์เซ็นไทล์จะคำนวณเอง

ข้อมูลที่ได้จากการวัดจะไม่ถูกแก้ไขหรือตัดทิ้งภายหลัง หากพบว่าการวัดครั้งใดผิดปกติ เช่น สายหลุดระหว่างทดลอง ผู้จัดทำจะบันทึกเหตุผลและทำการวัดใหม่เพิ่ม โดยเก็บค่าที่ผิดปกติไว้ในบันทึกด้วย เพื่อให้ผู้อ่านตรวจสอบย้อนหลังได้

**3.1 การทดลองที่ 1 การหาเกณฑ์ ON/OFF** ต่อโหลดแต่ละชนิดผ่าน INA219 บันทึกกระแสช่วงปิดและช่วงเปิดอย่างน้อยชนิดละ 60 วินาที แล้วเลือกเกณฑ์เปิดและเกณฑ์ปิดให้อยู่ระหว่างค่าสูงสุดช่วงปิดและค่าต่ำสุดช่วงเปิด

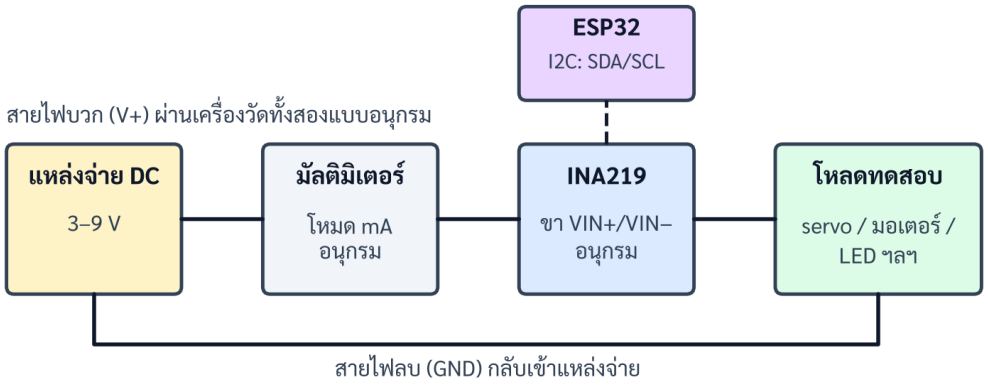
ขั้นตอนโดยละเอียด เริ่มจากต่อโหลดผ่าน INA219 แล้วเปิดระบบให้บันทึกค่า ปิดโหลดแล้วบันทึกกระแสไม่น้อยกว่า 60 วินาทีเพื่อหาค่ากระแสสูงสุดขณะปิด (OFF สูงสุด) จากนั้นเปิดโหลดและบันทึกอีกไม่น้อยกว่า 60 วินาทีเพื่อหาค่ากระแสต่ำสุดขณะเปิด (ON ต่ำสุด) สำหรับโหลดที่มีกระแสเริ่มต้นสูงชั่วขณะ เช่น เซอร์โว ให้ตัดช่วง 2 วินาทีแรกหลังเปิดออกจากการหา ON ต่ำสุด แต่เก็บค่าสูงสุดชั่วขณะ (Peak) ไว้รายงานแยกต่างหาก ทำซ้ำกับโหลดทุกชนิดแล้วเลือกเกณฑ์จากค่าทั้งสอง ดังนี้ ให้ช่องว่างระหว่างสองสถานะ (Gap) เท่ากับ ON ต่ำสุดลบ OFF สูงสุด

$$\text{เกณฑ์ปิด} = \text{OFF สูงสุด} + 0.25 \times \text{Gap} \qquad \text{เกณฑ์เปิด} = \text{ON ต่ำสุด} - 0.25 \times \text{Gap}$$

การวางเกณฑ์ที่หนึ่งในสี่ของช่องว่างจากแต่ละด้านทำให้เหลือช่วงกลางกว้างครึ่งหนึ่งของ Gap เป็นระยะกันชนของ Hysteresis หากโหลดใดมี Gap ไม่เกินศูนย์ คือกระแสขณะปิดสูงสุดไม่ต่ำกว่ากระแสขณะเปิดต่ำสุด แสดงว่าไม่สามารถแยกสถานะของโหลดนั้นด้วยกระแสได้ ผู้จัดทำจะรายงานเป็นข้อจำกัดของวิธีการ ไม่ปรับเกณฑ์ให้ผลดูดีขึ้น

**3.2 การทดลองที่ 2 ความแม่นยำของกระแส** ต่อมัลติมิเตอร์อนุกรมกับ INA219 อ่านค่าพร้อมกัน ที่ระดับกระแสต่าง ๆ อย่างน้อย 10 จุด

การต่อวงจรแสดงในภาพที่ 17 ต่อมัลติมิเตอร์โหมดวัดกระแสแบบอนุกรมกับ INA219 ในสายไฟบวกเดียวกัน เพื่อให้กระแสที่ไหลผ่านเครื่องมือทั้งสองเป็นค่าเดียวกัน ปรับระดับกระแสด้วยการเปลี่ยนโหลดหรือแรงดันแหล่งจ่าย ให้ได้ไม่น้อยกว่า 10 ระดับครอบคลุมช่วงต่ำถึงสูง ที่แต่ละระดับรอให้ค่านิ่งประมาณ 5 วินาทีแล้วจึงอ่านค่าจากทั้งสองเครื่องพร้อมกัน บันทึกย่านวัด ความละเอียด และความแม่นยำที่ผู้ผลิตมัลติมิเตอร์ระบุ เพราะความคลาดเคลื่อนที่คำนวณได้มาจากทั้งเซนเซอร์และมัลติมิเตอร์ และต้องไม่นำกระแสอ้างอิงที่ใกล้ศูนย์มาคำนวณเปอร์เซ็นต์ความคลาดเคลื่อน เพราะตัวหารที่เล็กมากทำให้ค่าร้อยละสูงเกินจริง




**ภาพที่ 17 การจัดวางอุปกรณ์ในการทดลองความแม่นยำของกระแส**

**ที่มา : ผู้จัดทำ**

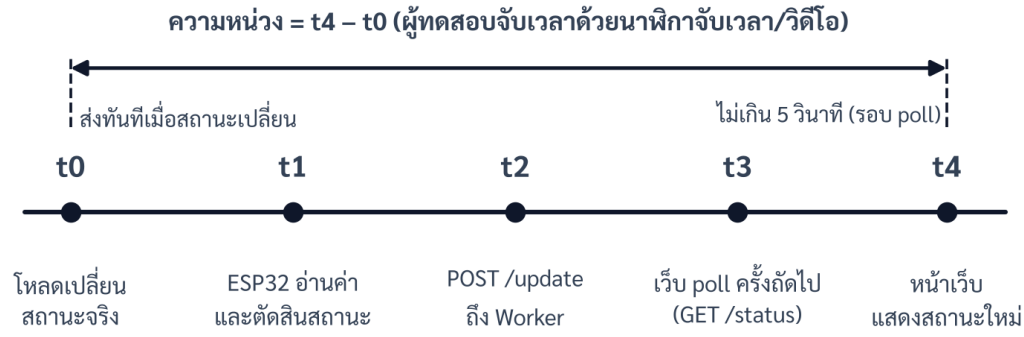
ข้อควรระวังคือมัลติมิเตอร์โหมดวัดกระแสมีความต้านทานภายในซึ่งทำให้แรงดันที่โหลดได้รับลดลงเล็กน้อย จึงควรใช้ย่านวัดที่ความต้านทานภายในต่ำที่สุดเท่าที่ค่าที่วัดอยู่ในย่านนั้นได้ และรายงานข้อสังเกตนี้ในการอภิปรายผล

**3.3 การทดลองที่ 3 ความถูกต้องของการตรวจจับ** เปิด-ปิดโหลดแต่ละชนิดอย่างน้อย 20 ครั้ง เทียบสถานะที่หน้าเว็บแสดงกับสถานะจริง

ในแต่ละครั้งผู้ทดสอบเปิดหรือปิดโหลดแล้วรอให้หน้าเว็บอัปเดตอย่างน้อย 15 วินาที จึงบันทึกสถานะที่เว็บแสดงเทียบกับสถานะจริงที่ผู้ทดสอบทราบ ลำดับการเปิดและปิดสลับกันไปจนครบ 20 ครั้งต่อโหลดแต่ละชนิด นับเป็นถูกต้องเมื่อสถานะบนเว็บตรงกับสถานะจริง และบันทึกกรณีที่หน้าเว็บแสดง N/A หรือ FAULT แยกออกมา ไม่นับรวมเป็นถูกหรือผิด เพื่อไม่ให้ความถูกต้องของการตรวจจับปนกับความพร้อมใช้งานของระบบ

**3.4 การทดลองที่ 4 ความหน่วงเวลา** จับเวลาตั้งแต่เปิด/ปิดโหลดจนหน้าเว็บแสดงสถานะใหม่ อย่างน้อย 10 ครั้ง

ช่วงเวลาที่วัดแสดงในภาพที่ 18 ผู้ทดสอบเริ่มจับเวลาเมื่อเปิดหรือปิดโหลด (t0) และหยุดเมื่อหน้าเว็บแสดงสถานะใหม่ (t4) ความหน่วงนี้ประกอบด้วยเวลาที่ ESP32 อ่านค่าและส่งข้อมูล ซึ่งเมื่อสถานะเปลี่ยนเฟิร์มแวร์จะส่งทันทีในรอบอ่านถัดไป (อ่านทุก 1 วินาที) ไม่ต้องรอรอบส่งปกติ 10 วินาที กับเวลารอรอบ polling ของเว็บไซต์ (ไม่เกิน 5 วินาที) และเวลาที่ใช้ส่งผ่านเครือข่ายและตอบกลับ การบันทึกใช้วิดีโอที่เห็นโหลดและหน้าจอพร้อมกัน แล้วอ่านเวลาจากวิดีโอ เพื่อลดความคลาดเคลื่อนจากเวลาตอบสนองของผู้ทดสอบ ทำซ้ำอย่างน้อย 10 ครั้งสำหรับการเปลี่ยนเป็น ON และอย่างน้อย 10 ครั้งสำหรับการเปลี่ยนเป็น OFF แล้วรายงานค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน ค่ากลาง และ P95




**ภาพที่ 18 ช่วงเวลาที่ใช้คำนวณความหน่วงของการแสดงสถานะ**

**ที่มา : ผู้จัดทำ**

**3.5 การทดลองที่ 5 การแสดง N/A และความเสถียร** ถอดไฟ ESP32 แล้วจับเวลาจนเว็บแสดง N/A และเปิดระบบทิ้งไว้ต่อเนื่อง 24 ชั่วโมง นับจำนวนครั้งที่ข้อมูลขาดหาย

ส่วนแรกของการทดลอง ถอดไฟเลี้ยงของ ESP32 ขณะหน้าเว็บแสดงสถานะปกติ แล้วจับเวลาจนหน้าเว็บแสดง N/A ทำซ้ำอย่างน้อย 5 ครั้ง ค่าที่คาดหวังอยู่ที่ประมาณ 30 วินาทีบวกเวลารอรอบ polling ไม่เกิน 5 วินาที ส่วนที่สอง เปิดระบบทิ้งไว้ต่อเนื่อง 24 ชั่วโมงในสภาพการใช้งานปกติ แล้วดึงประวัติจากตาราง readings มานับช่วงที่ไม่มีข้อมูลเกิน 30 วินาที บันทึกจำนวนครั้ง ระยะเวลาของแต่ละช่วง และสาเหตุที่ตรวจพบ เช่น Wi-Fi หลุดหรือไฟเลี้ยงขัดข้อง เพื่อประเมินความเสถียรและประสิทธิภาพของกลไก Offline Buffer

**3.6 การทดลองที่ 6 ข้อความสรุปทาง LINE** ตรวจว่าข้อความสรุป 16:10 น. ส่งตรงเวลาและตัวเลขตรงกับข้อมูลในฐานข้อมูล

ตรวจข้อความสรุปที่ส่งเวลา 16:10 น. เป็นเวลาอย่างน้อย 3 วันติดต่อกัน โดยบันทึกเวลาที่ข้อความมาถึงเทียบกับเวลาที่กำหนด และตรวจว่าตัวเลขในข้อความตรงกับข้อมูลที่ได้จาก GET /summary และตาราง readings ซึ่งผู้จัดทำคำนวณซ้ำด้วยตนเองจากข้อมูลดิบ เพื่อยืนยันว่าการสรุปไม่คลาดเคลื่อน [ปรับจำนวนวันตามที่ทดลองจริง]

**3.7 การทดลองที่ 7 การทดสอบแผงวงจรขยายที่ออกแบบเอง** วัดแรงดันขาออก 3.3 V และ 5 V ด้วยมัลติมิเตอร์ ทดสอบแหล่งจ่ายไฟทั้ง 3 ช่องทาง และตรวจว่า ESP32 กับ INA219 ทำงานบนแผงวงจรได้ครบ เปรียบเทียบกับบอร์ดต้นแบบ

วัดแรงดันขาออก 3.3 โวลต์และ 5 โวลต์ด้วยมัลติมิเตอร์ขณะไม่มีโหลดและขณะต่อ ESP32 แล้วเทียบกับบอร์ดต้นแบบภายใต้เงื่อนไขเดียวกัน ทดสอบแหล่งจ่ายทั้ง 3 ช่องทางแยกกัน และบันทึกว่า ESP32 เริ่มทำงาน เชื่อมต่อ Wi-Fi และอ่านค่า INA219 ได้ครบหรือไม่ในแต่ละช่องทาง ผลที่ได้ใช้ตอบวัตถุประสงค์ข้อ 4 ว่าแผงวงจรที่ออกแบบเองใช้งานแทนบอร์ดต้นแบบได้หรือไม่

**3.8 การทดลองที่ 8 เปรียบเทียบ Hysteresis กับเกณฑ์เดียว** บันทึกค่ากระแสดิบขณะเปิด-ปิดโหลดด้วยโปรแกรม ina-logger แล้วนำข้อมูลชุดเดียวกันมาจำลองการตัดสินสถานะทั้งสองวิธี นับจำนวนครั้งที่สถานะเปลี่ยนผิดพลาด (False ON/OFF)

บันทึกค่ากระแสดิบด้วย ina-logger ขณะเปิด-ปิดโหลดตามลำดับที่ผู้ทดสอบจดไว้ จากนั้นนำไฟล์ข้อมูลเดียวกันไปจำลองการตัดสินสถานะสองวิธี วิธีแรกใช้เกณฑ์เดียวที่ค่ากึ่งกลางของเกณฑ์สองระดับ (20 mA เมื่อใช้ค่าเริ่มต้น 25 และ 15 mA) วิธีที่สองใช้ Hysteresis ตามเกณฑ์ที่เลือกจากการทดลองที่ 1 นับจำนวนครั้งที่สถานะเปลี่ยนทั้งหมด และนับครั้งที่เปลี่ยนโดยไม่ตรงกับการเปิด-ปิดจริง (False ON หรือ False OFF) การใช้ข้อมูลชุดเดียวกันทำให้ความแตกต่างของผลมาจากวิธีตัดสินเท่านั้น ไม่ได้มาจากความผันผวนของการวัดคนละรอบ

**3.9 การทดลองที่ 9 การทดสอบการใช้งานหน้าเว็บไซต์ (Usability)** ให้ผู้ทดสอบทำภารกิจ 2 ข้อ คือ ดูสถานะของอุปกรณ์ และอธิบายความหมายของ N/A จากนั้นตอบแบบสอบถามความพึงพอใจมาตราส่วน 5 ระดับ

ผู้ทดสอบแต่ละคนได้รับการอธิบายเพียงว่าเว็บไซต์นี้แสดงสถานะของอุปกรณ์ไฟฟ้า ไม่ได้รับคำแนะนำเพิ่มเติม แล้วทำภารกิจที่ 1 คือบอกว่าอุปกรณ์แต่ละตัวเปิดหรือปิดอยู่ และภารกิจที่ 2 คืออธิบายความหมายของ N/A ผู้จัดทำบันทึกว่าทำสำเร็จหรือไม่ภายใน 60 วินาทีโดยไม่ต้องช่วย แล้วให้ตอบแบบสอบถามความพึงพอใจ 5 ระดับ (1 น้อยที่สุด ถึง 5 มากที่สุด) ตามแบบสอบถามในภาคผนวก จ ภารกิจที่ 2 มีความสำคัญเป็นพิเศษ เพราะสะท้อนว่าผู้ใช้เข้าใจความแตกต่างระหว่าง N/A กับ OFF หรือไม่

## 4. การวิเคราะห์ข้อมูล

**4.1** เปรียบเทียบสถานะเปิด-ปิดที่ระบบตรวจจับได้กับสถานะที่เกิดขึ้นจริง เพื่อคำนวณความถูกต้องของการตรวจจับ (Detection Accuracy)

**4.2** เปรียบเทียบค่ากระแสที่ INA219 วัดได้กับค่าที่วัดด้วยมัลติมิเตอร์ในช่วงเวลาเดียวกัน เพื่อคำนวณค่าความคลาดเคลื่อน (Error) ของเซนเซอร์

**4.3** วิเคราะห์ระยะเวลาตั้งแต่สถานะอุปกรณ์เปลี่ยนจนหน้าเว็บแสดงสถานะใหม่ และระยะเวลาที่เว็บแสดง N/A เมื่อไม่มีข้อมูล

**4.4** เปรียบเทียบจำนวนครั้งที่สถานะเปลี่ยนผิดพลาดระหว่างเกณฑ์สองระดับกับเกณฑ์เดียวจากข้อมูลกระแสชุดเดียวกัน

**4.5** ตรวจแรงดันและการทำงานของแผงวงจรที่ออกแบบ เทียบกับบอร์ดต้นแบบ

**4.6** สรุปอัตราความสำเร็จของภารกิจและคะแนนความพึงพอใจของผู้ทดสอบ

**4.7** นำเสนอผลการวิเคราะห์ในรูปแบบตารางเปรียบเทียบ เพื่อให้เห็นแนวโน้มความแม่นยำของระบบในแต่ละชนิดของอุปกรณ์

ก่อนวิเคราะห์ ผู้จัดทำตรวจความครบถ้วนของข้อมูลในแต่ละชีตว่าจำนวนครั้งที่บันทึกไม่ต่ำกว่าที่กำหนดไว้ในหัวข้อที่ 3 และแยกข้อมูลที่เซนเซอร์รายงาน ok เป็นเท็จออกไปรายงานต่างหาก ไม่นำไปคำนวณความคลาดเคลื่อนของกระแส ผลที่ได้นำไปเทียบกับสมมติฐานทั้ง 7 ข้อในบทที่ 1 โดยระบุชัดเจนว่าข้อใดเป็นจริง ไม่เป็นจริง หรือเป็นจริงเฉพาะบางเงื่อนไข

## 5. สถิติที่ใช้ในการวิเคราะห์

การวิเคราะห์ข้อมูลในโครงงานนี้ใช้สถิติเชิงพรรณนา (Descriptive Statistics) ดังนี้

**5.1 ค่าร้อยละ (Percentage)** ใช้คำนวณความถูกต้องของการตรวจจับสถานะเปิด-ปิดอุปกรณ์ไฟฟ้า จากสูตร

**ร้อยละความถูกต้อง = (จำนวนครั้งที่ตรวจจับถูกต้อง ÷ จำนวนครั้งทั้งหมด) × 100**

**5.2 ค่าเฉลี่ย (Mean)** ใช้หาค่าเฉลี่ยของกระแสและของระยะเวลาที่วัดได้จากระบบ

**5.3 ค่าความคลาดเคลื่อนร้อยละ (Percentage Error)** ใช้เปรียบเทียบค่าที่วัดได้จากระบบกับค่าอ้างอิงจากมัลติมิเตอร์ จากสูตร

**ค่าความคลาดเคลื่อนร้อยละ = |ค่าที่วัดได้ − ค่าอ้างอิง| ÷ ค่าอ้างอิง × 100**

**5.4 ส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation)** ใช้วิเคราะห์ความเสถียรของค่าที่วัดได้ในแต่ละครั้ง

**5.5 เปอร์เซ็นไทล์ (P50 และ P95)** ใช้รายงานความหน่วงเวลา โดย P50 คือค่ากลางของข้อมูล และ P95 คือค่าที่ข้อมูลร้อยละ 95 ไม่เกิน ซึ่งสะท้อนกรณีที่ช้าที่สุดได้ดีกว่าค่าเฉลี่ย

ตัวอย่างวิธีคำนวณต่อไปนี้ใช้ตัวเลขสมมติเพื่ออธิบายสูตรเท่านั้น ไม่ใช่ผลการทดลอง สมมติว่าวัดกระแสจากมัลติมิเตอร์ได้ 100.0 mA และ INA219 อ่านได้ 103.0 mA ความคลาดเคลื่อนร้อยละ = |103.0 − 100.0| ÷ 100.0 × 100 = 3.0 ซึ่งไม่เกินร้อยละ 5 ตามสมมติฐานข้อ 2 และสมมติว่าตรวจจับสถานะถูก 38 ครั้งจากการทดสอบ 40 ครั้ง ความถูกต้อง = 38 ÷ 40 × 100 = ร้อยละ 95 ซึ่งไม่น้อยกว่าร้อยละ 90 ตามสมมติฐานข้อ 1

สำหรับความหน่วงเวลา ถ้าวัดได้ 10 ค่า ให้เรียงจากน้อยไปมาก ค่า P50 คือค่ากลางของข้อมูล ส่วน P95 หาด้วยการประมาณค่าในช่วงระหว่างค่าที่เรียงอันดับ ซึ่งฟังก์ชัน PERCENTILE ของโปรแกรมตารางคำนวณคำนวณให้ ในกรณีที่มีข้อมูลเพียง 10 ค่า P95 จะใกล้เคียงค่าสูงสุดมาก จึงควรรายงานควบคู่กับค่าสูงสุดและจำนวนข้อมูลเสมอ

# บทที่ 4
# ผลการวิจัย/ผลการวิเคราะห์ข้อมูล

บทนี้รายงานผลที่ได้จากการทดลองตามบทที่ 3 โดยรายงานเฉพาะตัวเลขที่ได้จากการวัดจริง มิได้เอามาจากแหล่งใดนอกจากการทดสอบ

ในฉบับร่างนี้ ข้อมูลของการทดลองที่ 1–9 ยังอยู่ระหว่างการเก็บ ช่องที่ยังไม่มีผลจึงแสดงเป็น [รอผล] ผู้จัดทำจะกรอกเฉพาะค่าที่วัดได้จริงและแนบหลักฐาน โดยไม่ประมาณหรือสมมติตัวเลข แต่ละหัวข้อด้านล่างระบุว่าตารางแสดงอะไร เกณฑ์ที่ใช้ตัดสินคืออะไร และผู้อ่านควรดูอะไรจากผล เพื่อให้เมื่อกรอกข้อมูลครบแล้วอ่านผลเทียบกับสมมติฐานได้ทันที

การรายงานผลใช้หลักเดียวกันทุกหัวข้อ คือ รายงานจำนวนครั้งที่ทดสอบ ค่าที่วัดได้ และผลเทียบกับสมมติฐานที่เกี่ยวข้อง ถ้าผลไม่เป็นไปตามสมมติฐานก็รายงานตามที่ได้ และวิเคราะห์สาเหตุในบทที่ 5 ไม่ปรับเกณฑ์หรือเลือกข้อมูลเพื่อให้ผลผ่าน

## 1. ผลการพัฒนาระบบ

ผู้จัดทำพัฒนาระบบได้ครบทุกส่วน คือ เฟิร์มแวร์ ESP32 อ่านค่า INA219 สองตัว ส่วนหลังบ้านบน Cloudflare Worker และ D1 หน้า Dashboard และหน้าแนะนำโครงงานบน GitHub Pages และการส่งสรุปประจำวันผ่าน LINE ซอร์สโค้ดทั้งหมดเก็บไว้ที่ https://github.com/dentenplay5555/electrical-status-notification-system

ผลการพัฒนาแบ่งตามองค์ประกอบของระบบได้ดังนี้

1.1) เฟิร์มแวร์ ESP32 อ่านค่าแรงดัน กระแส และกำลังจาก INA219 สองตัวที่อยู่ 0x40 และ 0x41 ตัดสินสถานะด้วย Hysteresis ตรวจจับเซนเซอร์ที่ไม่ตอบสนองและรายงานเป็น ok เป็นเท็จ ส่งข้อมูลทุก 10 วินาทีพร้อมหมายเลขลำดับและ boot_id เก็บข้อมูลลง Offline Buffer ความจุ 20 ชุดเมื่อส่งไม่สำเร็จ และเชื่อมต่อ Wi-Fi ใหม่เองเมื่อสัญญาณหลุด

1.2) ส่วนหลังบ้านบน Cloudflare Worker มี Endpoint 5 กลุ่ม ได้แก่ POST /update รับข้อมูลจากอุปกรณ์ GET /status ให้เว็บอ่านสถานะล่าสุด GET /health ตรวจสถานะของระบบและฐานข้อมูล GET /summary และ GET /history สำหรับสรุปและดูประวัติ และหน้า /admin สำหรับผู้ดูแล นอกจากนี้มี Cron Trigger ส่งสรุปทาง LINE เวลา 16:10 น. และลบข้อมูล readings ที่เก่ากว่า 7 วัน

1.3) ฐานข้อมูล D1 มี 5 ตาราง ตามภาพที่ 16 โดยตาราง latest_status ทำให้การอ่านสถานะปัจจุบันไม่ต้องค้นประวัติทั้งหมด

1.4) เว็บไซต์ Dashboard เรียกข้อมูลทุก 5 วินาที แสดงสถานะ ON/OFF กระแส แรงดัน และกำลังของแต่ละเซนเซอร์ แสดง N/A เมื่อข้อมูลเก่าเกิน 30 วินาที มีตัวบ่งชี้การเชื่อมต่อ โหมดสว่างและมืด และหน้าตั้งค่าที่เปลี่ยนที่อยู่ Worker ได้โดยไม่ต้องแก้โค้ด

1.5) แผงวงจรขยายที่ออกแบบด้วย KiCad รองรับแหล่งจ่าย 3 ช่องทางผ่านวงจรไดโอดกันกระแสย้อนกลับ มีจุดต่อเซนเซอร์ ไฟแสดงสถานะ และสวิตช์เลือกแรงดันขาออก 3.3 หรือ 5 โวลต์ ผลการทดสอบแผงวงจรอยู่ในหัวข้อที่ 8

## 2. ผลการหาเกณฑ์ ON/OFF (การทดลองที่ 1)

การทดลองนี้ตอบคำถามว่าควรตั้งเกณฑ์เปิดและเกณฑ์ปิดที่กี่มิลลิแอมแปร์สำหรับโหลดแต่ละชนิด ตารางที่ 1 แสดงกระแสสูงสุดขณะปิดและกระแสต่ำสุดขณะเปิดของโหลดแต่ละชนิด พร้อมเกณฑ์ที่คำนวณตามสูตรในบทที่ 3 หัวข้อ 3.1

**ตารางที่ 1** ผลการวัดกระแสช่วงปิดและช่วงเปิด และเกณฑ์ที่เลือก

| อุปกรณ์ | OFF สูงสุด (mA) | ON ต่ำสุด (mA) | เกณฑ์ OFF (mA) | เกณฑ์ ON (mA) |
|---|---|---|---|---|
| อุปกรณ์ 1 | [รอผล] | [รอผล] | [รอผล] | [รอผล] |
| อุปกรณ์ 2 | [รอผล] | [รอผล] | [รอผล] | [รอผล] |

ผลการทดลอง: [รอผล สรุปว่าโหลดแต่ละชนิดมี Gap เท่าใด และเกณฑ์ 25 และ 15 mA ที่ตั้งไว้เริ่มต้นอยู่ในช่วงที่แยกสถานะได้หรือไม่]

สิ่งที่ต้องรายงานเพิ่ม คือโหลดใดที่เกณฑ์เริ่มต้นไม่เหมาะ เช่น โหลดที่กระแสขณะเปิดต่ำกว่า 25 mA จะถูกระบบมองเป็น OFF ตลอด และต้องระบุว่าเกณฑ์ที่เปลี่ยนไปนั้นเปลี่ยนที่ไฟล์ config.h ของเฟิร์มแวร์ ผลของหัวข้อนี้เป็นพื้นฐานของหัวข้อที่ 4 และ 9 เพราะทั้งสองหัวข้อใช้เกณฑ์ที่เลือกในที่นี้

## 3. ผลความแม่นยำของกระแส (การทดลองที่ 2)

ตารางที่ 2 เปรียบเทียบกระแสที่ INA219 วัดได้กับมัลติมิเตอร์ที่แต่ละระดับกระแส พร้อมความคลาดเคลื่อนร้อยละ เกณฑ์ที่ใช้เทียบคือสมมติฐานข้อ 2 ว่าไม่เกินร้อยละ 5

**ตารางที่ 2** เปรียบเทียบกระแสที่ INA219 วัดได้กับมัลติมิเตอร์

| ครั้งที่ | มัลติมิเตอร์ (mA) | INA219 (mA) | คลาดเคลื่อน (%) |
|---|---|---|---|
| 1 | [รอผล] | [รอผล] | [รอผล] |
| 2 | [รอผล] | [รอผล] | [รอผล] |
| 3 | [รอผล] | [รอผล] | [รอผล] |
| 4 | [รอผล] | [รอผล] | [รอผล] |
| 5 | [รอผล] | [รอผล] | [รอผล] |
| **เฉลี่ย** | **[รอผล]** | **[รอผล]** | **[รอผล]** |

ผลการทดลอง: [รอผล สรุปจำนวนระดับกระแสที่วัด ค่าคลาดเคลื่อนเฉลี่ย ค่าสูงสุด และส่วนเบี่ยงเบนมาตรฐาน]

ในการอ่านผลให้ดูว่าความคลาดเคลื่อนสูงเกิดที่ย่านกระแสใด หากเกิดที่กระแสต่ำ ต้องเทียบกับความละเอียดของเครื่องมือ เพราะที่ INA219 ละเอียดหน่วยละ 0.1 mA ค่าที่แตกต่างกัน 1 หน่วยที่กระแส 10 mA คิดเป็นร้อยละ 1 ขณะที่มัลติมิเตอร์บางย่านมีความละเอียดหยาบกว่านั้น ความคลาดเคลื่อนส่วนหนึ่งจึงมาจากเครื่องอ้างอิง ไม่ใช่จากเซนเซอร์เพียงอย่างเดียว

## 4. ผลความถูกต้องของการตรวจจับ (การทดลองที่ 3)

ตารางที่ 3 รายงานจำนวนครั้งที่ทดสอบ จำนวนครั้งที่หน้าเว็บแสดงสถานะตรงกับสถานะจริง และร้อยละความถูกต้องของโหลดแต่ละชนิด เกณฑ์ที่ใช้เทียบคือสมมติฐานข้อ 1 ว่าไม่น้อยกว่าร้อยละ 90

**ตารางที่ 3** ความถูกต้องในการตรวจจับสถานะเปิด-ปิด

| อุปกรณ์ | จำนวนครั้งทั้งหมด | ตรวจจับถูกต้อง | ร้อยละความถูกต้อง |
|---|---|---|---|
| อุปกรณ์ 1 | [รอผล] | [รอผล] | [รอผล] |
| อุปกรณ์ 2 | [รอผล] | [รอผล] | [รอผล] |
| อุปกรณ์ 3 | [รอผล] | [รอผล] | [รอผล] |
| **รวม** | **[รอผล]** | **[รอผล]** | **[รอผล]** |

ผลการทดลอง: [รอผล สรุปร้อยละความถูกต้องรวมและแยกตามโหลด และสถานะที่ผิดเป็นแบบ False ON หรือ False OFF]

ควรรายงานแยกว่าความผิดพลาดมักเกิดกับการเปลี่ยนไปเป็นสถานะใด และเกิดกับโหลดชนิดใด เพราะข้อมูลนี้บอกว่าควรปรับเกณฑ์หรือควรเพิ่มวิธีกรองสัญญาณ ส่วนครั้งที่หน้าเว็บแสดง N/A หรือ FAULT ให้รายงานแยกต่างหากตามที่กำหนดในบทที่ 3

## 5. ผลความหน่วงเวลา (การทดลองที่ 4)

ตารางที่ 4 แสดงเวลาตั้งแต่โหลดเปลี่ยนสถานะจนหน้าเว็บแสดงสถานะใหม่ ทั้งกรณีเปลี่ยนเป็น ON และเปลี่ยนเป็น OFF เกณฑ์ที่ใช้เทียบคือสมมติฐานข้อ 3 ว่าไม่เกิน 10 วินาที

**ตารางที่ 4** เวลาตั้งแต่สถานะจริงเปลี่ยนจนหน้าเว็บแสดงสถานะใหม่

| ครั้งที่ | เปลี่ยนเป็น ON (วินาที) | เปลี่ยนเป็น OFF (วินาที) |
|---|---|---|
| 1 | [รอผล] | [รอผล] |
| 2 | [รอผล] | [รอผล] |
| 3 | [รอผล] | [รอผล] |
| 4 | [รอผล] | [รอผล] |
| 5 | [รอผล] | [รอผล] |
| ค่าเฉลี่ย ± SD | [รอผล] | [รอผล] |
| P50 | [รอผล] | [รอผล] |
| **P95** | **[รอผล]** | **[รอผล]** |

ผลการทดลอง: [รอผล สรุปค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน ค่ากลาง P95 และค่าสูงสุด ทั้งสองทิศทาง]

ค่าที่ได้ควรเทียบกับค่าที่คาดจากการออกแบบ คือเมื่อสถานะเปลี่ยนเฟิร์มแวร์ส่งทันที ความหน่วงส่วนใหญ่จึงมาจากรอบ polling ของเว็บไซต์ไม่เกิน 5 วินาที หากพบค่าเกิน 10 วินาทีให้พิจารณาว่ามาจากการส่งซ้ำเมื่อเครือข่ายไม่เสถียรหรือจากสาเหตุอื่น โดยตรวจจากบันทึกทางพอร์ตอนุกรม และรายงานว่าสมมติฐานข้อ 3 เป็นจริงเฉพาะกรณีใด

## 6. ผลการแสดง N/A และความเสถียร (การทดลองที่ 5)

ตารางที่ 5 สรุปเวลาตั้งแต่ถอดไฟ ESP32 จนหน้าเว็บแสดง N/A และผลการเปิดระบบต่อเนื่อง 24 ชั่วโมง เกณฑ์ที่ใช้เทียบคือสมมติฐานข้อ 4 ว่าเมื่อไม่ได้รับข้อมูลเกิน 30 วินาทีหน้าเว็บต้องแสดง N/A

**ตารางที่ 5** ผลการทดสอบ N/A และความเสถียร

| รายการ | ผล |
|---|---|
| เวลาตั้งแต่ถอดไฟจนเว็บแสดง N/A (วินาที) | [รอผล] |
| ระยะเวลาทดสอบต่อเนื่อง (ชั่วโมง) | [รอผล] |
| จำนวนครั้งที่ข้อมูลขาดหาย / Wi-Fi หลุด | [รอผล] |

ผลการทดลอง: [รอผล สรุปเวลาที่เว็บแสดง N/A จำนวนครั้งที่ข้อมูลขาดหายใน 24 ชั่วโมง และสาเหตุ]

นอกจากเวลาที่แสดง N/A ให้รายงานด้วยว่าเมื่อต่อไฟกลับ หน้าเว็บกลับมาแสดงสถานะปกติภายในกี่วินาที และข้อมูลที่ค้างใน Offline Buffer ถูกส่งย้อนหลังครบหรือไม่ ซึ่งเป็นหลักฐานว่ากลไกป้องกันข้อมูลสูญหายทำงานจริง

## 7. ผลข้อความสรุปทาง LINE (การทดลองที่ 6)

ตารางที่ 6 ตรวจว่าข้อความสรุปประจำวันส่งตรงเวลา 16:10 น. และตัวเลขตรงกับข้อมูลในฐานข้อมูลหรือไม่

**ตารางที่ 6** ผลการตรวจข้อความสรุปประจำวัน

| รายการ | ผล |
|---|---|
| ส่งตรงเวลา 16:10 น. หรือไม่ | [รอผล] |
| ตัวเลขในข้อความตรงกับฐานข้อมูลหรือไม่ | [รอผล] |

ผลการทดลอง: [รอผล รายงานเวลาที่ข้อความมาถึงในแต่ละวันและผลเทียบตัวเลข] ข้อที่ต้องระวังคือเวลาที่ Cron Trigger ทำงานอาจคลาดเคลื่อนได้เล็กน้อย และจำนวนข้อความที่ส่งต้องอยู่ภายใต้ข้อจำกัดของบัญชี LINE ที่ใช้ ผู้จัดทำจึงส่งสรุปวันละหนึ่งครั้งตามที่ออกแบบไว้

## 8. ผลการทดสอบแผงวงจรขยายที่ออกแบบเอง (การทดลองที่ 7)

ตารางที่ 7 รายงานแรงดันขาออก 3.3 โวลต์และ 5 โวลต์ของแผงวงจรที่ผู้จัดทำสร้าง และผลการทำงานของแหล่งจ่ายแต่ละช่องทาง เทียบกับบอร์ดต้นแบบ

**ตารางที่ 7** ผลการทดสอบแผงวงจรขยายของ board หลัก

| รายการ | ผล |
|---|---|
| แรงดันขาออก 3.3 V ที่วัดได้ (โวลต์) | [รอผล] |
| แรงดันขาออก 5 V ที่วัดได้ (โวลต์) | [รอผล] |
| แหล่งจ่าย DC Jack / USB Micro-B / USB Type-C ใช้งานได้ทุกช่องทางหรือไม่ | [รอผล] |
| ESP32 และ INA219 ทำงานบนแผงวงจรได้ครบหรือไม่ | [รอผล] |
| ความแตกต่างจากบอร์ดต้นแบบที่พบ | [รอผล] |

ผลการทดลอง: [รอผล รายงานแรงดันที่วัดได้ ความคลาดเคลื่อนจากค่าที่ระบุ และผลของแหล่งจ่ายทั้ง 3 ช่องทาง] หากแผงวงจรมีข้อบกพร่อง เช่น แรงดันตกเมื่อต่อโหลด ให้บันทึกสาเหตุและวิธีแก้ไว้ในหัวข้อที่ 11 ด้วย

## 9. ผลการเปรียบเทียบ Hysteresis กับเกณฑ์เดียว (การทดลองที่ 8)

ตารางที่ 8 เปรียบเทียบจำนวนครั้งที่สถานะเปลี่ยนผิดพลาดของเกณฑ์เดียวกับเกณฑ์สองระดับ จากข้อมูลกระแสดิบชุดเดียวกัน ผลนี้ตอบวัตถุประสงค์ข้อ 5 ส่วนแรก

**ตารางที่ 8** จำนวนครั้งที่สถานะเปลี่ยนผิดพลาดของเกณฑ์เดียวและเกณฑ์สองระดับ

| รายการ | เกณฑ์เดียว | Hysteresis |
|---|---|---|
| จำนวนครั้งที่สถานะเปลี่ยนทั้งหมด | [รอผล] | [รอผล] |
| จำนวนครั้งที่เปลี่ยนผิดพลาด (False ON/OFF) | [รอผล] | [รอผล] |
| ร้อยละของการเปลี่ยนผิดพลาด | [รอผล] | [รอผล] |

ผลการทดลอง: [รอผล รายงานจำนวนครั้งที่สถานะเปลี่ยนทั้งหมดและที่ผิดพลาดของทั้งสองวิธี] ควรแนบกราฟกระแสกับสถานะที่ได้จากทั้งสองวิธีจากข้อมูลจริง เพื่อให้เห็นช่วงที่เกณฑ์เดียวเกิดการแกว่ง ภาพที่ 9 ในบทที่ 2 เป็นเพียงภาพประกอบแนวคิดจากข้อมูลสมมติ ไม่ใช่ผลของหัวข้อนี้ หากโหลดที่ทดสอบมีกระแสนิ่งอยู่ห่างจากเกณฑ์ ผลของสองวิธีอาจไม่ต่างกัน ซึ่งต้องรายงานตามจริงเช่นกัน

## 10. ผลการทดสอบการใช้งานหน้าเว็บไซต์ (การทดลองที่ 9)

ตารางที่ 9 สรุปผลการทำภารกิจ 2 ข้อของผู้ทดสอบ 10 คน และคะแนนความพึงพอใจ

**ตารางที่ 9** ผลการทดสอบการใช้งานหน้าเว็บไซต์

| รายการ | ผล |
|---|---|
| จำนวนผู้ทดสอบ (คน) | [รอผล] |
| ดูสถานะของอุปกรณ์ได้สำเร็จ (คน) | [รอผล] |
| อธิบายความหมายของ N/A ได้ถูกต้อง (คน) | [รอผล] |
| คะแนนความพึงพอใจเฉลี่ย (เต็ม 5) | [รอผล] |
| ข้อเสนอแนะจากผู้ทดสอบ | [รอผล] |

ผลการทดลอง: [รอผล รายงานจำนวนผู้ที่ทำภารกิจสำเร็จ และค่าเฉลี่ยกับส่วนเบี่ยงเบนมาตรฐานของคะแนนความพึงพอใจ] หากผู้ทดสอบบางคนตอบความหมายของ N/A ไม่ถูกต้อง ให้รายงานคำตอบที่พบและเสนอการปรับข้อความหรือการออกแบบหน้าเว็บในข้อเสนอแนะ

## 11. ปัญหาที่พบและวิธีแก้ไข

ปัญหาที่พบระหว่างการพัฒนาและวิธีแก้ไข มีดังนี้

1. Worker ตอบ error 1101 และ /update ล้มเหลว (no such table) เพราะรัน schema กับฐานข้อมูลในเครื่อง ไม่ใช่ฐานข้อมูลจริง แก้ไขโดยรัน schema ใหม่ด้วย --remote (แก้แล้ว)

2. ESP32 ไม่พบ INA219 ที่ที่อยู่ 0x40 เพราะต่อสาย SDA และ SCL สลับกัน แก้ไขโดยต่อ SDA = GPIO21 และ SCL = GPIO22 (แก้แล้ว)

3. ไม่พบ INA219 ตัวที่สองที่ 0x41 เพราะยังไม่ได้ต่อขา A0 เพื่อเปลี่ยนที่อยู่

แก้ไขโดยบัดกรีจุด A0 ของโมดูลตัวที่สองให้เชื่อมกับ VS+ แล้วสแกน I2C ซ้ำจนพบที่อยู่ 0x40 และ 0x41 ทั้งคู่ (สถานะ: [ยืนยันเมื่อบัดกรีและทดสอบแล้ว])

**ข้อจำกัดของระบบที่ทราบ**

นอกจากปัญหาที่พบระหว่างพัฒนา ผู้จัดทำสรุปข้อจำกัดของระบบจากการออกแบบและการตรวจสอบโค้ด เพื่อให้ผู้อ่านตีความผลทดลองได้ถูกต้อง

1) Offline Buffer เก็บอยู่ในหน่วยความจำ RAM ของ ESP32 ข้อมูลที่ค้างอยู่ในบัฟเฟอร์จะหายหากบอร์ดรีเซ็ตหรือไฟดับ และความจุ 20 ชุดรองรับการขาดการเชื่อมต่อได้ประมาณ 3 นาทีที่รอบส่ง 10 วินาที

2) กลไกหมายเลขลำดับและ boot_id ป้องกันข้อมูลซ้ำหรือข้อมูลเก่าที่เกิดจากความผิดพลาดของระบบ แต่ไม่ใช่การพิสูจน์ตัวตนด้วยการเข้ารหัส ผู้ที่ทราบ Bearer token ยังสามารถส่งข้อมูลปลอมได้ จึงต้องเก็บ token เป็นความลับ

3) Endpoint สำหรับอ่านสถานะ (GET /status) เปิดให้อ่านได้โดยไม่ต้องใช้ token เพื่อให้เว็บไซต์แบบสถิตเรียกได้ จึงเหมาะกับข้อมูลที่ไม่เป็นความลับเท่านั้น

4) ค่า N/A ตัดสินจากอายุของข้อมูลล่าสุดเทียบกับ 30 วินาที ซึ่งขึ้นกับนาฬิกาของเครื่องที่เปิดเว็บและนาฬิกาของเซิร์ฟเวอร์ หากนาฬิกาเครื่องผู้ใช้คลาดเคลื่อนมาก การแสดง N/A อาจเร็วหรือช้ากว่าความจริง

5) การทดสอบทำกับโหลดกระแสตรงแรงดัน 3–9 โวลต์ กระแสไม่เกินช่วงที่ตัวต้านทานชันต์ 0.1 โอห์มรองรับเท่านั้น ผลจึงไม่สามารถนำไปสรุปกับไฟบ้าน (AC) ได้ ซึ่งต้องใช้เซนเซอร์และวงจรแยกไฟต่างหาก

6) ความหน่วงที่ผู้ใช้เห็นมาจากรอบ polling 5 วินาทีเป็นหลักและเพิ่มขึ้นเมื่อการส่งต้องลองซ้ำ จึงไม่เหมาะกับงานที่ต้องการแจ้งเตือนทันทีในระดับเสี้ยววินาที

7) กลุ่มตัวอย่างโหลดและผู้ทดสอบมีขนาดเล็ก ผลสรุปได้เฉพาะในขอบเขตที่ทดสอบ

## 12. สรุปผลการทดลองเทียบกับสมมติฐาน

เมื่อการทดลองที่ 1–9 ครบ ผู้จัดทำจะรวมผลของแต่ละการทดลองไว้ในตารางเดียวเพื่อให้เห็นภาพรวมว่าสมมติฐานทั้ง 7 ข้อในบทที่ 1 เป็นจริงหรือไม่ ตารางที่ 10 แสดงเกณฑ์ของแต่ละข้อ ส่วนคอลัมน์ผลที่วัดได้และผลจะกรอกจากตารางที่ 1–9 ของบทนี้เท่านั้น โดยไม่ใช้ตัวเลขจากแหล่งอื่น

**ตารางที่ 10** สรุปผลการทดลองเทียบกับสมมติฐาน

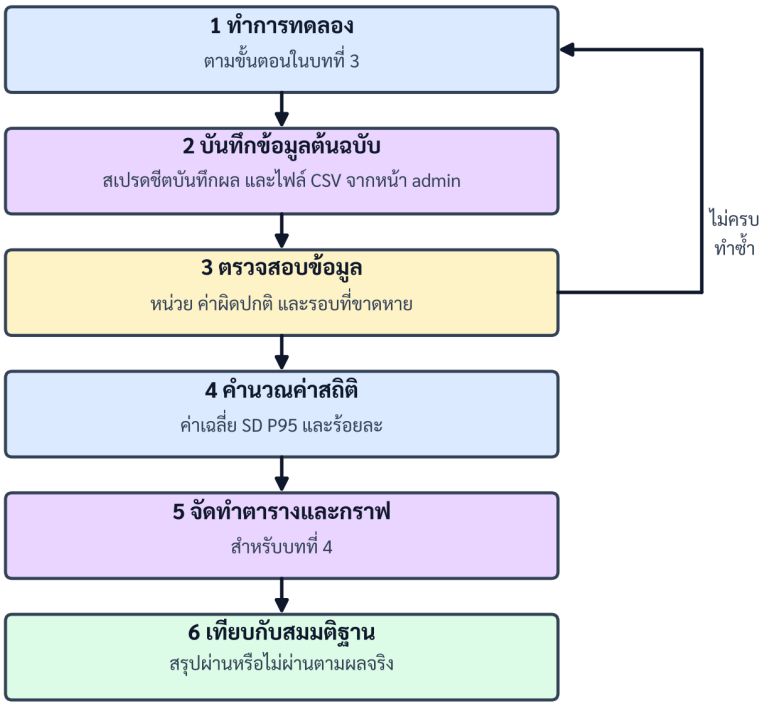
| สมมติฐาน | เกณฑ์ | ผลที่วัดได้ | ผล |
|---|---|---|---|
| ข้อ 1 ตรวจจับสถานะถูกต้อง | ไม่น้อยกว่าร้อยละ 90 | [รอผล] | [รอผล] |
| ข้อ 2 ความคลาดเคลื่อนของกระแส | ไม่เกินร้อยละ 5 | [รอผล] | [รอผล] |
| ข้อ 3 ความหน่วงการแสดงสถานะ | ไม่เกิน 10 วินาที | [รอผล] | [รอผล] |
| ข้อ 4 แสดง N/A เมื่อไม่มีข้อมูล | เกิน 30 วินาที | [รอผล] | [รอผล] |
| ข้อ 5 Hysteresis ลดการเปลี่ยนสถานะผิด | น้อยกว่าเกณฑ์เดียว | [รอผล] | [รอผล] |
| ข้อ 6 แรงดันของแผงวงจร 3.3 โวลต์ | คลาดเคลื่อนไม่เกินร้อยละ 5 | [รอผล] | [รอผล] |
| ข้อ 7 ผู้ทดสอบทำภารกิจบนเว็บสำเร็จ | ไม่น้อยกว่าร้อยละ 80 | [รอผล] | [รอผล] |

การตัดสินผลทำตามหลักต่อไปนี้ สมมติฐานข้อที่มีเกณฑ์เป็นตัวเลข เช่น ข้อ 1 2 3 6 และ 7 ถือว่าเป็นจริงเมื่อค่าที่วัดได้เป็นไปตามเกณฑ์ในตาราง สมมติฐานข้อ 4 เป็นการตรวจพฤติกรรมของหน้าเว็บ จึงบันทึกว่าแสดง N/A เมื่อเกิน 30 วินาทีหรือไม่ และกลับมาแสดงสถานะปกติเมื่อได้รับข้อมูลใหม่หรือไม่ ส่วนข้อ 5 เทียบจำนวนครั้งที่สถานะเปลี่ยนผิดพลาดของเกณฑ์สองระดับกับเกณฑ์เดียวจากข้อมูลชุดเดียวกัน

หากผลข้อใดไม่เป็นไปตามเกณฑ์ ผู้จัดทำจะรายงานว่าไม่เป็นไปตามสมมติฐานพร้อมสาเหตุที่ตรวจสอบได้ เช่น โหลดที่มีกระแสต่ำกว่าเกณฑ์ ความหน่วงของเครือข่าย หรือการต่อวงจรที่ไม่เหมาะสม และจะไม่ปรับเกณฑ์หลังเห็นผลเพื่อให้ผ่าน เพราะการปรับเช่นนั้นทำให้ผลทดสอบไม่น่าเชื่อถือ ผลที่ไม่ผ่านจึงเป็นข้อมูลที่มีประโยชน์ต่อการปรับปรุงระบบในบทที่ 5

## 13. แนวทางตรวจสอบและติดตามที่มาของข้อมูล

ตัวเลขทุกค่าในตารางของบทนี้ต้องย้อนกลับไปหาข้อมูลต้นฉบับได้ ผู้จัดทำจึงกำหนดเส้นทางของข้อมูลตามภาพที่ 19 เริ่มจากทำการทดลองตามขั้นตอนในบทที่ 3 บันทึกค่าลงสเปรดชีตบันทึกผลทันทีที่วัดได้ และนำออกข้อมูลจากหน้า admin ในรูปแบบไฟล์ CSV ในกรณีที่เป็นข้อมูลที่ระบบบันทึกไว้ จากนั้นจึงตรวจสอบก่อนคำนวณ




**ภาพที่ 19 เส้นทางของข้อมูลตั้งแต่การทดลองจนถึงการรายงานผล**

**ที่มา : ผู้จัดทำ**

การตรวจสอบข้อมูลก่อนคำนวณมี 3 เรื่อง คือ หน่วยของค่าต้องตรงกัน เช่น มิลลิแอมแปร์กับแอมแปร์ ค่าที่ผิดปกติต้องหาสาเหตุก่อนตัดสินใจว่าจะใช้หรือไม่ และรอบที่ขาดหายต้องบันทึกเหตุผล เช่น Wi-Fi หลุดหรือสายหลุด หากข้อมูลไม่ครบตามจำนวนรอบที่กำหนดจะทดลองซ้ำ ไม่ใช้วิธีเติมตัวเลขหรือประมาณค่า ค่าผิดปกติที่ตัดออกต้องระบุในหมายเหตุของตารางพร้อมเหตุผล

## 14. ข้อควรระวังในการอ่านและตีความผล

ผลการทดลองในโครงงานนี้เป็นการวัดจำนวนครั้งจำกัดกับอุปกรณ์จำนวนน้อย การอ่านผลจึงควรคำนึงถึงเรื่องต่อไปนี้

14.1) ค่าเฉลี่ยอย่างเดียวไม่พอ ต้องดูส่วนเบี่ยงเบนมาตรฐานและค่าสูงสุดควบคู่ เพราะค่าเฉลี่ยความหน่วงที่ดีอาจมีบางครั้งที่ช้ามาก ซึ่งเป็นสิ่งที่ผู้ใช้สังเกตเห็นได้จริง จึงรายงานค่า P95 ด้วย

14.2) ความคลาดเคลื่อนร้อยละไม่ควรคำนวณจากค่าอ้างอิงที่ใกล้ศูนย์ เพราะค่าต่างเพียงเล็กน้อยทำให้ร้อยละสูงเกินจริง ให้ดูค่าต่างเป็นมิลลิแอมแปร์ควบคู่กับความละเอียดของ INA219 และของมัลติมิเตอร์

14.3) ผลของโหลดชนิดหนึ่งไม่สามารถสรุปแทนโหลดทุกชนิดได้ โดยเฉพาะโหลดที่กระแสแกว่ง เช่น มอเตอร์ ซึ่งต้องเทียบกับช่วงกระแสของเกณฑ์ที่ตั้ง

14.4) ความหน่วงที่วัดได้ขึ้นกับเครือข่ายและบริการคลาวด์ในช่วงเวลาที่ทดสอบ ควรบันทึกเวลาและสภาพเครือข่ายของแต่ละรอบ และไม่นำไปสรุปว่าเป็นค่าตายตัวของระบบ

14.5) ผลการทดสอบกับผู้ใช้จำนวนน้อยบอกได้เพียงแนวโน้มว่าหน้าเว็บใช้งานได้หรือไม่ ไม่สามารถอ้างเป็นผลของผู้ใช้ทั่วไปได้

# บทที่ 5

# สรุปผล อภิปรายผลและข้อเสนอแนะ

## 1. สรุปผลการวิจัย

**1.1 ผลการพัฒนา** ผู้จัดทำพัฒนาระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้ากระแสตรงได้ตามวัตถุประสงค์ ประกอบด้วย ESP32 ที่อ่านค่า INA219 สองตัวและตัดสินสถานะด้วย Hysteresis ส่วนหลังบ้านบน Cloudflare Worker และ D1 ที่ตรวจ Bearer token ก่อนบันทึก หน้าเว็บที่แสดงสถานะและแสดง N/A เมื่อข้อมูลเก่าเกิน 30 วินาที และข้อความสรุปประจำวันทาง LINE

**1.2 ผลการทดสอบ** [สรุปตัวเลขจริงจากบทที่ 4 เทียบกับสมมติฐานทั้ง 7 ข้อ เช่น “ความถูกต้องของการตรวจจับร้อยละ ..., ความคลาดเคลื่อนของกระแสไม่เกินร้อยละ ..., ความหน่วงเฉลี่ย ... วินาที” และระบุว่าสมมติฐานข้อใดเป็นจริงหรือไม่]

ผลการทดสอบเทียบกับสมมติฐาน 7 ข้อ สรุปรูปแบบการรายงานได้ดังนี้ (กรอกตัวเลขจริงจากบทที่ 4)

สมมติฐานข้อ 1 ความถูกต้องของการตรวจจับสถานะไม่น้อยกว่าร้อยละ 90 ผลที่ได้ [รอผล] สรุปว่า [เป็นจริง / ไม่เป็นจริง / เป็นจริงบางโหลด]

สมมติฐานข้อ 2 ความคลาดเคลื่อนของกระแสไม่เกินร้อยละ 5 ผลที่ได้ [รอผล] สรุปว่า [เป็นจริง / ไม่เป็นจริง / เป็นจริงเฉพาะบางช่วงกระแส]

สมมติฐานข้อ 3 ความหน่วงในการแสดงสถานะไม่เกิน 10 วินาที ผลที่ได้ [รอผล] สรุปว่า [เป็นจริง / ไม่เป็นจริง]

สมมติฐานข้อ 4 เมื่อไม่ได้รับข้อมูลเกิน 30 วินาทีหน้าเว็บแสดง N/A ผลที่ได้ [รอผล] สรุปว่า [เป็นจริง / ไม่เป็นจริง]

สมมติฐานข้อ 5 ถึง 7 ได้แก่ ข้อความสรุปทาง LINE แผงวงจรที่ออกแบบเอง และการเปรียบเทียบ Hysteresis กับเกณฑ์เดียว รวมถึงการใช้งานหน้าเว็บ ผลที่ได้ [รอผล] และสรุปเป็นรายข้อในลักษณะเดียวกัน

ผู้จัดทำจะสรุปตามผลที่ได้จริง หากสมมติฐานข้อใดไม่เป็นจริงจะระบุเป็นข้อค้นพบของโครงงาน ไม่ถือว่าเป็นความล้มเหลว เพราะสมมติฐานถูกตั้งไว้เพื่อให้ตรวจสอบได้ด้วยการวัดจริง

## 2. การอภิปรายผล

การตัดสินสถานะจากกระแสไฟฟ้าจริงช่วยให้ทราบว่าอุปกรณ์ทำงานอยู่จริง ไม่ใช่เพียงมีไฟจ่ายถึง การใช้ Hysteresis ช่วยป้องกันสถานะกระพริบเมื่อกระแสผันผวน ส่วนการแสดง N/A เมื่อไม่มีข้อมูลทำให้ผู้ใช้ไม่เข้าใจผิดว่าอุปกรณ์ปิด เมื่อจริง ๆ แล้วเป็นเพียงอุปกรณ์วัดขาดการเชื่อมต่อ ซึ่งเป็นการออกแบบให้ล้มเหลวอย่างปลอดภัย (fail-safe) นอกจากนี้ การบังคับให้ POST ต้องมี token ที่ถูกต้อง ช่วยป้องกันผู้อื่นส่งข้อมูลปลอมเข้าระบบ

ประเด็นแรกที่ควรอภิปรายคือการออกแบบให้แยกสถานะ OFF ออกจาก N/A และ FAULT เมื่ออ่านค่าไม่ได้ ระบบไม่เดาว่าอุปกรณ์ปิด แต่รายงานว่าไม่ทราบสถานะ วิธีนี้แลกกับการที่หน้าเว็บอาจแสดง N/A บ่อยขึ้นเมื่อเครือข่ายไม่เสถียร ซึ่งผู้จัดทำเห็นว่าสมเหตุสมผลกว่าการแสดงสถานะผิด โดยเฉพาะกับการแจ้งเตือนว่าอุปกรณ์ปิดอยู่ทั้งที่จริงยังทำงานอยู่

ประเด็นที่สองคือการเลือกใช้ Hysteresis ผลของการทดลองที่ 8 จะแสดงว่าเกณฑ์สองระดับลดการเปลี่ยนสถานะที่ไม่จำเป็นได้เพียงใดกับโหลดที่ทดสอบ ข้อแลกเปลี่ยนคือเมื่อกระแสของโหลดอยู่ในช่วงกลางระหว่างสองเกณฑ์ ระบบจะคงสถานะเดิม จึงมีโอกาสที่การลดกระแสลงช้า ๆ จะถูกตรวจจับว่าปิดช้ากว่าความเป็นจริงเล็กน้อย และต้องเลือกเกณฑ์ให้เหมาะกับโหลดแต่ละชนิดตามผลการทดลองที่ 1

ประเด็นที่สามคือความน่าเชื่อถือของการส่งข้อมูล การใช้ Offline Buffer หมายเลขลำดับ และการลองส่งซ้ำแบบถอยหลังเป็นทวีคูณ ช่วยให้ข้อมูลไม่สูญหายเมื่อเครือข่ายขัดข้องชั่วคราว ผลการทดลองที่ 5 จะบอกว่าในการใช้งานจริง 24 ชั่วโมง กลไกนี้ลดช่วงข้อมูลขาดหายได้เพียงใด ส่วนข้อจำกัดคือบัฟเฟอร์อยู่ใน RAM และความจุจำกัด จึงรองรับได้เฉพาะการขัดข้องช่วงสั้น

ประเด็นที่สี่คือความหน่วง ความหน่วงที่ผู้ใช้เห็นมาจากการออกแบบแบบ Polling ซึ่งเลือกเพราะทำงานบนเว็บไซต์แบบสถิตได้ ถ้าต้องการลดความหน่วงลงอีกต้องเปลี่ยนไปใช้การเชื่อมต่อแบบค้าง เช่น WebSocket ซึ่งต้องมีส่วนบริการที่ซับซ้อนกว่า ผู้จัดทำเห็นว่าสำหรับการดูสถานะอุปกรณ์ไฟฟ้า ความหน่วงในระดับหลายวินาทียังยอมรับได้ แต่ต้องยืนยันด้วยผลการทดลองที่ 4

[อภิปรายผลการทดลองจริง เช่น เหตุที่ความคลาดเคลื่อนสูงหรือต่ำในช่วงกระแสต่ำ เปรียบเทียบกับสมมติฐาน และสาเหตุของข้อมูลขาดหาย]

ประเด็นสุดท้ายคือความปลอดภัยของระบบ การกำหนดให้ POST ต้องมี Bearer token การเก็บ token เป็นความลับของ Worker และการตรวจข้อมูลก่อนบันทึก ช่วยป้องกันการส่งข้อมูลมั่วเข้าระบบ แต่เนื่องจาก token เป็นความลับร่วมที่ฝังอยู่ในเฟิร์มแวร์ หากบอร์ดถูกนำไปอ่านค่าจากหน่วยความจำ token อาจรั่วไหลได้ ในการใช้งานจริงควรเปลี่ยน token เป็นระยะและจำกัดสิทธิ์ของ token

เมื่อเทียบกับงานวิจัยที่เกี่ยวข้อง ระบบนี้คล้ายกันในแนวคิดการใช้ IoT รวบรวมข้อมูลและแสดงผลผ่านเว็บ แต่ต่างกันที่ผู้จัดทำใช้การวัดกระแสเพื่อจำแนกสถานะของโหลดโดยตรง ใช้บริการคลาวด์แบบ serverless ที่ไม่ต้องดูแลเซิร์ฟเวอร์ และรายงานผลทดลองเทียบสมมติฐานที่ตรวจสอบได้ ข้อจำกัดคือขนาดของการทดลองยังเล็กกว่างานวิจัยดังกล่าว

### 2.1 ข้อจำกัดของการวิจัย

ข้อจำกัดของโครงงานมี 4 ประการ คือ ทดสอบกับโหลดกระแสตรงแรงดันต่ำเพียงไม่กี่ชนิด ใช้ผู้ทดสอบ 10 คนที่เลือกแบบเจาะจง ใช้มัลติมิเตอร์เครื่องเดียวเป็นค่าอ้างอิงซึ่งมีความคลาดเคลื่อนของตัวเอง และทดสอบความเสถียรต่อเนื่อง 24 ชั่วโมงเท่านั้น จึงไม่ครอบคลุมการทำงานระยะยาวเป็นสัปดาห์หรือเดือน ผู้อ่านจึงควรตีความผลเป็นหลักฐานว่าแนวทางนี้ใช้งานได้ในขอบเขตที่ทดสอบ ไม่ใช่การรับรองสำหรับอุปกรณ์ทุกชนิด

## 3. ข้อเสนอแนะ

1. ปรับเกณฑ์ ON/OFF ให้เหมาะกับอุปกรณ์แต่ละชนิด เช่น เก็บค่าเกณฑ์แยกรายอุปกรณ์

2. เพิ่มการแจ้งเตือนทันทีเมื่อสถานะเปลี่ยนหรืออุปกรณ์เปิดค้างนานเกินกำหนด โดยคำนึงถึงข้อจำกัดจำนวนข้อความของ LINE

3. หากต้องการขยายไปใช้กับไฟบ้าน AC ต้องใช้เซนเซอร์และโมดูลที่แยกวงจรและผ่านการรับรองความปลอดภัย ภายใต้การดูแลของผู้เชี่ยวชาญ ไม่ควรทดลองด้วยตนเอง

4. ทำกล่องหุ้มอุปกรณ์และเพิ่มระบบสำรองไฟเพื่อให้ใช้งานได้ต่อเนื่อง

5. เพิ่มความทนทานของระบบ เช่น ให้ ESP32 ส่งซ้ำพร้อมหน่วงเวลาเมื่อส่งไม่สำเร็จ ป้องกันการส่งสรุปทางไลน์ซ้ำ และป้องกันการส่งข้อมูลเก่าซ้ำ (Replay Protection)

6. ขยายระบบให้รองรับหลายอุปกรณ์และทดสอบภาระการเขียนฐานข้อมูล D1 พร้อมปรับหน้าเว็บให้รองรับมือถือและการเข้าถึง (Accessibility) มากขึ้น

7. เพิ่มการบันทึกข้อมูลเมื่อสถานะเปลี่ยน (Event Log) แยกจากข้อมูลวัดรายรอบ เพื่อให้ตอบคำถามเช่น เปิดใช้งานกี่ครั้งและแต่ละครั้งนานเท่าใดได้โดยไม่ต้องคำนวณจากประวัติทั้งหมด

8. ใช้หน่วยความจำถาวรของ ESP32 (เช่น LittleFS) เก็บ Offline Buffer เพื่อให้ข้อมูลที่ค้างไม่หายเมื่อบอร์ดรีเซ็ต พร้อมทดสอบจำนวนครั้งที่เขียนเพื่อไม่ให้หน่วยความจำเสื่อมเร็ว

9. เปลี่ยนวิธีพิสูจน์ตัวตนของอุปกรณ์จาก token ร่วมเป็น token แยกต่ออุปกรณ์ หรือการลงนามข้อมูล (Signature) เพื่อลดความเสี่ยงเมื่อ token รั่วไหล

10. ทำการทดลองกับโหลดชนิดอื่นและช่วงเวลานานขึ้น รวมถึงทดสอบกับผู้ใช้กลุ่มอื่นที่หลากหลายขึ้น เพื่อยืนยันว่าผลที่ได้ใช้ได้กว้างกว่ากลุ่มตัวอย่างเดิม

# บรรณานุกรม


Adafruit Industries. (n.d.). Adafruit INA219 current sensor breakout. สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://learn.adafruit.com/adafruit-ina219-current-sensor-breakout

Cloudflare. (n.d.-a). Cron Triggers. Cloudflare Workers documentation. สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://developers.cloudflare.com/workers/configuration/cron-triggers/

Cloudflare. (n.d.-b). Cloudflare D1. Cloudflare Developer documentation. สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://developers.cloudflare.com/d1/

Espressif Systems. (n.d.). ESP32 series datasheet. สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

GitHub. (n.d.). GitHub Pages documentation. สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://docs.github.com/en/pages

LINE Corporation. (n.d.). Messaging API overview. LINE Developers. สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://developers.line.biz/en/docs/messaging-api/overview/

NXP Semiconductors. (2021). I2C-bus specification and user manual (UM10204, Rev. 7). สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Texas Instruments. (n.d.). INA219 zero-drift, bidirectional current/power monitor with I2C interface (Datasheet). สืบค้นเมื่อ 31 ตุลาคม 2569, จาก https://www.ti.com/lit/ds/symlink/ina219.pdf

วิโรจน์ ขนเกษม. (2566) การพัฒนาระบบติดตามและแจ้งเตือนสถานะการทำงานของเครื่องอัดขึ้นรูปพิมพ์ร้อนด้วยเทคโนโลยีอินเทอร์เน็ตของสรรพสิ่ง [วิทยานิพนธ์, มหาวิทยาลัยมหาสารคาม]. สืบค้นเมื่อ 2 พฤษจิกายน 2569 http://202.28.34.124/dspace/bitstream/123456789/1609/1/63010381010.pdf

พิมลดา ค้ำดินพิทักษ์ และคณะ (2567) UPS Monitoring and Management System สืบค้นเมื่อ 2 พฤษจิกายน 2569 https://ph03.tci-thaijo.org/index.php/JEIT/article/view/2142

Espressif Systems. (n.d.). ESP32 series datasheet. สืบค้นเมื่อ 2 พฤษจิกายน 2569 https://documentation.espressif.com/esp32_datasheet_en.pdf

Cloudflare. (n.d.-a). Cloudflare Workers documentation. สืบค้นเมื่อ 2 พฤษจิกายน 2569 https://developers.cloudflare.com/workers/

Cloudflare. (n.d.-b). Cloudflare D1 documentation. สืบค้นเมื่อ 2 พฤษจิกายน 2569 https://developers.cloudflare.com/d1/get-started/

LINE Corporation. (n.d.). LINE Messaging API documentation. สืบค้นเมื่อ 2 พฤษจิกายน 2569 https://developers.line.biz/en/docs/messaging-api/

# ภาคผนวก ก

# เว็บไซต์และซอร์สโค้ด

**ซอร์สโค้ด:** https://github.com/dentenplay5555/electrical-status-notification-system

**เว็บไซต์โครงงาน:** https://dentenplay5555.github.io/electrical-status-notification-system/

**หน้า Dashboard:** https://dentenplay5555.github.io/electrical-status-notification-system/dashboard.html

**โครงสร้างที่เก็บโค้ด:** firmware/ (โค้ด ESP32), backend/ (Cloudflare Worker และ schema ฐานข้อมูล), frontend/ (เว็บไซต์), hardware/ (ไฟล์ออกแบบ PCB), docs/ (รายงานและแผนภาพ)

## ก.1 ข้อมูลอ้างอิงของที่เก็บโค้ด

โค้ดทั้งหมดในภาคผนวก ง คัดมาจากที่เก็บโค้ดของโครงงาน ณ commit c19b0b8 โดยตัดคำอธิบายภาษาไทยในโค้ดออกและตัดบรรทัดที่ยาวเพื่อให้พอดีหน้ากระดาษ ผู้ที่ต้องการอ่านฉบับเต็มควรเปิดจากที่เก็บโค้ดโดยตรง

## ก.2 ตัวอย่างการเรียก API

ตัวอย่างต่อไปนี้ใช้เรียก Endpoint ของ Worker จากเครื่องคอมพิวเตอร์เพื่อทดสอบ โดยแทนที่ <WORKER_URL> ด้วยที่อยู่ Worker จริง และ <DEVICE_TOKEN> ด้วย token ที่ตั้งไว้ (ห้ามนำ token จริงไปใส่ในรายงานหรือที่เก็บโค้ด) ตรวจความพร้อมของระบบด้วย GET /health

```
curl https://<WORKER_URL>/health
```

คำตอบเมื่อฐานข้อมูลปกติเป็น JSON ที่มีค่า status เท่ากับ ok พร้อมรายละเอียดสถานะของ D1 และเวลาตอบสนอง ถ้าฐานข้อมูลใช้ไม่ได้จะได้รหัส 503 และ status เท่ากับ degraded การส่งข้อมูลทดสอบเข้าระบบทำได้ด้วยคำสั่งต่อไปนี้

```
curl -X POST https://<WORKER_URL>/update \
  -H "Authorization: Bearer <DEVICE_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"device_id":"ESP32-01","sensors":[
    {"sensor_id":"INA219-01","voltage_v":5.0,
     "current_ma":120.0,"power_mw":600.0,
     "is_on":true,"ok":true}]}'
```

อ่านสถานะล่าสุดด้วย GET /status โดยระบุ device_id คำตอบมีฟิลด์ status เป็น online เมื่อข้อมูลไม่เก่ากว่า 30 วินาที และเป็น N/A เมื่อเก่ากว่านั้นหรือยังไม่เคยมีข้อมูล พร้อมค่า is_stale และ stale_seconds ที่บอกอายุของข้อมูลล่าสุด

```
curl "https://<WORKER_URL>/status?device_id=ESP32-01"
```

รหัสสถานะที่ POST /update อาจตอบ ได้แก่ 200 รับข้อมูลแล้ว 400 ข้อมูลไม่ตรงรูปแบบหรือไม่ใช่ JSON 401 token ไม่ถูกต้อง 409 หมายเลขลำดับซ้ำหรือเก่ากว่าที่เคยรับในรอบการทำงานเดียวกัน 503 ฐานข้อมูลใช้ไม่ได้ และ 500 เกิดข้อผิดพลาดภายใน

# ภาคผนวก ข
# ขั้นตอนติดตั้งโดยสรุป

1. ตั้งค่าและอัปโหลดเฟิร์มแวร์ด้วย PlatformIO (คัดลอก config.example.h เป็น config.h แล้วใส่ Wi-Fi และ token)
2. สร้างฐานข้อมูล D1 และรัน schema บนฐานข้อมูลจริง ต้องใส่ --remote มิฉะนั้นจะเกิดข้อผิดพลาด no such table

   wrangler d1 execute <ชื่อฐานข้อมูล> --remote --file=schema.sql
3. ตั้งค่า token ให้ตรงกับ DEVICE_TOKEN ใน config.h แล้ว deploy

   wrangler secret put DEVICE_TOKEN

   wrangler deploy
4. push โค้ดขึ้นสาขา master ให้ GitHub Actions เผยแพร่เว็บไซต์ไปที่ GitHub Pages

### ข.1 สิ่งที่ต้องเตรียมก่อนติดตั้ง

1) ติดตั้ง PlatformIO (ส่วนขยายของ VS Code หรือโปรแกรมบรรทัดคำสั่ง) และไดรเวอร์ USB ของบอร์ด ESP32

2) ติดตั้ง Node.js เพื่อใช้ wrangler ซึ่งเป็นเครื่องมือของ Cloudflare และมีบัญชี Cloudflare ที่ล็อกอินด้วยคำสั่ง npx wrangler login

3) มีบัญชี GitHub และเปิดใช้ GitHub Pages ของที่เก็บโค้ดโดยเลือกแหล่งที่มาเป็น GitHub Actions

4) มีเครือข่าย Wi-Fi ย่าน 2.4 กิกะเฮิรตซ์ที่ ESP32 เชื่อมต่อได้

### ข.2 ลำดับการติดตั้งโดยละเอียด

ขั้นที่ 1 ต่อ INA219 กับ ESP32 โดย SDA ต่อ GPIO21 SCL ต่อ GPIO22 ไฟเลี้ยง 3.3 โวลต์ และกราวด์ร่วมกัน บัดกรีจุด A0 ของโมดูลตัวที่สองเท่านั้น แล้วรันโปรแกรมสแกน I2C ด้วยคำสั่งอัปโหลดโปรไฟล์ testing จนพบ 0x40 และ 0x41

```
pio run -e testing -t upload
pio device monitor
```

ขั้นที่ 2 คัดลอก config.example.h เป็น config.h ในโฟลเดอร์ main-src แล้วกรอกชื่อและรหัสผ่าน Wi-Fi ที่อยู่ Worker (API_ENDPOINT) และ DEVICE_TOKEN ห้ามนำไฟล์ config.h ขึ้นที่เก็บโค้ด

ขั้นที่ 3 สร้างฐานข้อมูล D1 และรัน schema บนฐานข้อมูลจริงด้วยตัวเลือก --remote ซึ่งขาดไม่ได้

```
npx wrangler d1 create appliance-db
npx wrangler d1 execute appliance-db --remote --file=schema.sql
```

ขั้นที่ 4 ตั้งความลับของ Worker แล้วเผยแพร่ ค่า DEVICE_TOKEN ต้องตรงกับที่ใส่ใน config.h

```
npx wrangler secret put DEVICE_TOKEN
npx wrangler deploy
```

ขั้นที่ 5 อัปโหลดเฟิร์มแวร์จริงด้วยโปรไฟล์ main แล้วเปิดตัวตรวจสอบอนุกรมเพื่อดูบันทึกการส่งข้อมูล

```
pio run -e main -t upload
pio device monitor
```

ขั้นที่ 6 ตั้งค่า API_URL ในหน้าเว็บให้ชี้ไปที่ Worker (แก้ใน script.js หรือกดปุ่มตั้งค่าบนหน้าเว็บ) แล้ว push ขึ้นสาขา master เพื่อให้ GitHub Actions เผยแพร่

**ข.3 การตรวจสอบหลังติดตั้ง**

ตรวจตามลำดับว่า 1) GET /health ตอบ 200 และ d1 เป็น connected 2) บันทึกทางพอร์ตอนุกรมแสดงว่าส่งสำเร็จ (HTTP 200) 3) GET /status ตอบ status เป็น online พร้อมข้อมูลของเซนเซอร์ทั้งสองตัว 4) หน้า Dashboard แสดงสถานะตรงกับโหลดจริง และ 5) ถอดไฟ ESP32 แล้วหน้าเว็บแสดง N/A ภายในประมาณ 30 วินาที

**ข.4 ปัญหาที่พบบ่อย**

Worker ตอบ error 1101 หรือ no such table หมายถึงยังไม่ได้รัน schema กับฐานข้อมูลจริง ให้รันซ้ำด้วย --remote

ESP32 ได้รหัส 401 หมายถึง DEVICE_TOKEN ใน config.h ไม่ตรงกับความลับบน Worker หรือยังไม่ได้ตั้งค่าบน Worker

สแกน I2C ไม่พบอุปกรณ์เลย ให้ตรวจสาย SDA และ SCL ว่าไม่ได้สลับกัน และตรวจไฟเลี้ยงกราวด์ หากพบตัวเดียวให้ตรวจการบัดกรีจุด A0

หน้าเว็บแสดง N/A ตลอด ให้ตรวจว่า API_URL ถูกต้อง ESP32 เชื่อมต่อ Wi-Fi ได้ และ GET /status ตอบข้อมูลที่ไม่เก่ากว่า 30 วินาที

# ภาคผนวก ค

# แผงวงจรและรายการอุปกรณ์

[แทรกภาพผังวงจร (Schematic) จาก KiCad]

[แทรกภาพเลย์เอาต์ PCB และภาพแผงวงจรที่สร้างเสร็จ]

[แทรกรายการอุปกรณ์ (BOM): ชื่ออุปกรณ์ จำนวน ราคา]

รายการอุปกรณ์หลักที่ใช้ในโครงงาน ได้แก่ บอร์ด ESP32 DevKit (ESP32-WROOM-32) 1 ตัว โมดูล INA219 2 ตัว แผงวงจรขยายที่ผู้จัดทำสร้างเอง 1 ชุด แหล่งจ่ายไฟกระแสตรงปรับค่าได้ สายจัมเปอร์และเบรดบอร์ด มัลติมิเตอร์ดิจิทัล และโหลดทดสอบ (เซอร์โว SG90 และ MG995 เป็นต้น) [กรอกจำนวนและราคาจริงที่ซื้อ]

ไฟล์ออกแบบผังวงจรและเลย์เอาต์ PCB (ไฟล์ Circut.kicad_sch และ Circut.kicad_pcb) อยู่ในโฟลเดอร์ hardware ของที่เก็บโค้ด และแผนภาพวงจรในรูปภาพอยู่ในโฟลเดอร์ docs/diagrams/circuit

# ภาคผนวก ง

# โค้ดสำคัญของระบบ

ภาคผนวกนี้แสดงส่วนของโค้ดที่เกี่ยวข้องโดยตรงกับผลการวิจัย ได้แก่ การตัดสินสถานะด้วย Hysteresis การส่งข้อมูลพร้อมลองซ้ำ การจัดการ Offline Buffer ในวงรอบหลัก การตรวจ token และรูปแบบข้อมูลที่ Worker การตรวจหมายเลขลำดับ และโครงสร้างฐานข้อมูล ตัดจากที่เก็บโค้ดตามที่ระบุในภาคผนวก ก

## ง.1 การอ่านเซนเซอร์และตัดสินสถานะ (firmware/main-src/sensor.cpp)

```
static bool applyHysteresis(float current_ma, bool previousIsOn) {
    float absCurrent = fabs(current_ma);
    if (absCurrent > CURRENT_ON_THRESHOLD_MA) return true;
    if (absCurrent < CURRENT_OFF_THRESHOLD_MA) return false;
    return previousIsOn;
}
```

ฟังก์ชัน applyHysteresis ใช้ค่าสัมบูรณ์ของกระแส ตัดสินเป็น ON เมื่อมากกว่าเกณฑ์เปิด เป็น OFF เมื่อน้อยกว่าเกณฑ์ปิด และคงสถานะเดิมในช่วงกลาง ส่วนฟังก์ชัน readOne ตรวจ ACK ก่อนอ่านค่า

```
static SensorReading readOne(Adafruit_INA219 &sensor, const char
    *sensorId, uint8_t address, bool &lastIsOn) {
    SensorReading r;
    r.sensor_id = sensorId;

    if (!isDeviceResponding(address)) {
        r.voltage_v = 0.0f;
        r.current_ma = 0.0f;
        r.power_mw = 0.0f;
        r.is_on = false;
        r.ok = false;
        return r;
    }

    float v = sensor.getBusVoltage_V();
    float i = sensor.getCurrent_mA();
    float p = sensor.getPower_mW();

    if (isnan(v) || isnan(i) || isnan(p)) {
        r.voltage_v = 0.0f;
        r.current_ma = 0.0f;
        r.power_mw = 0.0f;
        r.is_on = false;
        r.ok = false;
        return r;
    }

    r.voltage_v = v;
    r.current_ma = i;
    r.power_mw = p;
```

```
    r.is_on = applyHysteresis(r.current_ma, lastIsOn);
    r.ok = true;
    lastIsOn = r.is_on;
    return r;
}

SensorReading readSensor1() {
```

### ง.2 การส่งข้อมูลพร้อมลองซ้ำแบบถอยหลังเป็นทวีคูณ

(firmware/main-src/api_client.cpp)

```
bool api_sendReadings(const SensorReading &reading1, const
    SensorReading &reading2, uint32_t sequenceNumber, const char
    *bootId) {
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID;
    if (bootId && strlen(bootId) > 0) {
        doc["boot_id"] = bootId;
    }
    doc["sequence_number"] = sequenceNumber;
    JsonArray sensorsArray = doc["sensors"].to<JsonArray>();
    addSensorToDoc(sensorsArray, reading1);
    addSensorToDoc(sensorsArray, reading2);
    String payload;
    serializeJson(doc, payload);

    unsigned long backoffMs = API_INITIAL_BACKOFF_MS;

    for (int attempt = 1; attempt <= API_MAX_RETRIES; attempt++) {
        HTTPClient http;
        http.begin(API_ENDPOINT);
        http.addHeader("Content-Type", "application/json");
        http.addHeader("Authorization", "Bearer " DEVICE_TOKEN);
        http.setTimeout(5000);

        int httpCode = http.POST(payload);
        bool ok = httpCode >= 200 && httpCode < 300;
        http.end();

        if (ok) {
            if (attempt > 1) {
                Serial.printf("[api] %d (HTTP %d)\n", attempt,
                    httpCode);
            } else {
                Serial.printf("[api] (HTTP %d, seq=%lu)\n",
                    httpCode, (unsigned long)sequenceNumber);
            }
            return true;
        }

        if (httpCode >= 400 && httpCode < 500 && httpCode != 429) {
            Serial.printf("[api] (HTTP %d) — client error,
                retry\n", httpCode);
            return false;
        }

        if (attempt < API_MAX_RETRIES) {
            Serial.printf("[api] (HTTP %d) — retry %d/%d %lu ms\n",
                          httpCode, attempt, API_MAX_RETRIES,
                              backoffMs);
            delay(backoffMs);
            backoffMs = min(backoffMs * 2, (unsigned
                long)API_MAX_BACKOFF_MS);
        } else {
            Serial.printf("[api] (HTTP %d) — retry (%d )\n",
```

```
                            httpCode, API_MAX_RETRIES);
        }
    }
    return false;
}
```

### ง.3 Offline Buffer ในเฟิร์มแวร์ (firmware/main-src/main.cpp)

```
struct BufferedReading {
    SensorReading r1;
    SensorReading r2;
    uint32_t seq;
};

static BufferedReading offlineBuffer[OFFLINE_BUFFER_CAPACITY];
static int bufferHead = 0;
static int bufferCount = 0;

static void bufferPush(const SensorReading &r1, const SensorReading
    &r2, uint32_t seq) {
    if (bufferCount >= OFFLINE_BUFFER_CAPACITY) {
        bufferHead = (bufferHead + 1) % OFFLINE_BUFFER_CAPACITY;
        bufferCount--;
        Serial.println("[buffer] Offline buffer : ");
    }
    int tail = (bufferHead + bufferCount) % OFFLINE_BUFFER_CAPACITY;
    offlineBuffer[tail] = {r1, r2, seq};
    bufferCount++;
    Serial.printf("[buffer] Offline Buffer (: %d , seq=%lu)\n",
        bufferCount, (unsigned long)seq);
}

static bool drainOfflineBuffer() {
    int flushed = 0;
    while (bufferCount > 0 && flushed < 5) {
        BufferedReading &item = offlineBuffer[bufferHead];

        Serial.printf("[recovery] Offline Buffer (seq=%lu)...\n",
            (unsigned long)item.seq);
        bool ok = api_sendReadings(item.r1, item.r2, item.seq, bootId);
        if (!ok) {
            Serial.println("[recovery] — buffer ");
            return false;
        }

        bufferHead = (bufferHead + 1) % OFFLINE_BUFFER_CAPACITY;
        bufferCount--;
        flushed++;
        delay(150);
    }
    if (flushed > 0) {
        Serial.printf("[recovery] %d (: %d )\n", flushed,
            bufferCount);
    }
```

### ง.4 การตรวจ token และรูปแบบข้อมูลที่ Worker (backend/src/routes/update.ts)

```
async function checkBearerToken(request: Request, env: Env):
    Promise<boolean> {
    if (env.ALLOW_NO_AUTH === "true") {
        return true;
    }

    if (!env.DEVICE_TOKEN) {
        console.error(
            "[auth] DEVICE_TOKEN is not configured on Worker (fail-
                closed). " +
            "Run 'npx wrangler secret put DEVICE_TOKEN' on
                production, " +
            "or set ALLOW_NO_AUTH=true in .dev.vars for local
                development."
        );
        return false;
    }

    const auth = request.headers.get("Authorization") ?? "";
    if (!auth.startsWith("Bearer ")) {
        return false;
    }

    const token = auth.slice(7).trim();
    if (!token) return false;

    return safeEqual(token, env.DEVICE_TOKEN);
}

function isValidSensor(s: unknown): s is SensorReading {
    if (typeof s !== "object" || s === null) return false;
    const obj = s as Record<string, unknown>;
    return (
        typeof obj.sensor_id === "string" &&
        obj.sensor_id.trim().length > 0 &&
        typeof obj.voltage_v === "number" &&
        Number.isFinite(obj.voltage_v) &&
        typeof obj.current_ma === "number" &&
        Number.isFinite(obj.current_ma) &&
        typeof obj.power_mw === "number" &&
        Number.isFinite(obj.power_mw) &&
        typeof obj.is_on === "boolean" &&
        typeof obj.ok === "boolean"
    );
}

function isValidPayload(body: unknown): body is DeviceUpdate {
    if (typeof body !== "object" || body === null) return false;
    const obj = body as Record<string, unknown>;
    if (typeof obj.device_id !== "string" ||
        obj.device_id.trim().length === 0) return false;
    if (obj.boot_id !== undefined && (typeof obj.boot_id !==
        "string" || obj.boot_id.trim().length === 0)) {
        return false;
    }
    if (!Array.isArray(obj.sensors) || obj.sensors.length === 0)
        return false;
    if (obj.sequence_number !== undefined) {
        if (typeof obj.sequence_number !== "number" ||
            !Number.isInteger(obj.sequence_number) ||
            obj.sequence_number < 0) {
            return false;
        }
    }
```

```
    return obj.sensors.every(isValidSensor);
}
```

### ง.5 การตรวจหมายเลขลำดับและ boot_id (backend/src/lib/db.ts)

```
export async function checkAndUpdateSequence(
    env: Env,
    deviceId: string,
    newSeq: number,
    bootId?: string
): Promise<{ allowed: boolean; lastSeq: number; isNewSession?:
    boolean }> {
    const now = new Date().toISOString();

    const row = await env.DB.prepare(
        `SELECT boot_id, last_sequence FROM device_sequence WHERE
            device_id = ?`
    )
        .bind(deviceId)
        .first<{ boot_id: string | null; last_sequence: number }>();

    if (row) {
        if (bootId && row.boot_id !== bootId) {
            await env.DB.prepare(
                `UPDATE device_sequence SET boot_id = ?,
                    last_sequence = ?, updated_at = ? WHERE device_id =
                    ?`
            )
                .bind(bootId, newSeq, now, deviceId)
                .run();

            return { allowed: true, lastSeq: 0, isNewSession: true };
        }

        const lastSeq = row.last_sequence ?? 0;

        if (newSeq <= lastSeq) {
            return { allowed: false, lastSeq, isNewSession: false };
        }

        await env.DB.prepare(
            `UPDATE device_sequence SET last_sequence = ?,
                updated_at = ? WHERE device_id = ?`
        )
            .bind(newSeq, now, deviceId)
            .run();

        return { allowed: true, lastSeq, isNewSession: false };
    }
```

### ง.6 โครงสร้างฐานข้อมูล (backend/schema.sql)

```
CREATE TABLE IF NOT EXISTS devices (
    device_id TEXT PRIMARY KEY,
    name TEXT,
    last_seen TEXT
);
CREATE TABLE IF NOT EXISTS sensors (
    sensor_id TEXT PRIMARY KEY,
    device_id TEXT NOT NULL REFERENCES devices(device_id),
    name TEXT
);
CREATE TABLE IF NOT EXISTS readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id TEXT NOT NULL,
    sensor_id TEXT NOT NULL,
    voltage_v REAL,
    current_ma REAL,
    power_mw REAL,
    is_on INTEGER NOT NULL,
    ok INTEGER NOT NULL,
    created_at TEXT NOT NULL -- ISO timestamp (UTC)
);
CREATE INDEX IF NOT EXISTS idx_readings_device_time ON readings
    (device_id, created_at);
CREATE INDEX IF NOT EXISTS idx_readings_sensor_time ON readings
    (sensor_id, created_at);
CREATE TABLE IF NOT EXISTS latest_status (
    device_id TEXT PRIMARY KEY,
    sensors_json TEXT NOT NULL,
    updated_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS device_sequence (
    device_id TEXT PRIMARY KEY,
    boot_id TEXT,
    last_sequence INTEGER NOT NULL DEFAULT 0,
    updated_at TEXT NOT NULL
);
```

# ภาคผนวก จ

# แบบบันทึกและแบบสอบถามที่ใช้ในการทดลอง

ภาคผนวกนี้เป็นแบบฟอร์มที่ใช้เก็บข้อมูลในการทดลองที่ 9 และแบบบันทึกข้อมูลทั่วไป ใช้กระดาษพิมพ์ให้ผู้ทดสอบกรอกแล้วนำคะแนนไปคำนวณในไฟล์ตารางคำนวณ

**จ.1 แบบบันทึกการทดสอบการใช้งานหน้าเว็บไซต์ (การทดลองที่ 9)**

ผู้ทดสอบคนที่ ..... วันที่ ..... เวลา ..... (ไม่ต้องระบุชื่อ)

ภารกิจที่ 1 บอกสถานะของอุปกรณ์แต่ละตัวที่แสดงบนหน้าเว็บ ทำสำเร็จภายใน 60 วินาที โดยไม่ต้องช่วย ( ) สำเร็จ ( ) ไม่สำเร็จ เวลาที่ใช้ ..... วินาที

ภารกิจที่ 2 อธิบายความหมายของ N/A ทำสำเร็จภายใน 60 วินาทีโดยไม่ต้องช่วย ( ) สำเร็จ ( ) ไม่สำเร็จ คำตอบของผู้ทดสอบ: .....

หมายเหตุของผู้สังเกต: .....

**จ.2 แบบสอบถามความพึงพอใจ (มาตราส่วน 5 ระดับ)**

คำชี้แจง ให้วงกลมคะแนนที่ตรงกับความรู้สึกของท่านมากที่สุด โดย 1 หมายถึงน้อยที่สุด 2 น้อย 3 ปานกลาง 4 มาก และ 5 มากที่สุด

1) หน้าเว็บอ่านง่ายและดูสถานะได้ชัดเจน 1 2 3 4 5

2) ข้อความและสีที่ใช้แสดง ON, OFF และ N/A เข้าใจง่าย 1 2 3 4 5

3) ข้อมูลกระแส แรงดัน และกำลังไฟฟ้าที่แสดงเป็นประโยชน์ 1 2 3 4 5

4) ความเร็วในการอัปเดตสถานะอยู่ในระดับที่ยอมรับได้ 1 2 3 4 5

5) โดยรวม ท่านพึงพอใจกับการใช้งานหน้าเว็บนี้ 1 2 3 4 5

ข้อเสนอแนะเพิ่มเติม: .....

**จ.3 แบบบันทึกข้อมูลทั่วไปของการทดลอง**

วันที่และเวลา: ..... ผู้บันทึก: .....

การทดลองที่: ..... ชนิดและรุ่นของโหลด: ..... แรงดันแหล่งจ่าย: .....
โวลต์

เครือข่าย Wi-Fi ที่ใช้: ..... เวอร์ชันเฟิร์มแวร์ (commit): .....

เกณฑ์ที่ตั้ง: ON ..... mA OFF ..... mA มัลติมิเตอร์รุ่น: ..... ย่านวัด: .....

หลักฐาน (ชื่อไฟล์ภาพหรือวิดีโอในโฟลเดอร์
docs/evidence): .....

เหตุการณ์ผิดปกติระหว่างทดลอง: .....

# ประวัติผู้จัดทำ

| | |
|---|---|
| รูปถ่าย<br>(ขนาด 1.5 นิ้ว) | ชื่อ-สกุล : เด็กชายกฤตภาส ใจใหญ่<br>ชั้น ม.3/1 เลขที่ 1<br>ที่อยู่ : [กรอก]<br>E-mail : [กรอก]<br>หน้าที่ในโครงงาน : [กรอก] |

| | |
|---|---|
| รูปถ่าย<br>(ขนาด 1.5 นิ้ว) | ชื่อ-สกุล : เด็กชายกิตติศักดิ์ วงษา<br>ชั้น ม.3/1 เลขที่ 4<br>ที่อยู่ : [กรอก]<br>E-mail : [กรอก]<br>หน้าที่ในโครงงาน : [กรอก] |

| | |
|---|---|
| รูปถ่าย<br>(ขนาด 1.5 นิ้ว) | ชื่อ-สกุล : เด็กชายปกรณ์เกียรติ แหวนแก้ว<br>ชั้น ม.3/1 เลขที่ 15<br>ที่อยู่ : [กรอก]<br>E-mail : [กรอก]<br>หน้าที่ในโครงงาน : ทุกอย่างเพราะให้คนอื่นทำแล้วขก. อธิบาย |

| | |
|---|---|
| รูปถ่าย<br>(ขนาด 1.5 นิ้ว) | ชื่อ-สกุล : เด็กชายวรัญชัย พุทธวงค์<br>ชั้น ม.3/1 เลขที่ 19<br>ที่อยู่ : [กรอก]<br>E-mail : [กรอก]<br>หน้าที่ในโครงงาน : [กรอก] |

| | |
|---|---|
| รูปถ่าย<br>(ขนาด 1.5 นิ้ว) | ชื่อ-สกุล : เด็กชายวิชญะ เสาไธสง<br>ชั้น ม.3/1 เลขที่ 20<br>ที่อยู่ : [กรอก]<br>E-mail : [กรอก]<br>หน้าที่ในโครงงาน : [กรอก] |

| | |
|---|---|
| รูปถ่าย<br>(ขนาด 1.5 นิ้ว) | ชื่อ-สกุล : เด็กชายอชิระ ลิ้มสุวรรณ<br>ชั้น ม.3/1 เลขที่ 21<br>ที่อยู่ : [กรอก]<br>E-mail : [กรอก]<br>หน้าที่ในโครงงาน : [กรอก] |